



(19) 대한민국특허청(KR)
(12) 등록특허공보(B1)

(45) 공고일자 2012년02월28일
(11) 등록번호 10-1121606
(24) 등록일자 2012년01월29일

(51) Int. Cl.
G06F 9/30 (2006.01) G06F 9/38 (2006.01)
G06F 9/46 (2006.01) G06F 13/38 (2006.01)
(21) 출원번호 10-2009-7023628
(22) 출원일자(국제출원일자) 2008년06월27일
심사청구일자 2009년11월13일
(85) 번역문제출일자 2009년11월12일
(65) 공개번호 10-2010-0032359
(43) 공개일자 2010년03월25일
(86) 국제출원번호 PCT/US2008/068566
(87) 국제공개번호 WO 2009/157943
국제공개일자 2009년12월30일
(30) 우선권주장
12/147,332 2008년06월26일 미국(US)
(56) 선행기술조사문헌
KR1020060049710 A
US20070028010 A1
US20070192568 A1

(73) 특허권자
피시, 러셀, 에이치. III
미국, 텍사스 75250, 달라스, 프레스턴 오크스 로드 5400
(72) 발명자
피시, 러셀, 에이치. III
미국, 텍사스 75250, 달라스, 프레스턴 오크스 로드 5400
(74) 대리인
박윤원, 강명구

전체 청구항 수 : 총 12 항

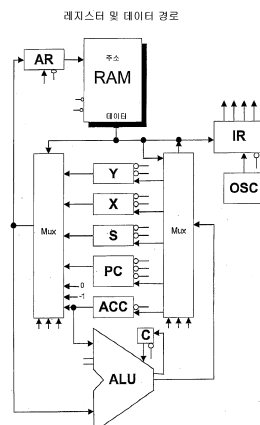
심사관 : 황승희

(54) 스레드 최적화 멀티프로세서 아키텍처

(57) 요약

하나의 양태에서, 본 발명은 하나의 시스템을 포함하며, 상기 시스템은 (a) 하나의 단일 칩 상의 다수의 병렬 프로세서와, (b) 상기 칩 상에 위치하며, 각각의 프로세서에 의해 액세스 가능한 컴퓨터 메모리를 포함하며, 각각의 프로세서는 최소 허용되는 인스트럭션 세트를 처리하고, 각각의 프로세서는 상기 프로세서 내의 3개 이상의 특정 레지스터에 특화된 로컬 캐시를 포함한다. 또 다른 양태에서, 본 발명은 하나의 시스템을 포함하며, 상기 시스템은 (a) 하나의 단일 칩 상의 다수의 병렬 프로세서와, (b) 상기 칩 상에 위치하고 각각의 프로세서에 의해 액세스 가능한 컴퓨터 메모리를 포함하며, 이때, 각각의 프로세서는 스레드-레벨 병렬 프로세싱에 최적화된 인스트럭션 세트를 처리하고, 각각의 프로세서는 칩 상의 컴퓨터 메모리의 내부 데이터 버스를 액세스하며, 상기 내부 데이터 버스는 메모리의 하나의 로우(row)의 폭을 갖는다.

대표도 - 도1



특허청구의 범위

청구항 1

삭제

청구항 2

삭제

청구항 3

- 단일 칩 상의 다수의 프로세서, 및

- 단일 칩 상에 위치하고, 상기 다수의 프로세서 각각에 의해 액세스 가능한 컴퓨터 메모리를 포함하되,

각각의 프로세서는 최소 허용되는(*de minimis*) 인스트럭션을 처리하며, 각각의 프로세서는 상기 프로세서 내 3개 이상의 특정 레지스터 각각에 대해 특화된 로컬 캐시를 포함하고,

최소 허용되는(*de minimis*) 인스트럭션 세트는, 프로세서 상에서 인스트럭션 시퀀스의 실행을 최적화하기 위한 다수의 인스트럭션 확장을 포함하며, 이러한 인스트럭션 확장은 20개 이하의 인스트럭션으로 구성되는 것을 특징으로 하는 시스템.

청구항 4

- 단일 칩 상의 다수의 프로세서, 및

- 단일 칩 상에 위치하고, 상기 다수의 프로세서 각각에 의해 액세스 가능한 컴퓨터 메모리를 포함하되,

각각의 프로세서는 최소 허용되는(*de minimis*) 인스트럭션을 처리하며, 각각의 프로세서는 상기 프로세서 내 3개 이상의 특정 레지스터 각각에 대해 특화된 로컬 캐시를 포함하고,

모놀리식 메모리 디바이스(monolithic memory device)용 반도체 제조 공정을 이용하여, 상기 다수의 프로세서가 상기 칩 상에 위치하는 컴퓨터 메모리와 함께 상기 칩 상에서 제조될 수 있는 것을 특징으로 하는 시스템.

청구항 5

제 4 항에 있어서, 상기 반도체 제조 공정은 4개 미만의 금속 인터커넥트 층을 이용하는 것을 특징으로 하는 시스템.

청구항 6

제 4 항에 있어서, 상기 반도체 제조 공정은 3개 미만의 금속 인터커넥트 층을 이용하는 것을 특징으로 하는 시스템.

청구항 7

- 단일 칩 상의 다수의 프로세서, 및

- 단일 칩 상에 위치하고, 상기 다수의 프로세서 각각에 의해 액세스 가능한 컴퓨터 메모리를 포함하되,

각각의 프로세서는 최소 허용되는(*de minimis*) 인스트럭션을 처리하며, 각각의 프로세서는 상기 프로세서 내 3개 이상의 특정 레지스터 각각에 대해 특화된 로컬 캐시를 포함하고,

상기 다수의 프로세서를 컴퓨터 메모리 회로에 집적하는 것은, 30% 미만의 칩 다이 크기 증가를 도출하는 것을 특징으로 하는 시스템.

청구항 8

- 단일 칩 상의 다수의 프로세서, 및

- 단일 칩 상에 위치하고, 상기 다수의 프로세서 각각에 의해 액세스 가능한 컴퓨터 메모리를 포함하되,

각각의 프로세서는 최소 허용되는(*de minimis*) 인스트럭션을 처리하며, 각각의 프로세서는 상기 프로세서 내 3

개 이상의 특정 레지스터 각각에 대해 특화된 로컬 캐시를 포함하고,

상기 다수의 프로세서를 컴퓨터 메모리 회로에 집적하는 것은, 20% 미만의 칩 다이 크기 증가, 또는 10% 미만의 칩 다이 크기 증가, 또는 5% 미만의 칩 다이 크기 증가를 도출하는 것을 특징으로 하는 시스템.

청구항 9

- 단일 칩 상의 다수의 프로세서, 및

- 단일 칩 상에 위치하고, 상기 다수의 프로세서 각각에 의해 액세스 가능한 컴퓨터 메모리를 포함하되,

각각의 프로세서는 최소 허용되는(*de minimis*) 인스트럭션을 처리하며, 각각의 프로세서는 상기 프로세서 내 3개 이상의 특정 레지스터 각각에 대해 특화된 로컬 캐시를 포함하고,

상기 칩 상에 각각의 프로세서를 생성하기 위해, 250,000개 미만의 트랜지스터가 사용되는 것을 특징으로 하는 시스템.

청구항 10

- 단일 칩 상의 다수의 프로세서, 및

- 단일 칩 상에 위치하고, 상기 다수의 프로세서 각각에 의해 액세스 가능한 컴퓨터 메모리를 포함하되,

각각의 프로세서는 최소 허용되는(*de minimis*) 인스트럭션을 처리하며, 각각의 프로세서는 상기 프로세서 내 3개 이상의 특정 레지스터 각각에 대해 특화된 로컬 캐시를 포함하고,

4개 미만의 금속 인터커넥트 층을 이용하는 반도체 제조 공정을 이용하여 상기 칩이 제조되는 것을 특징으로 하는 시스템.

청구항 11

- 동적 랜덤 액세스 메모리(DRAM) 다이에 내장되는(embedded) 다수의 병렬 프로세서

를 포함하는 시스템으로서, 이때, 상기 다수의 병렬 프로세서는 외부 메모리 컨트롤러 및 외부 프로세서와 통신하고,

상기 각각의 병렬 프로세서는 스레드-레벨 병렬처리에 대해 최적화된 인스트럭션 세트를 처리하는 것을 특징으로 하는 시스템.

청구항 12

제 11 항에 있어서, 상기 병렬 프로세서는 듀얼 인라인 메모리 모듈(dual inline memory module) 상에 장착되는 것을 특징으로 하는 시스템.

청구항 13

삭제

청구항 14

- 메모리 모듈 상에 장착되는 다수의 병렬 프로세서,

- 외부 메모리 컨트롤러, 및

- 범용 중앙 처리 유닛을 포함하되,

상기 다수의 병렬 프로세서는 각각 스레드-레벨 병렬 처리에 최적화된 인스트럭션 세트를 처리하고,

상기 메모리 모듈은 듀얼 인라인 메모리 모듈인 것을 특징으로 하는 시스템.

청구항 15

- 메모리 모듈 상에 장착되는 다수의 병렬 프로세서,

- 외부 메모리 컨트롤러, 및

- 범용 중앙 처리 유닛을 포함하되,

상기 다수의 병렬 프로세서는 각각 스레드-레벨 병렬 처리에 최적화된 인스트럭션 세트를 처리하고,

상기 메모리 모듈은 DRAM, SRAM 및 FLASH 메모리 중 하나 이상인 것을 특징으로 하는 시스템.

명세서

기술분야

[0001] 본 출원은 US 특허출원 제11/702,979호의 동시계속 출원이며, 2007년 2월 5일자 US 특허출원 제11/702,979호, 2006년 2월 3일자 US 가특허출원 제60/764,955호로부터 우선권을 주장한다. 상기 US 특허출원 제11/702,979호와 US 가특허출원 제60/764,955호의 전체 내용은 본원에서 참조로서 인용된다.

배경기술

[0002] 컴퓨터 속도는 2가지 일반적인 접근법을 이용하여 증가될 수 있는데, 즉, 인스트럭션 실행 속도를 증가시키는 접근법과, 더 많은 인스트럭션 실행을 병렬로 이뤄지게 하는 접근법이 있다. 인스트럭션 실행 속도가 실리콘에 서의 전자 이동 한계에 근접함에 따라, 병렬처리가 컴퓨터 속도를 증가시키는 것의 가장 좋은 대안이 되고 있다.

[0003] 병렬처리에 대한 이전의 시도는 다음과 같다:

[0004] 1. 다음 인스트럭션 인출을 현재 인스트럭션 실행과 겹치는 것.

[0005] 2. 인스트럭션 파이프라이닝(instruction pipelining). 인스트럭션 파이프라인은 각각의 인스트럭션을 가능한 많은 조각으로 쪼개고, 순차적 인스트럭션을 병렬 실행 유닛(execution unit)으로 매핑(mapping)하도록 시도한다. 다중-단계 인스트럭션의 비효율성과, 많은 소프트웨어 프로그램이 병렬 실행 유닛이 채워진 상태를 유지하기에 충분한 순차적 인스트럭션을 제공할 수 없다는 것과, 브랜치, 또는 루프, 또는 케이스 구조가 실행 유닛의 재-충전의 요구에 직면할 때 부가되는 긴 시간 페널티 때문에, 이론상의 최대 개선치는 거의 얻어지기 힘들다.

[0006] 3. 단일 인스트럭션 다중 데이터, 즉, SIMD(Single instruction multiple data). 이러한 타입의 기술은 Intel SSE 인스트럭션 셋에서 발견된다. 이러한 기법에서, 단일 인스트럭션(single instruction)이 다중 데이터(multiple data) 세트 상에서 실행된다. 이 기법은 비디오 그래픽 렌더링 등의 특수 애플리케이션에 대해서만 유용하다.

[0007] 4. 하이퍼큐브(Hypercube). 이 기법은 큰 2차원 어레이를 이용하며, 때때로 프로세서와 로컬 메모리의 3차원 어레이를 이용한다. 이들 프로세서의 어레이를 지원하기에 필수적인 통신 및 상호연결이 이 기법을 매우 특수화된 애플리케이션으로 국한시킨다.

[0008] 파이프라인은 하나의 인스트럭션의 실행(가령, 인출, 디코딩, 실행, 저장 등)의 조각을 연속적으로 수행하는 다수의 순차적 단계로 구성된 인스트럭션 실행 유닛(instruction execution unit)이다. 몇 개의 파이프라인이 병렬로 위치되어, 프로그램 인스트럭션이 차례로, 모든 파이프라인이 하나의 인스트럭션을 실행할 때까지, 각각의 파이프라인으로 유입될 수 있다. 그 후, 인스트럭션 채우기(instruction filling)는 본래의 파이프라인을 이용하여 반복된다. N개의 파이프라인이 인스트럭션으로 채워지고, 실행할 때, 성능 효과는 단일 실행 유닛에 대한 실행 속도의 N배 증가와 이론적으로 동일하다.

[0009] 성공적인 파이프라이닝은 다음의 조건에 따라 좌우된다.

[0010] 1. 하나의 인스트럭션의 실행이 몇 개의 연속적인 단계로서 정의될 수 있어야 한다.

[0011] 2. 각각의 인스트럭션은 동일한 개수의 단계를 가져야 한다.

[0012] 3. 인스트럭션 당 상태의 개수에 의해 병렬 실행 유닛의 최대 개수가 판단된다.

[0013] 파이프라이닝이 병렬 파이프라인의 개수를 바탕으로 하는 성능 증가를 성취할 수 있기 때문에, 그리고 병렬 파이프라인의 개수는 하나의 인스트럭션의 단계의 개수에 의해 판단되기 때문에, 파이프라인은 복잡한 다중-단계 인스트럭션을 촉진한다.

[0014] 과하게 파이프라이닝된 컴퓨터는 병렬 파이프라인 실행 유닛으로부터 기대되어지는 이론적 성능 향상만큼도 거

의 성취하지 않는다. 이러한 파이프라인 패널티에 대한 몇 가지 이유로는 다음과 같다:

- [0015] 1. 소프트웨어 프로그램은 단지 순차적 인스트럭션만으로 이뤄지는 것이 아니다. 다양한 연구에 따라 실행 흐름의 변화가 매 8-10개의 인스트럭션마다 발생한다. 프로그램 흐름을 바꾸는 임의의 브랜치가 파이프라인을 혼란시킨다. 파이프라인 혼란을 최소화하기 위한 시도는 복잡하며, 불완전하게 완화시키는 경향이 있다.
- [0016] 2. 모든 인스트럭션이 동일한 개수의 단계를 갖도록 강제하는 것은 종종 최소 공통 분모의(즉, 가장 느리고 가장 복잡한) 인스트럭션을 갖는 실행 파이프라인을 초래한다. 파이프라인 때문에, 모든 인스트럭션이 동일한 개수의 단계로 강제된다. 상기 인스트럭션들이 상기 개수의 단계를 요구하느냐의 여부에 관계없이 말이다. 예를 들어, 논리 연산(가령, AND, 또는 OR)은 크기 순으로 ADD보다 더 빨리 실행되지만, 종종 이들 모두는 동일한 실행 시간을 할당받는다.
- [0017] 3. 파이프라인은 다중-단계 복합 인스트럭션을 촉진시킨다. 2개의 단계를 필요로 할 수 있는 인스트럭션은 20개의 단계를 채우기 위해 늘려지는데, 이는 파이프라인의 깊이(depth) 때문이다. (Intel Pentium 4는 20단계 파이프라인을 이용한다)
- [0018] 4. 각각의 파이프라인 단계에 대하여 요구되는 시간은 논리 회로 및 이에 연계된 트랜지스터를 통과하는 전파 딜레이와, 이에 추가로, 특정 단계에 대한 설계 오차에 해당한다.
- [0019] 5. 파이프라인 레지스터 및 그 밖의 다른 자원 액세스에 대한 중재는 종종, 중재 로직(arbitration logic)의 트랜지스터의 전파 딜레이로 인해 성능을 감소시킨다.
- [0020] 6. 추가적인 단계가 실행의 속도를 높이기보다는 실제로는 실행을 느리게 하기 전에, 인스트럭션이 쪼개질 수 있는 단계의 개수에는 상한이 존재한다. 일부 연구들에 따르면, Digital Equipment Corporation의 Alpha 프로세서의 마지막 세대의 파이프라인 아키텍처(pipeline architecture)는 이러한 포인트를 초과했고, 실제로는 이전의 더 짧은 파이프라인의 프로세서 버전보다 더 느리게 수행됨을 알 수 있다.

[0021] 파이프라인 쪼개기(Splitting Apart the Pipelines)

[0022] CPU 디자인의 리팩토링(re-factoring)에 대한 한 가지 양태는, 다수(N)의 단순화된 프로세서로 쪼개지는 파이프라이닝된 실행 유닛의 개념이다. (이러한 디자인에서 레지스터와 그 밖의 다른 일부 로직은 복제(duplicate)될 필요가 있다) N개의 단순화된 프로세서 각각은 앞서 언급된 파이프라이닝된 아키텍처에 비해 다음의 장점을 갖는다.

- [0023] 1. 어떠한 파이프라인도 지연되지 않는다. 어떠한 브랜치 예측도 필요하지 않다.
- [0024] 2. 모든 인스트럭션이 가장 느린 인스트럭션과 동일한 실행 시간을 할당받는 것이 아니라, 인스트럭션들은 필요한 시간만큼을 취할 수 있다.
- [0025] 3. 필수적인 실행 단계로 축소시키고, 이로 인해서 파이프라인 패널티를 축소시킴으로써, 인스트럭션은 단순화될 수 있다.
- [0026] 4. 파이프라인으로부터 제거되는 각각의 단계에 의해, 전파 딜레이가 제거될 수 있고, 단계에 필요한 설계 오차가 제거될 수 있다.
- [0027] 5. 레지스터 중재(register arbitration)가 생략될 수 있다.

[0028] 덧붙이자면, N개의 단순화된 프로세서를 갖는 시스템은 파이프라이닝된 CPU(pipelined CPU)에 비교할 때, 다음의 장점들을 가질 수 있다:

- [0029] 1. 최대 파이프라인 병렬처리의 한계가 없어질 것이다.
- [0030] 2. 파이프라이닝된 프로세서와 달리, 다중 자립형 프로세서(standalone processor)는 선택적으로 파워-다운(power down)되어, 사용되지 않을 때의 전력 소모를 줄일 수 있다.

[0031] 병렬처리의 현재 접근법에 동반되는 그 밖의 다른 문제점들

[0032] 병렬처리의 다수의 구현예들이 Amdahl의 법칙을 극복하지 못한다. 병렬처리를 통한 가속은 문제점들 중 비-직렬

화 부분으로 인한 오버헤드에 의해 제한된다. 필수적으로, 병렬처리의 양이 증가함에 따라서, 이를 지원하기에 필수적인 통신(communication)이 병렬처리로 인한 이득을 압도한다.

[0033] 레드라인에 위치하는 정지등(Stoplight Sitting at Redline)

[0034] 현재의 프로세서의 또 다른 비효율은, 즉각적인 컴퓨팅 요구를 충족시키도록 컴퓨팅 파워를 확장할 수 없다는 것이다. 대부분의 컴퓨터는 무언가가 발생하기를 대기하면서 시간을 낭비한다. 컴퓨터들은 I/O, 다음 인스트럭션, 메모리 액세스, 또는 때때로 인간과의 인터페이스를 대기한다. 이러한 대기는 컴퓨팅 파워의 비효율적인 낭비이다. 덧붙이자면, 컴퓨터의 대기에 소모되는 시간은 종종, 증가된 파워 소모와 열 발생을 초래한다.

[0035] 엔진 제어기, 신호 프로세서 및 방화벽 라우터 등의 애플리케이션은 대기 규칙의 예외이다. 이들 애플리케이션은 문제점 및 해결책 세트의 정의된 속성 때문에 병렬처리 가속의 바람직한 후보이다. N번의 독립적인 곱셈의 곱을 요구하는 문제는 N번 승수를 이용하여 더 빠르게 해결될 수 있다.

[0036] 범용 컴퓨터의 인지 성능(perceived performance)은 자신의 피크 성능이다. 이에 근접하게 범용 컴퓨터가 바빠지는 것은 빠른 스크린 재생을 동반하는 비디오 게임 실행, 또는 커다란 소스 파일 컴파일하기, 또는 데이터베이스 검색하기이다. 최적의 환경에서, 비디오 렌더링은 특수 목적의 음영/변환/렌더링 하드웨어의 한 요소가 될 것이다. 프로그래밍을 이러한 특수 목적의 하드웨어의 한 요소로 하는 하나의 방법이 “스레드(thread)”의 사용이다.

[0037] 스레드는, 자급식이면서 드물게 다른 스레드와 데이터를 통신하는 독립적인 프로그램이다. 스레드의 일반적인 용도는 느린 실시간 활동으로부터 데이터를 수집하고, 조합된 결과를 제공하는 것이다. 스레드는 또한 디스플레이 상의 변화를 렌더링하기 위해 사용될 수 있다. 스레드는 또 다른 스레드와의 추가적인 상호작용을 요청하기 전에, 수천, 또는 수백 개의 단계를 통해 변천할 수 있다. 독립적인 스레드는 병렬처리를 통한 증가된 성능에 대한 기회를 제공한다.

[0038] 다수의 소프트웨어 컴파일러가, 소프트웨어 디자인 프로세스를 팩터링하기 위한 목적으로 스레드의 생성과 관리를 지원한다. 동일한 팩터링이, 바람직한 실시예의 스레드에 의해 최적화되는 마이크로프로세서(TOMI)에서의 스레드 레벨 병렬처리의 기법을 통해 다중 CPU 병렬 프로세싱을 지원할 것이다.

[0039] 스레드 레벨 병렬처리(Thread Level Parallelism)

[0040] 스레딩은 소프트웨어 프로그램을 하나의 단일 CPU 상에 팩터링(factoring)하기 위한 잘 알려진 기법이다. 스레드 레벨 병렬처리는 TOMI 프로세서를 사용함으로써, 프로그램 가속을 얻을 수 있다.

[0041] 다른 병렬처리 접근법에 비한 TOMI 프로세서의 한 가지 확실한 이점은 TOMI 프로세서가 현재 소프트웨어 프로그래밍 기법에 대한 최소의 변화를 요구한다는 것이다. 새로운 알고리즘은 개발될 필요가 없다. 많은 기존의 프로그램들이 재-컴파일(re-compile)될 필요는 있지만, 다시 쓰여질 필요는 없다.

[0042] 효과적인 TOMI 컴퓨터 아키텍처는 다수의 단순화된 프로세서 주변에서 구축되어야 한다. 여러 다른 타입의 컴퓨팅 문제에 대하여 여러 다른 구조가 사용될 수 있다.

[0043] 기본 컴퓨터 연산(Fundamental Computer Operation)

[0044] 범용 컴퓨터에 있어서, 감소하는 주파수 수준의 가장 일반적인 연산은 로드 및 저장(Load and store)과, 시퀀싱(sequencing)과, 수리 및 논리(math and logic)이다.

[0045] 로드 및 저장(Load and store)

[0046] LOAD 및 STORE의 매개변수는 출발지(source)와 도착지(destination)이다. LOAD 및 STORE의 파워는 출발지와 도착지의 범위이다(가령, 4Gbyte가 256byte보다 더 파워풀한 범위이다). 많은 데이터 세트에 있어서, 현재 출발지와 도착지에 대한 집약성(locality)이 중요하다. 플러스 1, 마이너스 1이 가장 유용하다. 현재의 출발지와 도착지로부터의 오프셋(offset)을 증가시킴으로써, 점진적으로 덜 유용해진다.

- [0047] LOAD 및 STORE는 또한 메모리 계층구조(memory hierarchy)에 의해 영향 받을 수 있다. 저장장치로부터의 LOAD는 CPU가 수행할 수 있는 가장 느린 연산일 수 있다.
- [0048] **시퀀싱(Sequencing)**
- [0049] 브랜치(branch)와 루프(loop)가 기본 시퀀싱 인스트럭션이다. 인스트럭션 시퀀스는 컴퓨터가 결정을 내리는 방식인 테스트를 기반으로 변한다.
- [0050] **수리 및 논리(Math and Logic)**
- [0051] 수리 및 논리 연산은 3가지 연산 중 가장 적게 사용된다. 논리 연산은 CPU가 수행할 수 있고, 단일 로직 게이트의 딜레이만큼 적은 딜레이를 필요로 할 수 있는 가장 빠른 연산이다. 상위 비트(higher order bit)가 하위 비트(lower order bit) 연산의 결과에 따라 좌우되기 때문에, 수리 연산은 더 복잡하다. 캐리 예측(carry lookahead)을 이용할 때조차, 32-비트 ADD는 32개 이상의 게이트 딜레이를 필요로 할 수 있다. 쉬프트 앤드 애드(shift and add) 기법을 이용하는 MULTIPLY는 32개의 ADD에 상응하는 것을 필요로 할 수 있다.
- [0052] **인스트럭션 크기의 상충(Tradeoffs of Instruction Size)**
- [0053] 완벽한 인스트럭션 세트는 무한개의 가능한 출발지, 도착지, 연산 및 다음 인스트럭션을 선택하기에 충분히 큰 op-코드로 구성될 것이다. 불행하게도, 완벽한 인스트럭션 세트의 op-코드는 무한으로 넓을 것이며, 따라서 인스트럭션 대역폭이 0이 될 것이다.
- [0054] 높은 인스트럭션 대역폭을 위한 컴퓨터 설계는, 가장 적은 수의 op-코드 비트를 갖고 가장 일반적인 출발지, 도착지, 연산 및 다음 인스트럭션을 효과적으로 정의할 수 있는 op-코드를 갖는 인스트럭션 세트의 생성을 포함한다.
- [0055] 넓은 op-코드는 높은 인스트럭션 버스 대역폭을 요구하고, 최종 아키텍처는, 컴퓨터의 성능이, 메모리로부터 인스트럭션이 인출되는 속도에 의해 제한되는 폰 노이만 병목현상(Von Neumann bottleneck)에 의해 빠르게 제한될 것이다.
- [0056] 메모리 버스가 64비트 폭을 갖는 경우, 각각의 메모리 사이클에서, 하나의 64-비트 인스트럭션, 또는 2개의 32-비트 인스트럭션, 또는 4개의 16-비트 인스트럭션, 또는 8개의 8-비트 인스트럭션을 인출할 수 있다. 32-비트 인스트럭션은 16-비트 인스트럭션의 2배만큼 유용한데, 왜냐하면 인스트럭션 대역폭이 절반이기 때문이다.
- [0057] 인스트럭션 세트 설계의 주요 목적은 인스트럭션 중복(redundancy)을 감소시키는 것이다. 일반적으로, 최적화된 효율적인 인스트럭션 세트는 인스트럭션과 데이터 모두의 집약성(locality)을 이용한다. 가장 쉬운 인스트럭션 최적화가 감당되어왔다. 대부분의 컴퓨터 프로그램에 있어서, 가장 가능성 높은 다음 인스트럭션은 메모리에서 순차적으로 다음에 위치하는 인스트럭션이다. 따라서 모든 다음 인스트럭션 필드를 갖는 인스트럭션 대신, 대부분의 인스트럭션은 다음 인스트럭션은 현재 인스트럭션 +1이라고 가정한다. 출발지에 대하여 0비트를, 도착지에 대하여 0비트를 갖는 아키텍처를 생성하는 것이 가능하다.
- [0058] **스택 아키텍처(Stack Architecture)**
- [0059] 또한 스택 구조 컴퓨터는 0 피연산자 아키텍처(zero operand architecture)라고 일컬어진다. 스택 아키텍처는 푸시 다운 스택(push down stack)의 내용물을 기반으로 하여 모든 연산을 수행한다. 2개의 피연산자 연산은 2개의 피연산자 모두 스택 상에 존재할 것을 필요로 한다. 연산이 실행될 때, 두 개의 피연산자 모두는 스택으로부터 팝(POP)되며, 연산이 실행되어, 결과가 스택 상으로 다시 푸시(PUSH)될 것이다. 출발지와 도착지가 스택 상에서 존재하는 것처럼 여겨질 수 있기 때문에, 스택 아키텍처 컴퓨터는 매우 짧은 op-코드를 가질 수 있다.
- [0060] 대부분의 프로그램은 필요할 때마다 스택 상에서 이용가능한 것은 아닐 수 있는 전역 레지스터(global register)의 내용물을 필요로 한다. 이러한 사건을 최소화하기 위한 시도에는, 스택의 가장 위쪽에 존재하는 피연산자가 아닌 다른 피연산자를 액세스하는 것을 가능케 하는 스택 인덱싱(stack indexing)이 포함된다. 스택 인덱싱은, 스택 인덱스 값을 스택 자체에게 위치시키기 위한 더 큰 인스트럭션이나 추가적인 연산을 야기하는

추가적인 op-코드 비트를 필요로 한다. 종종 하나 이상의 추가적인 스택이 형성된다. 더 낫지만, 최적화된 것은 아닌 솔루션은 스택/레지스터 아키텍처 조합이다.

[0061] 또한 스택 아키텍처 연산은 종종, 명백한 최적화에 반하는 방식으로 중복이다. 예를 들자면, 각각의 POP과 PUSH 연산은, 스택이 메모리에서 조작되기 때문에 시간을 낭비하는 메모리 연산을 초래할 가능성을 갖는다. 덧붙이자면, 스택 연산은 다음 연산에서 즉시 필요한 피연산자를 소모할 수 있으며, 이로 인해서, 또 다른 메모리 연산의 가능성을 갖는 피연산자 복제(operand duplication)가 필요할 수 있다. 1차원 어레이의 모든 요소에 15를 곱하는(multiply) 연산의 예를 들어보자.

[0062] 스택 아키텍처 상에서, 이 연산은 다음과 같다:

[0063] 1. 어레이의 시작 주소를 푸시하기 (PUSH start address of array)

[0064] 2. 주소를 복제하기(DUPLICATE address) (따라서 어레이의 결과를 저장하기 위한 주소를 갖는다)

[0065] 3. 주소를 복제하기(DUPLICATE address) (따라서 어레이로부터 관독할 주소를 갖는다)

[0066] 4. 간접 주소 푸시하기(PUSH INDIRECT) (스택의 가장 상위(top)에 의해 가르켜지는(pointed) 어레이 위치의 내용을 푸시하기)

[0067] 5. 15를 푸시하기(PUSH 15)

[0068] 6. 곱셈(MULTIPLY) (라인 3.에서 읽은 어레이 내용을 15배하기)

[0069] 7. 교환(SWAP) (다음 인스트럭션에 대하여 스택의 가장 상위 상의 어레이 주소를 얻기)

[0070] 8. 간접 주소 팝하기(PUSH INDIRECT) (곱셈 결과를 POP하고, 어레이에 다시 저장한다)

[0071] 9. 증연산(INCREMENT) (다음 어레이 아이템을 가리키기)

[0072] 10. 어레이가 완료될 때까지 단계2.로 가기(GO TO)

[0073] 라인 9.에서의 루프 카운터는 추가적인 매개변수를 필요로 할 것이다. 일부 아키텍처에서, 이러한 매개변수는 또 다른 스택에 저장된다.

[0074] 가설적 레지스터/어큐물레이터 아키텍처에서의 예는 다음과 같이 구현될 수 있다:

[0075] 1. 어레이 시작 주소의 포인터를 저장하기(STORE POINTER start address of array)

[0076] 2. 포인터 읽기(READ POINTER) (가르켜지는 주소의 내용을 어큐물레이터로 읽기)

[0077] 3. 15를 곱하기(MULTIPLY 15)

[0078] 4. 포인터 저장하기(STORE POINTER) (가르키고 있는 주소로 결과를 저장하기)

[0079] 5. 포인터 증연산하기(INCREMENT POINTER)

[0080] 6. 어레이가 완료될 때까지 라인 2.로 가기(GO TO)

[0081] 앞의 예에 대하여, 스택 아키텍처의 9개의 단계와 레지스터 아키텍처의 5개의 단계를 비교해 보자. 덧붙이자면, 스택 연산은 스택 연산으로 인한 추가적인 메모리 액세스에 대한 3번 이상의 기회를 갖는다. 가설적 레지스터/어큐물레이터 아키텍처의 루프 제어는 레지스터에서 쉽게 다뤄질 수 있다.

[0082] 수식의 값을 구하기에는 스택이 유용하며, 대부분의 컴파일러에서 마찬가지로 사용된다. 또한 스택은 함수 호출 같은 중첩 연산에 대하여 유용하다. 대부분의 C 컴파일러는 스택을 이용하여 함수 호출을 구현한다. 그러나 범용 저장장치에 의한 보조 없이, 스택 아키텍처는 다수의 추가적인 데이터 이동 및 조작을 필요로 한다. 최적화 목적을 위해, 또한 스택 PUSH 및 POP 연산이 수리 및 논리 연산으로부터 분리되어야 한다. 그러나 앞의 예에서 나타나듯이, 데이터를 반복적으로 로딩하고 저장할 때, PUSH INDIRECT 및 POP INDIRECT에 의해 어레이 주소가 소모되기 때문에, 스택은 특히 비효율적이다.

발명의 상세한 설명

- [0083] 하나의 양태에서, 본 발명은 하나의 시스템을 포함하는데, 상기 시스템은, (a) 하나의 단일 칩 상의 다수의 병렬 프로세서와, (b) 상기 칩 상에 위치하고 각각의 프로세서에 의해 액세스 가능한 컴퓨터 메모리를 포함하며, 여기서 상기 각각의 프로세서는 최소 허용되는(*de minimis*) 인스트럭션 세트를 처리하고, 각각의 프로세서는 상기 프로세서 내 3개 이상의 특정 레지스터 각각에 특화된 로컬 캐시를 포함한다.
- [0084] 다양한 양태에서, (1) 각각의 로컬 캐시의 크기는 칩 상의 랜덤 액세스 메모리의 하나의 로우(row)에 대응하며, (2) 관련 캐시를 갖는 3개 이상의 특정 레지스터는 인스트럭션 레지스터, 출발지 레지스터 및 도착지 레지스터를 포함하고, (3) 상기 최소 허용되는(*de minimis*) 인스트럭션 세트는 4개의 기본 인스트럭션을 포함하며, (4) 각각의 프로세서는 단일 스레드를 처리하며, (5) 어큐뮬레이터는 모든 인스트럭션에 대한 피연산자이며, (6) 각각의 기본 인스트럭션에 대한 도착지는 항상 피연산자 레지스터이며, (7) 3개의 레지스터는 자동-증연산하고, 3개의 레지스터는 자동-감연산하며, (8) 상기 인스트럭션 세트는 어떠한 BRANCH 인스트럭션 및 어떠한 JUMP 인스트럭션도 포함하지 않으며, (9) 각각의 인스트럭션은 최대 8비트 길이를 가지며, (10) 단일 마스터 프로세서는 각각의 병렬 프로세서를 관리하는 것을 담당한다.
- [0085] 또 다른 양태에서, 본 발명은 하나의 시스템을 포함하며, 상기 시스템은 (a) 하나의 단일 칩 상의 다수의 병렬 프로세서와, (b) 상기 칩 상에 위치하고, 각각의 프로세서에 의해 액세스 가능한 컴퓨터 메모리를 포함하며, 여기서 각각의 프로세서는 스레드-레벨 병렬 프로세싱에 대한 최적화된 인스트럭션 세트를 처리한다.
- [0086] 다양한 양태에서, (1) 각각의 프로세서는 최소 허용되는 인스트럭션 세트를 처리하고, (2) 각각의 프로세서는 상기 프로세서 내 3개 이상의 특정 레지스터 각각에 특화된 로컬 캐시를 포함하며, (3) 상기 각각의 로컬 캐시의 크기는 칩 상의 랜덤 액세스 메모리의 하나의 로우(row)에 상응하며, (4) 상기 3개 이상의 특정 레지스터는 인스트럭션 레지스터, 출발지 레지스터 및 도착지 레지스터를 포함하며, (5) 상기 최소 허용되는 인스트럭션 세트는 7개의 기본 인스트럭션을 포함하며, (6) 각각의 프로세서가 단일 스레드를 처리하며, (7) 단일 마스터 프로세서는 각각의 병렬 프로세서를 관리하며, (8) 상기 최소 허용되는 인스트럭션 세트는 프로세서 연산을 최적화시키고, 소프트웨어 컴파일러 효율을 촉진시키는 인스트럭션 확장의 최소 세트를 포함한다.
- [0087] 또 하나의 실시예에서, 본 발명은 다수의 병렬 프로세서, 마스터 프로세서 및 단일 칩 상의 컴퓨터 메모리를 이용하는 스레드-레벨 병렬 프로세싱의 방법을 포함하며, 여기서 각각의 다수의 프로세서는 최소 허용되는 인스트럭션 세트를 처리하고, 단일 스레드를 처리하며, 상기 방법은 (a) 각각의 다수의 프로세서에서 로컬 캐시를 특정 3개의 레지스터에 할당하는 단계와, (b) 단일 스레드를 처리하기 위해 상기 각각의 프로세서 중 하나를 할당하는 단계와, (c) 프로세서가 각각의 할당된 스레드를 처리(*processing*)하는 단계와, (d) 상기 프로세서에 의해 처리된 각각의 스레드로부터의 결과를 처리하는 단계와, (e) 스레드가 처리된 후, 다수의 프로세서 중 하나를 역-할당(*de-allocating*)하는 단계를 포함하며, (f) 상기 최소 허용되는 인스트럭션 세트는 프로세서 관리를 최적화시키기 위한 인스트럭션의 최소 세트를 포함한다.
- [0088] 다양한 실시예에서, 상기 최소 허용되는 인스트럭션 세트는 7개의 기본 인스트럭션을 포함하며, 상기 최소 허용되는 인스트럭션 세트의 인스트럭션은 최대 8비트 길이를 갖는다. 상기 최소 허용되는 인스트럭션 세트는 또한, 7개의 기본 인스트럭션 외에, 확장 인스트럭션의 세트를 포함할 수 있으며, 이는 TOMI CPU의 내부 연산을 최적화하고, TOMI CPU에 의해 실행되는 소프트웨어 프로그램 인스트럭션의 실행을 최적화하고, TOMI CPU에 대한 소프트웨어 컴파일러의 연산을 최적화하는 것을 보조한다. 다수의 TOMI CPU 코어를 포함하는 본 발명의 실시예는 또한, 다수의 CPU 코어를 관리하기 위해 사용되는 프로세서 관리 인스트럭션의 제한된 세트를 포함할 수 있다.
- [0089] 또 하나의 양태에서, 본 발명은 하나의 시스템을 포함하며, 상기 시스템은 (a) 메모리 모듈 상에 장착되는 다수의 병렬 프로세서와, (b) 외부 메모리 컨트롤러와, (c) 범용 중앙 처리 유닛을 포함하며, 각각의 병렬 프로세서는 스레드-레벨 병렬 프로세싱에 최적화된 인스트럭션 세트를 처리한다.
- [0090] 다양한 실시예에서, (1) 각각의 병렬 프로세서는 최소 허용된 인스트럭션 세트를 처리하며, (2) 메모리 모드 레지스터에서 할당된 하나 이상의 비트가 상기 병렬 프로세서 중 하나 이상을 활성화, 또는 비활성화하며, (3) 상기 메모리 모듈은 듀얼 인라인 메모리 모듈이고, (4) 각각의 프로세서는 단일 스레드를 처리하며, (5) 다수의 스레드는 공유 메모리를 통해 데이터를 공유하고, (6) 다수의 스레드는 하나 이상의 공유 변수를 통해 데이터를 공유하며, (7) 상기 메모리 모듈은 DRAM, SRAM, FLASH 메모리 중 하나 이상이며, (8) 병렬 프로세서의 중 하나 이상은 마스터 프로세서로서 취급되고, 그 나머지 병렬 프로세서는 슬레이브 프로세서로서 취급되며, (9) 각각

의 프로세서는 특정 클럭 속도를 갖고, 마스터 프로세서가 아닌 각각의 프로세서는 성능, 또는 전력 소모를 최적화하기 위해 조정된 프로세서의 클럭 속도를 가지며, (10) 각각의 프로세서는 마스터 프로세서, 또는 슬레이브 프로세서로서 처리되며, (11) 상기 마스터 프로세서는 몇 개의 슬레이브 프로세서에 의한 처리를 요청하고, 상기 몇 개의 슬레이브 프로세서로부터의 출력에 대해 대기하며, 상기 출력들을 조합하고, (12) 출력이 각각의 프로세서로부터 수신될 때, 상기 마스터 프로세서는 몇 개의 프로세서로부터의 출력을 조합하며, (13) 병렬 프로세서 중 하나 이상을 중단되게 함으로써, 낮은 전력 소모가 제공되며, (14) 각각의 병렬 프로세서는 프로그램 카운터와 연계되고, 병렬 프로세서와 연계되는 프로그램 카운터로 모두 1을 기록함으로써, 상기 병렬 프로세서는 중단된다.

[0091] 또 하나의 양태에서, 본 발명은 하나의 시스템을 포함하며, 상기 시스템은 동적 램덤 액세스 메모리(DRAM) 다이로 내장되는 다수의 병렬 프로세서를 포함하며, 상기 다수의 병렬 프로세서는 외부 메모리 컨트롤러 및 외부 프로세서와 통신하며, 각각의 병렬 프로세서는 스레드-레벨 병렬 프로세싱에 최적화된 인스트럭션 세트를 처리한다.

[0092] 그 밖의 다른 다양한 실시예에서, (1) 다이는 DRAM 핀아웃과 패키징되고, (2) 병렬 프로세서는 듀얼 인라인 메모리 모듈 상에 장착되며, (3) 상기 시스템은, 상기 프로세서가 DRAM 모드 레지스터를 통해 활성화될 때를 제외하고는, DRAM으로서 기능하며, (4) 외부 프로세서는 관련 영구 저장 장치로부터 DRAM으로 데이터 및 인스트럭션을 전송하며, (5) 상기 영구 저장 장치는 FLASH 메모리이고, (6) 상기 외부 프로세서는 상기 병렬 프로세서와 외부 장치 간에 입/출력 인터페이스를 제공한다.

[0093] 또 하나의 양태에서, 본 발명은 하나의 시스템을 포함하며, 상기 시스템은 (a) 하나의 단일 칩 상의 다수의 프로세서와, (b) 상기 칩 상에 위치하고 각각의 프로세서에 의해 액세스 가능한 컴퓨터 메모리를 포함하며, 각각의 프로세서는 최소 허용되는 인스트럭션 세트를 처리하고, 각각의 프로세서는 프로세서 내 3개 이상의 특정 레지스터에 대해 특화된 로컬 캐시를 포함한다.

[0094] 그 밖의 다른 다양한 실시예에서, (1) 각각의 로컬 캐시의 크기는 칩 상의 랜덤 액세스 메모리의 하나의 로우에 상응하며, (2) 각각의 프로세서는 칩 상의 랜덤 액세스 메모리의 내부 데이터 버스를 액세스하며, 상기 내부 데이터 버스는 랜덤 액세스 메모리의 하나의 로우의 폭을 가지며, (3) 상기 내부 데이터 버스의 폭은 1024, 2048, 4096, 8192, 16328 또는 32656비트이며, (4) 상기 내부 데이터 버스의 폭은 1024 비트의 정수 배이며, (5) 프로세서 내 3개 이상의 특정 레지스터 각각에 특화된 로컬 캐시는, 하나의 메모리 판독, 또는 기록 사이클로 채워지거나 플러쉬(flush)되며, 상기 최소 허용되는 인스트럭션 세트는 7개의 기본 인스트럭션으로 구성되고, (7) 기본 인스트럭션 세트는 ADD, XOR, INC, AND, STOREACC, LOADACC 및 LOADI 인스트럭션을 포함하며, (8) 최소 허용된 인스트럭션 세트 내의 각각의 인스트럭션은 최대 8비트 길이를 갖고, (9) 상기 최소 허용된 인스트럭션 세트는 프로세서 상에서의 인스트럭션 시퀀스의 실행을 최적화하기 위한 다수의 인스트럭션 확장을 포함하며, 이러한 인스트럭션 확장은 본질적으로 20개 미만의 인스트럭션으로 구성되고, (10) 각각의 인스트럭션 확장은 최대 8비트 길이를 갖고, (11) 상기 최소 허용된 인스트럭션 세트는 칩 상의 다수의 프로세서를 선택적으로 제어하기 위한 인스트럭션의 세트를 포함하며, (12) 각각의 프로세서 제어 인스트럭션은 최대 8비트의 길이를 갖고, (13) 상기 다수의 프로세서는, 모놀리식 메모리 장치용 반도체 제조 공정을 이용하여 칩 상에 위치하는 컴퓨터 메모리와 함께 칩 상에서 제조되고, (14) 상기 반도체 제조 공정은 4개 미만의 금속 인터커넥트 층을 이용하며, (15) 상기 반도체 제조 공정은 3개 미만의 금속 인터커넥트 층을 사용하며, (16) 다수의 프로세서를 컴퓨터 메모리 회로에 집적하는 것은 30% 미만의 칩 다이 크기 증가를 도출하며, (17) 다수의 프로세서를 컴퓨터 메모리 회로에 집적하는 것은 20% 미만의 칩 다이 크기 증가를 도출하며, (18) 다수의 프로세서를 컴퓨터 메모리 회로에 집적하는 것은 10% 미만의 칩 다이 크기 증가를 도출하며, 다수의 프로세서를 컴퓨터 메모리 회로에 집적하는 것은 5% 미만의 칩 다이 크기 증가를 도출하며, (20) 칩 상에 각각의 프로세서를 생성하기 위해 250,000개 미만의 트랜지스터가 사용되며, (21) 상기 칩은 4개 미만의 금속 인터커넥트를 이용하는 반도체 제조 공정을 이용하여 제조되며, (22) 각각의 프로세서는 단일 스레드를 처리하고, (23) 어큐뮬레이터는 증연산 인스트럭션을 제외한 모든 기본 인스트럭션에 대해 피연산자이며, (24) 각각의 기본 인스트럭션에 대한 도착지는 항상 피연산자 레지스터이며, (25) 상기 3개의 레지스터는 자동-증연산하고 자동-감연산하며, (26) 각각의 기본 인스트럭션은 종료하기 위해 단 하나의 클럭 사이클을 필요로 하며, (27) 상기 인스트럭션 세트는 어떠한 BRANCH 인스트럭션 및 JUMP 인스트럭션도 포함하지 않고, (28) 단일 마스터 프로세서는 각각의 병렬 프로세서를 관리하는 일을 담당한다.

[0095] 또 다른 양태에서, 본 발명은 하나의 시스템을 포함하며, 상기 시스템은 (a) 하나의 단일 칩 상의 다수의 병렬 프로세서와, (b) 상기 칩 상에 위치하고 각각의 프로세서에 의해 액세스 가능한 컴퓨터 메모리를 포함하며, 각

각의 프로세서는 스레드-레벨 병렬 프로세싱에 대해 최적화된 인스트럭션 세트를 처리하며, 각각의 프로세서는 칩 상의 컴퓨터 메모리의 내부 데이터 버스를 액세스하며, 상기 내부 데이터 버스는 메모리의 하나의 로우(row)보다 더 넓지 않다.

[0096] 다양한 실시예에서, (1) 각각의 프로세서는 최소 허용되는 인스트럭션 세트를 처리하고, (2) 각각의 프로세서는 프로세서 내 3개 이상의 특정 레지스터 각각에 대해 특화된 로컬 캐시를 포함하며, (3) 각각의 로컬 캐시의 크기는 칩 상의 컴퓨터 메모리의 하나의 로우에 상응하고, (4) 3개 이상의 특정 레지스터는 인스트럭션 레지스터, 출발지 레지스터 및 도착지 레지스터를 포함하며, (5) 상기 최소 허용되는 인스트럭션 세트는 본질적으로 7개의 기본 인스트럭션으로 구성되며, (6) 상기 기본 인스트럭션 세트는 ADD, XOR, INC, ADN, STOREACC, LOADACC 및 LOADI 인스트럭션을 포함하며, (7) 인스트럭션 세트의 각각의 인스트럭션은 최대 8비트 길이를 갖고, (8) 각각의 프로세서는 단일 스레드를 처리하며, (9) 하나의 마스터 프로세서가 각각의 병렬 프로세서를 관리하며, (10) 상기 최소 허용되는 인스트럭션 세트는 프로세서 상의 인스트럭션 시퀀스의 실행을 최적화하기 위한 다수의 인스트럭션 확장을 포함하며, 이러한 인스트럭션 확장은 20개 미만의 인스트럭션을 포함하고, (11) 각각의 인스트럭션 확장은 최대 8비트 길이를 가지며, (12) 상기 최소 허용되는 인스트럭션 세트는 칩 상의 다수의 프로세서를 선택적으로 제어하기 위한 인스트럭션의 세트를 포함하고, (13) 각각의 프로세서 제어 인스트럭션은 최대 8비트 길이를 가지며, (14)는 다수의 프로세서는, 모놀리식 메모리 디바이스용 반도체 제조 공정을 이용하여 칩 상에 위치하는 컴퓨터 메모리와 함께, 칩 상에서 제조된다.

[0097] 또 다른 양태에서, 본 발명은 다수의 병렬 프로세서, 마스터 프로세서 및 단일 칩 상의 컴퓨터 메모리를 이용하는 스레드-레벨 병렬 프로세싱의 방법을 포함하며, 여기서 상기 다수의 프로세서 각각은 최소 허용되는 인스트럭션 세트를 처리하고 단일 스레드를 처리하며, 상기 방법은, 상기 다수의 프로세서 각각에서 3개 이상의 특정 레지스터에 로컬 캐시를 할당하는 단계와, (b) 하나의 단일 스레드를 처리하기 위해 상기 다수의 프로세서 중 하나를 할당하는 단계와, (c) 상기 프로세서가 각각의 할당된 스레드를 처리하는 단계와, (d) 상기 프로세서에 의해 처리되는 각각의 스레드로부터의 결과를 처리하는 단계와, (e) 스레드가 처리된 후, 상기 각각 다수의 프로세서 중 하나를 역-할당하는 단계를 포함한다.

[0098] 다양한 실시예에서, (1) 상기 최소 허용되는 인스트럭션 세트는 필수적으로 7개의 기본 인스트럭션을 포함하며, (2) 상기 기본 인스트럭션은 ADD, XOR, INC, AND, STOREACC, LOADACC 및 LOADI 인스트럭션을 포함하며, (3) 상기 최소 허용되는 인스트럭션 세트는 다수의 프로세서를 선택적으로 제어하기 위한 인스트럭션의 세트를 포함하며, (4) 각각의 프로세서 제어 인스트럭션은 최대 8비트 길이를 가지며, (5) 상기 방법은 메모리의 내부 데이터 버스를 이용하여 각각의 프로세서가 컴퓨터 메모리에 액세스하는 단계를 더 포함하고, 이때 상기 내부 데이터 버스는 칩 상의 메모리의 하나의 로우의 폭에 상응하고, (6) 상기 최소 허용되는 인스트럭션 세트의 각각의 인스트럭션은 최대 8비트 길이를 갖는다.

[0099] 또 하나의 양태에서, 본 발명은 하나의 시스템을 포함하고, 상기 시스템은 (a) 전자 산업 표준 패키징과 호환되는 하나의 메모리 칩에 내장되는 다수의 프로세서 및 이러한 메모리 디바이스를 위한 핀 레이아웃과, (b) 메모리 칩의 메모리 모드 레지스터로 전송되는 정보를 통해 프로세서 중 하나 이상은 활성화될 수 있으며, 이때 상기 메모리 칩은, 프로세서 중 하나 이상이 메모리 모드 레지스터를 통해 활성화될 때를 제외하고는, 산업 표준 메모리 디바이스의 동작과 기능적으로 호환된다.

실시예

[0130] 본 발명의 하나 이상의 실시예의 TOMI 아키텍처는 범용 컴퓨터로서 동작하기 가능한 최소한의 로직을 이용하는 것이 바람직하다.

[0131] 가장 일반적인 연산이 우선순위를 부여받는다. 대부분의 연산이 가시적(visible)이며, 규칙적이고 컴파일러 최적화를 위해 이용가능하다.

[0132] 하나의 실시예에서, 도 1에서 도시된 바와 같이, TOMI 아키텍처는 어큐물레이터, 레지스터 및 스택 아키텍처의 변형이다. 이 실시예에서:

[0133] 1. 어큐물레이터 아키텍처처럼, 어큐물레이터는 항상, 증연산 인스트럭션(increment instruction)에 대해서는 제외하고, 피연산자 중 하나이다.

[0134] 2. 레지스터 아키텍처처럼, 도착지(destination)는 항상 피연산자 레지스터 중 하나이다.

- [0135] 3. 또한 상기 어큐뮬레이터와 프로그램 카운터는 레지스터 공간에 있으며, 따라서 그 곳에서 연산될 수 있다.
- [0136] 4. 3개의 특수 레지스터는 자동-증연산(auto-increment) 및 자동-감연산(auto-decrement)하며, 입력 및 출력의 스택 및 스트림을 생성하기 위해 유용하다.
- [0137] 5. 모든 인스트럭션은 8-비트의 길이를 가지며, 인스트럭션 디코딩을 단순화시키고, 속도를 높인다.
- [0138] 6. BRANCH, 또는 JUMP 인스트럭션은 없다.
- [0139] 7. 도 2에서 도시된 바와 같이, 8-비트 인스트럭션으로부터 3비트 연산자의 선택을 가능하게 하는, 단 7개의 기본 인스트럭션만 존재한다.
- [0140] 바람직한 실시예의 이점 중 일부는 다음과 같다:
- [0141] 1. 모든 연산이, 파이프라인에 의해 요구되는 동등성(equality)로 인해 제한되는 최대 속도가 아닌, 로직에 의해 허용되는 최대 속도로 실행된다. 논리 연산이 가장 빠르다. 수리 연산이 그 다음 빠르다. 메모리 액세스를 필요로 하는 연산이 가장 느리다.
- [0142] 2. 상기 아키텍처는 임의의 데이터 폭으로 스케일링되며, 패키지 핀, 가산기 캐리 시간 및 유용성에 의해서만 제한된다.
- [0143] 3. 상기 아키텍처는 범용 컴퓨터의 모든 연산을 수행하기에 필요한 최소 가능한 기능에 근접한다.
- [0144] 4. 상기 아키텍처는 매우 투명하며(very transparent), 매우 규칙적이며, 대부분의 연산이 최적화 컴파일러에 대해 이용가능하다.
- [0145] 상기 아키텍처는 단일 모놀리식 칩 상에서 여러 번 복제되기에 충분히 단순하도록 설계된다. 하나의 실시예는 CPU의 다수의 복사본을 메모리와 모놀리식하게 내장(embed)한다. 단순화된 32-비트 CPU는 1,500개 이하의 게이트로 구현될 수 있으며, 이때 게이트의 대부분이 레지스터를 형성한다. 하나의 Intel Pentium 4와 동일한 개수의 트랜지스터를 이용하여, 본 발명의 바람직한 실시예의 약 1,000개의 TOMI CPU가 구현될 수 있다.
- [0146] TOMI CPU의 감소된 인스트럭션 세트는, 범용 컴퓨터에서 요구되는 연산을 실행하기에 적합하다. 하나의 프로세서에 대한 인스트럭션 세트가 작아질수록, 더 효과적으로 실행될 것이다. TOMI CPU는, 최근의 프로세서 아키텍처와 비교할 때, 매우 낮은 개수의 인스트럭션을 갖도록 설계된다. 예를 들어, 286개의 인스트럭션을 갖는 Intel Pentium Processor, 195개의 인스트럭션을 갖는 Intel Itanium Montecito processor, 127개의 인스트럭션을 갖는 StrongARM processor, 400개 이상의 인스트럭션을 갖는 IBM Cell processor에 비교할 때, TOMI CPU의 하나의 실시예는 25개의 인스트럭션을 갖는다.
- [0147] 가장 최근 세대의 Pentium 프로세서가 30개의 클럭 사이클을 요구하는 것과 달리, TOMI CPU를 위한 인스트럭션의 기본 세트는 단일 시스템 클럭 사이클로 실행되도록 단순화되고, 설계된다. 상기 TOMI CPU 아키텍처는 “무(無)-파이프라인(pipelineless)” 아키텍처이다. 이 아키텍처 및 단일 클럭 사이클 인스트럭션 실행은, 다른 병렬처리, 또는 파이프라이닝된 아키텍처에서 발견되는 멈춤(stall), 종속(dependency) 및 낭비되는 클럭 사이클을 상당히 감소시키거나, 또는 제거시킨다. 기본 인스트럭션은 실행되기 위해 단일 클럭 사이클만 요구하지만, 클럭 속도가 증가함에 따라(그리고 클럭 사이클 시간이 감소함에 따라), 복잡한 수리적 인스트럭션(가령, ADD)을 위한 실행 결과가 회로 트랜지스터 게이트를 통해 전파되기 위해 필요한 시간은 단일 클럭 사이클의 한계에 도달할 수 있다. 이러한 경우, 더 빠른 인스트럭션의 실행의 속도를 늦추지 않기 위해, 특정 인스트럭션의 실행에 대해서는 2개의 클럭 사이클을 허용하는 것이 최적일 수 있다. 이는 시스템 클럭 속도, 제조 프로세스 및 회로 레이아웃에 대한 CPU 설계의 최적화에 따라 달라진다.
- [0148] TOMI 단순화된 인스트럭션 세트에 의해, 32-비트 TOMI CPU는 5,000개 미만의 트랜지스터(캐시는 미포함)로 구축될 수 있다. 단일 32-비트 TOMI CPU의 탑-레벨 개략도가 도 15A 내지 15D에서 도시되며, 신호 설명은 도 15E에서 도시된다. 가장 최근 세대의 Intel Pentium 마이크로프로세서 칩이 250,000,000개의 트랜지스터를 필요로 하는 것에 비교할 때, 32-비트 TOMI CPU는, 캐시 및 관련 디코드 로직을 포함해서라도, (CPU 캐시의 크기에 따라) 40,000 내지 200,000개의 트랜지스터를 이용하여 구축될 수 있다. 공지된 마이크로프로세서 아키텍처(가령, Intel Pentium, Itanium, IBM Cell 및 StrongARM 등)는 프로세싱 능력의 점증 증가라도 얻기 위해서는 방대한 개수의 트랜지스터를 필요로 한다. TOMI CPU 아키텍처는 CPU 코어 당 매우 낮은 개수의 트랜지스터를 이용함으로써, 이러한 산업의 진행을 부정한다. TOMI CPU에 대한 낮은 트랜지스터 총 개수는 많은 이점을 제공한다.
- [0149] TOMI CPU의 컴팩트한 크기 때문에, 다수의 CPU가 하나의 실리콘 칩 상에 구축될 수 있다. 또한 이로 인해서, 다

수의 CPU가, DRAM 등의 메인 메모리와 동일한 칩 상에, DRAM 칩 자체의 제조비용을 넘어서는 제조비용의 추가는 거의 없이, 구축될 수 있다. 따라서 DRAM 칩에 대한 다이 크기 및 제조비용이 최소한으로 증가된 채, 다수의 TOMI CPU가 병렬 프로세싱을 위해 단일 칩 상에 위치할 수 있다. 예를 들어, 512MB DRAM은 약 7억 개의 트랜지스터를 포함한다. 64개의 TOMI CPU(하나의 TOMI CPU가 200,000개의 트랜지스터를 포함한다고 가정)는 임의의 DRAM 설계에 1280만 개의 트랜지스터만 추가할 것이다. 512MB DRAM에 있어서, 64개의 TOMI CPU는 다이 크기를 5% 이하만큼만 증가시킬 것이다.

[0150] TOMI CPU는 DRAM, SRAM 및 FLASH 메모리 디바이스 등의 비용이 낮은 기존의 기성품(commodity) 메모리 제조 프로세스를 이용하여 제조되도록 설계된다. TOMI CPU에 대한 낮은 트랜지스터 총 개수는 CPU가 작은 면적 내에서 구축될 수 있음을 의미하고, 8개 이상의 금속 인터커넥트의 층(layer)을 이용하는 큰 마이크로프로세서 칩(가령, Intel Pentium)을 제조하기 위한 복잡하고 비용이 높은 제조 공정, 또는 그 밖의 다른 로직 프로세스가 아닌, 2개의 금속 인터커넥트의 층을 이용하는 비용이 낮은 반도체 제조 공정을 이용하여 실리콘으로 인터커넥트되기 쉽다는 것을 의미한다. 최근의 DRAM 및 그 밖의 다른 기성품 메모리 칩은, 더 낮은 제조비용, 더 높은 프로덕트 볼륨(product volume), 더 높은 수율을 성취하기 위해, 더 낮은 개수(가령 2)의 금속 인터커넥트 층을 이용하는 더 단순하며, 더 낮은 비용의 반도체 제조 공정을 이용한다. 최근의 마이크로프로세서를 구축하기 위해 사용되는 공정은 트랜지스터-레벨에서의 저-전류 누설 값보다는, 높은 속도와 높은 성능 특성을 구하는 반면에, 기성품 메모리 디바이스를 위한 반도체 제조 공정은 저-전류 누설 디바이스 작업을 특징으로 갖는다. DRAM 및 그 밖의 다른 메모리 디바이스용으로 사용되는 것과 동일한 제조 공정을 이용하여 TOMI CPU가 효과적으로 구현될 수 있음으로써, 상기 TOMI CPU는 기존의 DRAM 칩(또는 그 밖의 다른 메모리 칩) 내에 내장되어, 저비용의, 높은 수율의 메모리 칩 제조 공정을 이용할 수 있다. 또한 이는, 현재 DRAM 및 그 밖의 다른 메모리 칩 산업에서 사용되는 것과 동일한 패키징 및 디바이스 핀 레이아웃(예를 들어, 메모리 디바이스를 위한 JEDEC 표준에 부합하는), 조립 설비, 특성 측정 장치(test fixture), 테스트 벡터를 이용하여 TOMI CPU가 제조될 수 있다는 이점을 제공한다. 바람직한 실시예는 CPU가 기존의 DRAM으로 내장(embedding)되는 것을 기술한다. 이는 DRAM을 기존의 마이크로프로세서에 내장하는 것에 대조된다. 본 발명의 바람직한 실시예에 비교되는 트랜지스터의 높은 총 개수 때문에, 종래의 마이크로프로세서는 내장된 DRAM의 성능과 신뢰성에 악영향을 미칠 수 있는 훨씬 더 큰 수준의 전기적 노이즈 및 열을 발생시킬 가능성이 높을 것이다. 덧붙이자면, 종래의 마이크로프로세서 상에 내장된 DRAM은, 바람직한 실시예의 2개의 층에 비교되는 8개 이상의 금속 인터커넥트 층을 포함하는 공정을 필요로 할 가능성이 높을 것이다. 비교하자면, 종래의 마이크로프로세서에 내장되는 최종 DRAM은 더 높은 비용, 더 낮은 수율, 더 높은 전력 소모율, 그리고 궁극적으로는 더 낮은 성능을 가질 가능성이 높다.

[0151] 바람직한 실시예의 또 다른 이점은, TOMI CPU가 충분히 작아서(그리고 거의 전력을 소비하지 않아서), DRAM 회로 옆에 물리적으로 위치할 수 있고, CPU가 초 광대역(ultra-wide)의 내부 DRAM 데이터 버스로 액세스할 수 있다. 최근의 DRAM에서, 이 버스는 1024, 4096, 또는 8192비트(또는 이들의 정수배)의 폭을 가지며, 상기 폭은 통상적으로, DRAM 설계 내의 데이터 뱅크의 하나의 로우(row)의 데이터의 폭에 대응한다. (비교하자면, Intel Pentium 데이터 버스는 64비트 폭을, Intel Itanium 버스는 128 비트 폭을 갖는다) TOMI CPU의 내부 캐시의 크기는 DRAM 로우(row)의 크기에 매칭되도록 정해져서, 하나의 단일 DRAM 메모리 관독, 또는 기록 사이클로 CPU 캐시가 채워질(즉, 플러쉬(flush)될) 수 있다. 상기 TOMI CPU는, TOMI CPU에 대한 데이터 버스로서 초-광대역 내부 DRAM 데이터 버스를 이용한다. 효과적인 레이아웃과 회로 동작을 위해, 가령, TOMI CPU 캐시로의 데이터 전송을 포함하여, 상기 TOMI CPU 캐시는 DRAM 로우 및/또는 컬럼 래치 회로의 설계를 반영하도록 설계될 수 있다.

[0152] 바람직한 실시예의 또 하나의 이점은, TOMI CPU가 발생하는 전기적 노이즈의 낮은 수준인데, 이는 낮은 트랜지스터 총 개수 때문이고, CPU가 오프-칩 메모리를 액세스하기 위해 I/O 회로를 일정하게 구동시키기 보다는, 메모리를 액세스하기 위해 초-광대역 내부 DRAM 데이터 버스를 이용하기 때문이다. 온-칩 CPU 캐시는 프로세싱되기 위한 데이터의 순간적인 액세스를 가능하게 하며, 따라서 오프-칩 메모리 액세스에 대한 필요성을 최소화할 수 있다.

[0153] 본 발명의 프로세서 아키텍처의 설계 목적은, 프로세싱 용량(processing capacity)과 속도를 최대화하면서, 상기 프로세싱 속도를 얻기 위해 요구되는 전력은 최소화하는 것이다. TOMI CPU 아키텍처는 극도로 낮은 전력 소모량을 갖는 고속 프로세서(high-speed processor)이다. 프로세서의 전력 소모량은 설계 시 사용되는 트랜지스터의 개수에 직접 관련이 있다. TOMI CPU에 대한 낮은 트랜지스터 총 개수는 전력 소모량을 최소화한다. 또한, 단순화되고, 효율적인 인스트럭션 세트에 의해, TOMI CPU는 자신의 전력 소모량을 감소시킬 수 있다. 덧붙이자면, TOMI CPU는 광대역 내부 DRAM 데이터 버스를 이용하여 온-칩 메모리를 캐싱하고 액세스하여, 오프-칩 메모

리 액세스를 위해 I/O 회로를 일정하게 구동시키는 것에 대한 필요성이 제거된다. 1GHz 클록 속도에서 동작하는 하나의 단일 TOMI CPU는 대략 20 내지 25 밀리와트(milliwatt)의 전력을 소모할 것이다. 이와 달리, Intel Pentium 4 프로세서는 2.93GHz에서 130와트를 필요로 하고, Intel Itanium 프로세서는 1.6GHz에서 52와트를 필요로 하고, StrongARM 프로세서는 200MHz에서 1와트를 필요로 하며, IBM Cell 프로세서는 3.2GHz에서 100와트를 필요로 한다. 프로세서에서의 열 발생은 프로세서가 필요로 하는 전력의 크기에 비례함은 잘 알려져 있다. 극도로 낮은 전력의 TOMI CPU 아키텍처에 의해, 현재 마이크로프로세서 아키텍처에서 발견되는 팬(fan), 큰 열 흡수원(heat sink), 외부 냉각 수단이 필요하지 않게 된다. 동시에, 저-전력 TOMI CPU 아키텍처에 의해, 새로운 저-전력 배터리 및 태양열에 의해 전력이 공급되는 적용예가 실현가능해진다.

[0154] 인스트럭션 세트

[0155] 예시적 인스트럭션 세트 중 7개의 기본 인스트럭션이, 이들의 비트 매핑(bit mapping)과 함께, 도 2에서 도시된다. 각각의 인스트럭션은 하나의 단일 8-비트 워드로 구성되는 것이 바람직하다.

[0156] 주소 지정 방식

[0157] 도 3은 여러 다른 주소 지정 방식의 유효 주소를 도시한다.

[0158] 상기 주소 지정 방식은,

[0159] 즉시(immediate) 주소 지정 방식,

[0160] 레지스터(register) 주소 지정 방식,

[0161] 레지스터 간접(register indirect) 주소 지정 방식,

[0162] 레지스터 간접 자동-증연산(register indirect auto-increment) 주소 지정 방식,

[0163] 레지스터 간접 자동-감연산(register indirect auto-decrement) 주소 지정 방식

[0164] 이다.

[0165] 특수 경우

[0166] 레지스터 0 및 레지스터 1은 모두 프로그램 카운터(PC)를 일컫는다. 하나의 실시예에서, 피연산자로서 레지스터 0(PC)과 이뤄지는 모든 연산은 어큐뮬레이터 캐리 비트(C)가 1일 때를 조건으로 한다. C = 1인 경우, PC의 예전 값이 어큐뮬레이터(ACC)로 교환(swapped)된다. 피연산자로서 레지스터 1(PC)과 이뤄지는 모든 연산은 조건 없이 이뤄진다.

[0167] 대안적 실시예에서, 도착지(destination)로서 레지스터 0(PC)과 이뤄지는 기록 연산은 캐리 비트(C)가 0일 때를 조건으로 한다. C = 1이면, 어떠한 연산도 수행되지 않는다. C = 0인 경우, 어큐뮬레이터(ACC)의 값이 PC로 기록(write)되며, 프로그램 컨트롤이 새 PC 주소로 변경된다. 도착지로서 레지스터 1(PC)과 이뤄지는 기록 연산은 조건 없이 이뤄진다. 어큐뮬레이터(ACC)의 값이 PC로 기록되며, 프로그램 컨트롤은 새 PC 주소로 변경된다.

[0168] 출발지(source)로서 레지스터 0과 이뤄지는 판독(read) 연산은 PC 값에 +3을 로딩한다. 이러한 방식으로, 루프의 탑(top)의 주소가 판독되고, 추후 사용을 위해 저장될 수 있다. 대부분의 경우에서, 루프의 주소가 스택(S)에 푸시(push)될 것이다. 출발지로서 레지스터 1과 이뤄지는 판독 연산이, PC에 의해 주소 지정된 다음번 풀 워드에 의해 참조되는 값을 로딩한다. 이러한 방식으로, 32-비트 즉시 피연산자가 로딩될 수 있다. 32-비트 즉시 피연산자는 워드 정렬(word align)되지만, LOADACC 인스트럭션은, 32-비트 즉시 피연산자 바로 앞에 위치하는 4-바이트 워드 내 임의의 바이트 위치에 위치할 수 있다. 판독의 실행 후에, PC는 증연산되어, 상기 32-비트 즉시 피연산자 뒤에 위치하는 첫 번째 워드 정렬되는 인스트럭션을 주소 지정할 것이다.

[0169] 브랜치 없음(There Is No Branch)

[0170] 보통 브랜치(Branch) 및 점프(Jump) 연산은 CPU 설계자들에게 있어 문제거리인데, 왜냐하면, 이들은 귀중한 op-

코드 공간의 다수의 비트를 필요로 하기 때문이다. LOADACC, xx 인스트럭션을 이용하여 요망 브랜치 주소를 ACC로 로딩하고, STOREACC, PC 인스트럭션을 이용하여, 브랜치를 유효함으로써, 브랜치 생성 함수(branching function)가 만들어질 수 있다. 브랜치는, 레지스터 0으로 저장함에 따른 C의 상태에 따라 조건적으로 만들어질 수 있다.

[0171] 스킵(Skip)

[0172] 스킵은 INC, PC를 실행함으로써 생성될 수 있다. 실행은 2개의 사이클을 필요로 할 것인데, 하나는 현재 PC 증연산 사이클의 종료를 위한 것이며, 다른 하나는 INC를 위한 것이다. 스킵은 레지스터 0을 증연산함에 따른 C의 상태에 따라 조건적으로 만들어질 수 있다.

[0173] 상대 브랜치(Relative Branch)

[0174] ACC로 요망 오프셋을 로딩하고, ADD, PC 인스트럭션을 실행함으로써, 상대 브랜치가 생성될 수 있다. 상대 브랜치는 레지스터 0으로의 가산 연산(adding)에 의한 C의 상태에 따라 조건적으로 만들어질 수 있다.

[0175] 순방향 브랜치(Forward Branch)

[0176] 순방향 브랜치는 역방향 브랜치보다 더 유용한데, 왜냐하면 루프의 탐을 최초로 통과하는 프로그램 단계로서 PC를 저장함으로써, 루프에 대해 필수적인 역방향 브랜치의 위치는 쉽게 캡처되기 때문이다.

[0177] 브랜치의 종점(endpoint)의 최하위 비트를 ACC로 로딩하고, PC로 저장함으로써, 상대 브랜치보다 더 효과적인 순방향 브랜치가 생성될 수 있다. 레지스터 0, 또는 레지스터 1의 사용에 따라, PC는 조건적으로, 또는 무조건적으로 액세스될 수 있기 때문에, PC 레지스터(레지스터 0, 또는 레지스터 1)를 도착지 피연산자로서 선택함에 따라, 상기 순방향 브랜치가 또한 조건적, 또는 무조건적일 수 있다. 예를 들어,

[0178] LOADI, #1C

[0179] STOREACC, PC

[0180] ACC의 최상위 비트가 0인 경우, 최하위 6비트만 PC 레지스터로 전송된다. 현재 PC 레지스터의 최하위 6비트가 로딩될 ACC 값보다 작은 경우, 레지스터의 최상위 비트는 변경되지 않은 채 유지된다. 현재 PC 레지스터의 최하위 6비트가 로딩될 ACC 값보다 큰 경우, 현재 PC 레지스터는 7번째 비트에서 시작하여 증연산된다.

[0181] 이는 효과적으로 31개의 인스트럭션까지의 순방향 브랜치를 가능하게 한다. 순방향 브랜치 생성을 위한 이러한 방법은 가능할 때면 언제나 사용되어야 하는데, 왜냐하면 상대 브랜치를 위한 3개의 인스트럭션에 비해, 2개의 인스트럭션만 필요로 할 뿐 아니라, 가장 느린 연산 중 하나인 가산기를 통과할 것을 필요로 하지 않기 때문이다. 도 2A는 연산 중의 순방향 브랜치를 나타낸다.

[0182] 루프(Loop)

[0183] LOADACC, PC를 이용하여 루프의 탐이 저장된다. 그 후, 루프 아키텍처의 탐을 가리키는 최종 포인터가 레지스터에 저장되거나, 자동인덱싱 레지스터(autoindexing register) 중 하나에 푸시(push)될 수 있다. 루프의 바텀(bottom)에서, LOADACC, EA를 이용하여 포인터가 불러와지고(retrieve), STOREACC, PC를 이용하여 PC로 재-저장되며, 이로 인해서, 역방향 루프가 발생할 수 있다. 상기 루프는, 레지스터 0을 저장함에 따른 C의 상태에 따라 조건적으로 만들어질 수 있으며, 이로 인해서 조건적 역방향 루프가 발생할 수 있다.

[0184] 자기 수정 코드(Self Modifying code)

[0185] STOREACC, PC를 이용하여 자기-수정 코드를 기록하는 것이 가능하다. 인스트럭션이 생성, 또는 ACC로 인출(fetch)되고, PC에 저장될 수 있으며, 여기서 다음 인스트럭션으로서 실행될 것이다. 이러한 기법은 CASE 아키텍처를 생성하기 위해 사용될 수 있다.

- [0186] N개의 주소와 JUMPTABLE의 기본 주소(base address)로 구성되는 메모리의 점프 테이블 어레이를 가정하자. 편의를 위해, JUMPTABLE은 낮은 메모리(low memory)에 위치하여, 그 주소가 LOADI를 이용하여, 또는 LOADI와 하나 이상의 우향 쉬프트(right shift), ADD, ACC를 이용하여 생성될 수 있다.
- [0187] 점프 테이블의 인덱스가 ACC 내에 존재하며, 상기 점프 테이블의 기본 주소는 JUMPTABLE이라는 명칭의 범용 레지스터 내에 존재한다고 가정하자.
- [0188] ADD, JUMPTABLE 점프 테이블의 기본 주소에 인덱스를 더하기
- [0189] LOADACC, (JUMPTABLE) 인덱싱된 주소를 로딩하기
- [0190] STOREACC, PC 점프를 실행하기
- [0191] 0000에서 시작하는 저차 메모리가 시스템 호출(system call)에 할당되는 경우, 각각의 시스템 호출은 다음과 같이 실행될 수 있으며, 이때, SPECIAL FUNCTION은 즉시 피연산자 0-63의 이름이다.
- [0192] LOADI, SPECIAL FUNCTION 시스템 호출 번호를 로딩하기
- [0193] LOADACC, (ACC) 시스템 호출의 주소를 로딩하기
- [0194] STOREACC, PC 함수로 점프하기
- [0195] **우향 쉬프트(Right Shift)**
- [0196] 본 발명의 기본 아키텍처는 우향 쉬프트 연산을 고려하지 않는다. 이러한 연산이 필요한 경우, 바람직한 실시예의 해결책은 범용 레지스터 중 하나를 “우향 쉬프트 레지스터”로 지정하는 것이다. STOREACC, RIGHTSHIFT는 ACC의 우향 쉬프트된 단일 위치를 “우향 쉬프트 레지스터”로 저장할 것이며, 여기서 그 값은 LOADACC, RIGHTSHIFT를 이용하여 판독될 수 있다.
- [0197] **아키텍처 확장성(Architectural Scalability)**
- [0198] TOMI 아키텍처는 8-비트 인스트럭션을 특징으로 하는 것이 바람직하지만, 데이터 폭이 제한될 필요는 없다. 도 4는 4 내지 32비트의 임의의 폭 데이터 경로가 얼마나 쉽게 생성되는가를 도시한다. 더 큰 폭의 데이터를 생성하는 것은 레지스터 세트의 폭, 내부 데이터 경로 및 ALU를 희망 폭으로 증가시키는 것만 필요로 한다. 데이터 경로의 상한은 가산기의 캐리 전파 딜레이와 트랜지스터 예산에 의해서만 제한된다.
- [0199] 단순성을 위해 바람직한 TOMI 아키텍처가 폰 노이만 메모리 구성(Von Neumann memory configuration)으로 구현되지만, (개별 데이터 및 인스트럭션 버스를 갖는) 하바드 아키텍처(Harvard architecture) 구현도 또한 가능하다.
- [0200] **일반 수리 연산(Common Math Operation)**
- [0201] 2의 보수 수식은 몇 가지 방식으로 해결될 수 있다. 하나의 범용 레지스터가 모두 “1”이라고 사전에 구성될 수 있으며, ALLONES이라고 명명될 수 있다. 피연산자는 OPERAND라는 명칭의 레지스터 내에 있다고 가정된다:
- [0202] LOADACC, ALLONES
- [0203] XOR, OPERAND
- [0204] INC, OPERAND OPERAND에 2의 보수가 남겨진다.
- [0205] **일반적인 컴파일러 구성(Common Compiler construction)**
- [0206] 대부분의 컴퓨터 프로그램은 컴파일러에 의해 생성된다. 따라서 유용한 컴퓨터 아키텍처는 일반적인 컴파일러 아키텍처에 적응되어야 할 것이다.
- [0207] 일반적으로 C 컴파일러는 매개변수를 함수 호출로 전달시키기 위한 스택을 유지할 것이다. S, X, 또는 Y 레지스

터가 스택 포인터로서 사용될 수 있다. 함수 호출은, 예를 들어, STOREACC, (X)+를 이용하여, 스택으로서 기능하는 자동인덱싱 레지스터 중 하나에 매개변수를 푸시(push)할 것이다. 함수가 시작되면, 매개변수는 사용되기 위해 범용 레지스터로 팝(pop)될 것이다.

[0208] **스택 상대 주소 지정(Stack Relative Addressing)**

[0209] 범용 레지스터에 알맞게 딱 맞는 것보다 함수 호출에서 전달되는 더 많은 요소가 존재하는 시점이 존재할 것이다. 다음의 예에서, 스택 푸시 연산이 스택을 감연산(decrement)한다고 가정하자. S가 스택 레지스터로서 사용되는 경우, 스택의 탑(top)에서부터 N번째 아이템을 판독하기 위해,

[0210] LOAD, N

[0211] STOREACC, X

[0212] LOADACC, S

[0213] ADD, X

[0214] LOADACC, (X) 10

[0215] **어레이로 인덱싱하기(Indexing Into an Array)**

[0216] 어레이 함수가 시작되면, ARRAY라는 명칭의 범용 레지스터에서 어레이의 기본 주소의 위치가 파악된다. 어레이의 N번째 요소를 판독하기 위해,

[0217] LOADI, N

[0218] STOREACC, X

[0219] LOADACC, ARRAY

[0220] ADD, X

[0221] LOADACC, (X)

[0222] **N 워드 요소 어레이로 인덱싱하기(Indexing Into an N Word Element Array)**

[0223] 가끔 N 워드 폭의 요소에 대해 어레이가 할당될 것이다. ARRAY라는 명칭의 범용 레지스터에서 상기 어레이의 기본 주소의 위치가 파악된다. 5 워드 폭 어레이에서 N번째 요소의 첫 번째 워드를 액세스하기 위해,

[0224] LOADI, N

[0225] STOREACC, X 임시 레지스터에 저장하기

[0226] ADD, ACC 2를 곱하기

[0227] ADD, ACC 다시 2를 곱하기(=4)

[0228] ADD, X 1을 더하기(=5)

[0229] LOADACC, ARRAY

[0230] ADD, X 어레이의 기본 주소를 더하기

[0231] LOADACC (X)

[0232] **인스트럭션 세트 확장(Instruction Set Extension)**

[0233] 본 발명의 또 하나의 실시예는 도 2에서 나타난 7개의 기본 인스트럭션의 확장을 포함한다. 도 2E에서 나타난

인스트럭션 세트 확장은 TOMI 프로세서의 내부 연산, TOMI 프로세서를 위한 소프트웨어 프로그램 인스트럭션 및 소프트웨어 컴파일러를 추가로 최적화하는데 도움이 된다.

- [0234] SAVELOOP - 이 인스트럭션은 프로그램 카운터의 현재 값을 스택에 푸시(push)한다. SAVELOOP은 루프 아키텍처의 탑(top)에서 실행될 가능성이 가장 높다. 루프의 바텀(bottom)에서, 저장된 프로그램 카운터 값 10은 스택으로부터 복사되어, 프로그램 카운터로 저장되어, 상기 루프의 탑으로의 역방향 점프(backward jump)를 개시한다.
- [0235] SHIFTLOADBYTE - 이 인스트럭션은 ACC를 왼쪽으로 8비트 쉬프트시키고, 8-비트 바이트 다음의 인스트럭션을 판독하고, 이를 ACC의 최하위 8-비트에 위치시킨다. 이러한 방식으로, 인스트럭션의 시퀀스를 이용하여 긴 즉시 피연산자가 로딩될 수 있다. 예를 들어, 14-비트 즉시 피연산자를 로딩하기 위해,
- [0236] LOAD, #14 \ \ 14 비트 피연산자의 최상위 6비트를 로딩하기
- [0237] SHIFTLOADBYTE \ \ 상기 6비트를 왼쪽으로 8위치만큼 쉬프트하고, 다음의 8-비트 값을 로딩하기
- [0238] CONSTANT #E8 \ \ 8비트 즉시 피연산자
- [0239] ACC의 최종 16진수 값은 14E8이다.
- [0240] LOOP - 이 인스트럭션은 스택의 탑을 프로그램 카운터에 복사한다. LOOP은 루프의 탑에서의 프로그램 카운터를 저장하기 위한 SAVELOOP의 실행에 뒤 이어, 루프 아키텍처의 바텀(bottom)에서 실행될 가능성이 가장 높을 수 있다. LOOP가 실행될 때, 저장된 프로그램 카운터가 스택으로부터 복사되고, 프로그램 카운터로 저장되어, 루프의 탑으로의 역방향 점프가 개시될 것이다.
- [0241] LOOP_IF - 이 인스트럭션은 스택의 탑을 프로그램 카운터로 복사한다. 이 인스트럭션은 C의 값을 바탕으로 하는 조건부 루프(conditional loop)를 수행한다. LOOP_IF는, 루프의 탑에서의 프로그램 카운터를 저장하기 위한 SAVELOOP의 실행에 뒤 이어서, 루프 아키텍처의 바텀에서 실행될 가능성이 가장 높을 것이다. LOOP_IF가 실행될 때, C=0이면, 저장된 프로그램 카운터가 스택으로부터 복사되고, 프로그램 카운터로 저장되어, 루프의 탑으로의 역방향 점프가 개시될 것이다. C=1인 경우, 상기 프로그램 카운터는 증연산(increment)되어, 다음 순차의 인스트럭션을 가리킬(point) 수 있다.
- [0242] NOTACC - ACC의 각각의 비트를 보수화(complement)한다. ACC=0인 경우, C를 1로 설정한다. 그렇지 않은 경우, C를 0으로 설정한다.
- [0243] ROTATELEFT8 - ACC를 왼쪽으로 8비트만큼 회전한다. 각각의 회전 단계에서, ACC 밖으로 쉬프트되는 MSB가 ACC의 LSB로 쉬프트된다.
- [0244] ORSTACK - ACC와 스택의 탑의 값과의 논리 OR을 수행한다. 그 결과를 ACC에 놓는다. ACC=0인 경우, C는 1로 설정된다. 그렇지 않은 경우, C는 0으로 설정된다.
- [0245] ORSTACK+ - ACC와 스택의 탑의 값과의 논리 OR을 수행한다. 그 결과를 ACC에 놓는다. 논리 연산 후에, 스택 포인터 S가 증연산된다. ACC=0인 경우, C는 1로 설정된다. 그렇지 않은 경우, C는 0으로 설정된다.
- [0246] RIGHTSHIFTACC - ACC를 오른쪽으로 1비트만큼 쉬프트한다. ACC의 LSB는 C로 쉬프트된다.
- [0247] SETMSB - ACC의 최상위 비트를 설정한다. C에 대한 변화는 없다. 이 인스트럭션은 부호 타입 비교를 수행할 때 사용된다.
- [0248] **로컬 TOMI 캐싱(Local TOMI Caching)**
- [0249] 캐시는 메인 메모리보다 더 작은 크기와 더 빠른 액세스 시간을 갖는 메모리이다. 감소된 액세스 시간과, 프로그램 및 데이터 액세스의 로컬리티(locality)로 인해서, 캐시 연산은 많은 연산에 대해 바람직한 TOMI 프로세서의 성능을 증가시킬 수 있다. 또 다른 관점에서, 메인 메모리에 대한 TOMI 프로세서의 독립성을 증가시킴으로써, 캐시는 병렬 프로세싱 성능을 증가시킨다. 메인 메모리에 대한 캐시의 상대적 성능과, 또 다른 메인 메모리가 상기 캐시로부터의 로딩, 또는 캐시로의 저장을 요청하기 전에 상기 TOMI 프로세서가 실행할 수 있는 사이클의 횟수가, TOMI 프로세서의 병렬처리로 인한 성능 증가의 크기를 결정한다.
- [0250] TOMI 로컬 캐시가 TOMI 프로세서의 병렬처리로 인한 성능 증가를 강화한다. 도 5에서 도시된 바와 같이, 각각의 TOMI 프로세서는 3개의 연계된 로컬 캐시를 갖는 것이 바람직하다:

- [0251] 인스트럭션 - PC와 연계
- [0252] 출발지(Source) - X 레지스터와 연계
- [0253] 도착지(Destination) - Y 레지스터와 연계
- [0254] “데이터” 나 “인스트럭션” 이 인출되는 것이 아닌, 캐시가 특정 레지스터와 연계되어 있기 때문에, 캐시 제어 로직이 단순화되고, 캐시 지연시간(cache latency)이 상당히 감소된다. 캐시의 최적 크기는 경우에 따라 다르다. 통상의 실시예는 각각의 캐시에 대해 1024바이트(byte)를 요구할 수 있다. 다시 말하자면, 1024개의 인스트럭션, 256개의 32-비트 워드 출발지 및 도착지이다. 둘 이상의 요인이 캐시의 최적 크기를 결정한다. 첫째는, 또 다른 캐시의 로드, 또는 저장 연산이 요청되기 전에 TOMI 프로세서가 사이클링할 수 있는 상태(state)의 개수이다. 둘째는, 메인 메모리 연산 동안 가능한 TOMI 프로세서 실행 사이클의 횟수에 대한, 메인 메모리로부터의 캐시의 로드, 또는 저장 연산의 비용이다.
- [0255] **TOMI 프로세서를 RAM에 내장하기(Embedding TOMI Processors in RAM)**
- [0256] 하나의 실시예에서, 광대역 버스(wide bus)가 큰 내장 메모리를 캐시로 연결하여, 캐시로의 로드, 또는 저장 연산이 빠르게 발생할 수 있다. RAM에 내장되는 TOMI 프로세서를 이용하여, 전체 캐시의 로드, 또는 저장은 RAM의 컬럼(column)에 대해 하나의 메모리 사이클로 이뤄질 것이다. 하나의 실시예에서, 내장된 메모리는 63 TOMI 프로세서의 요청에 응답하며, 이에 따라서, 하나의 TOMI 프로세서 캐시의 로드, 또는 저장이 완료되는 동안, 또 하나의 TOMI 프로세서에 대한 캐시 로드, 또는 저장의 응답 시간이 연장될 수 있다.
- [0257] 도 6에서 도시된 바와 같이, 캐시는 연계된 메모리 주소 지정 레지스터 X, Y, PC의 변화를 바탕으로, 저장 및 로드될 수 있다. 예를 들어, PC 레지스터의 총 폭은 24비트일 수 있다. PC 캐시가 1024바이트인 경우, PC의 하위 10비트는 PC 캐시 내에서의 액세스를 정의할 것이다. 상위 14비트의 변화가 존재하도록 PC가 기록될 때, 캐시 로드 사이클이 요청될 것이다. 캐시 로드 사이클이 완료되고, 지시된 인스트럭션이 PC 캐시로부터 인출될 수 있을 때까지, 상기 PC 캐시와 연계되는 TOMI CPU는 실행을 중단할 것이다.
- [0258] **캐시 이중 버퍼링(Cache Double Buffering)**
- [0259] 보조 캐시가 캐시 로드 요구치를 예상하여 로드될 수 있다. 2개의 캐시는 동일하고, PC의 상위 14비트의 내용을 바탕으로, 선택 및 선택해제될 것이다. 앞의 예에서, 보조 캐시에 선-캐싱된(pre-cached) 데이터의 상위 14비트와 일치하도록 PC의 상위 14비트가 변경될 때, 상기 보조 캐시가 주 캐시로서 선택될 것이다. 이전의 주 캐시는 보조 캐시가 될 것이다. 대부분의 컴퓨터 프로그램이 메모리를 선형으로 증가시키기 때문에, 본 발명의 하나의 실시예는, 현재 PC 더하기 1의 상위 14비트에 의해 참조되는 메인 메모리의 캐시 내용의 내용을 항상 인출하는 보조 캐시를 가질 것이다.
- [0260] 보조 캐시를 추가함으로써, 현재 캐시의 경계 밖으로 이동할 때, 메인 메모리로부터 메모리 데이터가 인출되는 것을 TOMI 프로세서가 대기해야 하는 시간이 감소될 것이다. 보조 캐시의 추가는 TOMI 프로세서의 복잡도를 거의 2배로 증가시킨다. 최적 시스템에 있어서, 이러한 복잡도의 2배 증가는 이에 따른 TOMI 프로세서의 성능의 2배 증가가 상쇄해야 할 것이다. 그렇지 않은 경우, 보조 캐시가 없는 2개의 더 단순한 TOMI 프로세서가 동일한 트랜지스터 총 개수를 갖고 구현될 수 있다.
- [0261] **고속 곱셈, 부동 소수점 연산, 추가적인 기능(High Speed Multiply, Floating Point Operations, Additional Functionality)**
- [0262] 정수 곱셈 및 모든 부동 소수점 연산은, 특수 목적의 하드웨어를 이용할 때조차, 많은 사이클이 수행될 것을 필요로 한다. 따라서 이들 연산은, 기본 TOMI 프로세서에 포함되기보다는 다른 프로세서로 분할(factor)될 수 있다. 그러나 (1000개 이하의 트랜지스터를 이용하는) 단순한 16비트x16비트 곱셈기(multiplier)가 TOMI CPU에 추가되어, TOMI CPU 아키텍처에 추가적인 기능을 제공하고, 융통성을 제공할 수 있다.
- [0263] 디지털 신호 프로세싱(DSP: Digital Signal Processing) 연산은 종종, 총 곱셈이 다수의 사이클을 요구할지라도, 매 사이클마다 결과를 생성하는 깊게 파이프라이닝된 곱셈기를 사용한다. 동일한 알고리즘을 계

속하여 반복하는 신호 프로세싱 애플리케이션에 있어서, 이러한 곱셈기의 아키텍처는 최적이고, 주변 프로세서(peripheral processor)로서, TOMI 프로세서로 통합될 수 있다. 그러나 이러한 곱셈기가 TOMI 프로세서로 직접 통합된다면, 복잡도를 증가시키고, 전체 성능을 떨어뜨릴 가능성이 높다. 도 7A는 광대역 시스템의 RAM 버스를 이용하는 구성된 추가적인 프로세싱 기능의 하나의 예를 도시한다.

[0264] 인접 메모리 뱅크의 액세스(Accessing Adjacent Memory Banks)

[0265] 메모리 칩 내의 메모리 회로의 물리적 레이아웃은 종종, 메모리 트랜지스터가 메모리 셀의 큰 뱅크로 레이아웃 되도록, 설계된다. 상기 뱅크는 동일한 크기의 사각형 영역으로 구성되고, 칩 상에서 둘 이상의 컬럼(column)으로 위치한다. 셀의 큰 뱅크에서의 메모리 셀의 레이아웃은 메모리 판독 및/또는 기록 액세스의 속도를 높이기 위해 사용될 수 있다.

[0266] 본 발명의 하나의 실시예에서, 둘 이상의 TOMI 프로세서는 메모리 칩 내의 메모리 셀 뱅크의 2개의 컬럼 사이에 위치할 수 있다. 도 7B에서 도시되는 로직을 이용하여, TOMI 프로세서가 셀렉트 A, 또는 셀렉트 B를 활성화함으로써 메모리 뱅크 A, 또는 메모리 뱅크 B를 액세스할 수 있도록, 2개의 메모리 뱅크의 로우(row) 데이터 라인이 인터리빙(interleave)될 수 있다. 이러한 방식으로, 메모리 칩의 특정 TOMI 프로세서에 의해 직접 주소 지정될 수 있는 메모리는 이중화될 수 있다.

[0267] TOMI 인터럽트 전략

[0268] 인터럽트는 프로세서의 보통의 순차적 연산 외부에서 발생하는 이벤트로서, 프로세서가 자신의 연산의 시퀀스를 즉시 변경하게 한다. 인터럽트의 예로는, 외부 장치에 의한 연산의 종료, 또는 일부 하드웨어에 의한 에러 상태가 있을 수 있다. 전통적인 프로세서는 인터럽트에 의해 야기되는 이벤트는 무엇이든지 다루기 위해, 보통의 순차적 연산을 빨리 중단하고, 현재 연산의 상태를 저장하고, 임의의 특수 연산을 수행하기를 시작하며, 상기 특수 연산이 완료될 때, 이전 상태 및 계속되는 순차적 연산을 복구하도록 노력한다. 인터럽트 핸들링 품질의 주요 척도는 응답 시간이다. 전통적인 프로세서에서 인터럽트는 몇 가지 문제점을 드러낸다. 인터럽트는 실행 시간을 불명확하게 만든다. 이들은 상태를 저장하고, 나중에 복구하는 프로세서 사이클을 낭비한다. 이들은 프로세서 설계를 복잡하게 하며, 모든 프로세서 연산을 느리게 하는 지연(delay)을 유도할 수 있다.

[0269] 에러 핸들링의 경우와 프로세서가 실세계 활동과 직접 인터페이싱하는 경우를 제외하고, 즉각적인 인터럽트 반응은 대부분의 프로세서에게 불필요하다.

[0270] 마이크로프로세서 TOMI 시스템의 하나의 실시예에서, 단 하나의 프로세서만 주 인터럽트 기능을 갖는다. 그 밖의 다른 모든 프로세서는 할당된 작업을 완료하고 스스로 중단될 때까지, 또는 조정 프로세서(coordinating processor)에 의해 중단될 때까지 인터럽트되지 않은 채 운영된다.

[0271] 입/출력(I/O)

[0272] TOMI 프로세서 환경의 하나의 실시예에서, 단일 프로세서는 외부 세계로의 모든 인터페이싱을 담당한다.

[0273] 직접 메모리 액세스(DMA) 컨트롤

[0274] 하나의 실시예에서, TOMI 프로세서에서 외부 세계에 대한 즉각적인 반응은 DMA 컨트롤러를 통해 발생한다. DMA 컨트롤러는, 외부 장치에 의해 요청될 때, 시스템의 RAM으로 기록하기 위한 데이터를 외부 장치로부터 내부 데이터 버스로 전송한다. 또한 상기 컨트롤러는, 요청될 때, 데이터를 시스템 RAM에서 외부 장치로 전송한다. DMA 요청이 내부 버스 액세스에 대한 가장 높은 우선순위를 가질 것이다.

[0275] TOMI 프로세서의 어레이 구성하기(Organizing an Array of TOMI Processor)

[0276] 본 발명의 바람직한 실시예의 TOMI 프로세서는 복수 개로 복제되며, 모놀리식 칩(monolithic chip) 상에서 추가적인 프로세싱 기능, 초 광대역 내부 버스 및 시스템 메모리와 조합되도록 설계된다. 이러한 시스템에 대한 바

람직한 메모리 맵(memory map)이 도 8에서 도시된다.

[0277] 각각의 프로세서에 대한 메모리 맵은 처음 32개의 로케이션(1F hex)을 각각의 프로세서를 위한 로컬 레지스터로 둔다(도 3 참조). 모든 프로세서는 자신의 캐시 레지스터를 통해 메모리 맵의 나머지 부분을 주소 지정할 수 있다(도 6 참조). 시스템 RAM의 주소 지정 능력(addressability)은 로컬 캐시와 연계되는 3개의 레지스터(PC, X 및 Y)의 폭에 의해서만 제한받는다. 레지스터가 24비트 폭을 갖는 경우, 총 주소 지정 능력은 16Mbyte일 것이지만, 상한은 없다.

[0278] 하나의 실시예에서, 64개의 TOMI 프로세서가 메모리와 모놀리식하게 구현된다. 하나의 마스터 프로세서(master processor)가 나머지 63개의 프로세서를 관리한다. 슬레이브 프로세서(slave processor) 중 하나가 아이들(idle) 상태일 때, 클럭킹하지 않으며, 이로 인해서 전력을 거의, 또는 전혀 소모하지 않으며, 열을 거의, 또는 전혀 발생시키지 않는다. 초기화 중에는 마스터 프로세서만 동작한다. 스레드(thread)가 시작되어야 하는 시점까지 상기 마스터는 인스트럭션을 인출하고 실행하기 시작한다. 각각의 스레드는 선-컴파일(pre-compile)되어, 메모리로 로드되었다. 스레드를 시작하기 위해, 마스터는 상기 스레드를 TOMI CPU 중 하나에게 할당한다.

[0279] 프로세서 가용도(Processor Availability)

[0280] 바람직하게 작업하기 위한 TOMI 프로세서의 가용도의 조절은 도 9의 프로세서 가용도 테이블에 의해 다뤄진다. 조절하는 프로세서(마스터 프로세서)는 다음의 기능을 수행할 수 있는 것이 바람직하다:

- [0281] 1. 슬레이브 프로세서에 대한 호출 매개변수(가령, 스레드의 실행 주소, 출발지 메모리 및 도착지 메모리)를 자신의 스택에 푸시(push)하기.
- [0282] 2. 슬레이브 프로세서를 시작하기.
- [0283] 3. 폴링(polling)함으로써, 또는 인터럽트에 응답함으로써, 슬레이브 프로세서 스레드 종료 이벤트에 응답하기.

[0284] 프로세서 요청하기(Requesting a Processor)

[0285] 조절 프로세서는 상기 가용도 테이블로부터 프로세서를 요청할 수 있다. “0”으로 설정된 available_flag를 갖는 최하위 프로세서의 번호가 리턴(return)된다. 그 후, 조절 프로세서는 이용 가능한 프로세서와 연계된 가용도 플래그를 “1”로 설정할 수 있으며, 이로 인해서, 슬레이브 프로세서가 시작될 수 있다. 어떠한 프로세서도 이용가능하지 않다면, 상기 요청은 에러(error)를 리턴할 것이다. 또는, 수행될 것이 요청된 작업과 연계된 우선순위 레벨을 기초로 하여, 조절 프로세서에 의해 프로세서가 할당될 수 있다. 우선순위 방식을 토대로 하는 자원 할당 기법은 공지된 바 있다. 도 10은 프로세서 할당의 3가지 바람직한 구성요소, 즉, 조절 프로세서의 연산 개시와, 슬레이브 프로세서의 연산과, 인터럽트 응답을 통한 조절 프로세서 결과 핸들링을 도시한다.

[0286] 슬레이브 프로세서의 단계별 시작(Step-by-Step Starting a Slave Processor)

- [0287] 1. 조절 프로세서는 스레드가 실행되기 위한 매개변수를 자신의 고유 스택에 푸시한다. 상기 매개변수는, 스레드의 시작 주소, 상기 스레드에 대한 출발지 메모리, 상기 스레드에 대한 도착지 메모리, 마지막 매개변수 카운트를 포함할 수 있다.
- [0288] 2. 조절 프로세서가 이용 가능한 프로세서를 요청한다.
- [0289] 3. 프로세서 할당 로직(processor allocation logic)은 설정된 자신의 관련 available_flag와 빈(cleared) 자신의 관련 done_flag를 모두 갖는 숫자상으로 가장 작은 슬레이브 프로세서의 번호, 또는 에러를 리턴한다.
- [0290] 4. 에러가 리턴된 경우, 조절 프로세서는, 슬레이브 프로세서가 이용가능해질 때까지 요청을 재-시도하거나, 에러를 처리하기 위한 임의의 특수 연산을 수행할 수 있다.
- [0291] 5. 이용 가능한 프로세서 번호가 리턴된 경우, 상기 조절 프로세서는 지시된 프로세서에 대한 available_flag를 비운다(clear). 이 연산은 스택 매개변수의 parameter_count 숫자를 선택된 슬레이브 프로세서의 스택으로 푸시한다. done_flag가 0으로 비워진다.

- [0292] 6. 상기 슬레이브 프로세서가 탑 스택에 위치하는 아이템을 검색하고, 이를 슬레이브 프로세서의 프로그램 카운터로 전송한다.
- [0293] 7. 그 후, 상기 슬레이브 프로세서가 상기 프로그램 카운터에 의해 지시되는 메모리 컬럼을 인스트럭션 캐시로 인출한다.
- [0294] 8. 슬레이브 프로세서는 인스트럭션 캐시의 시작 부분에서부터 인스트럭션을 실행하기 시작한다. 첫 번째 인스트럭션은, 상기 스택에서 호출 매개변수를 검색할 가능성이 높을 것이다.
- [0295] 9. 슬레이브 프로세서는 인스트럭션 캐시로부터 스레드를 실행한다. 스레드가 완료될 때, 상기 슬레이브 프로세서는 자신의 관련 done-flag의 상태를 체크한다. done_flag가 설정되는 경우, 상기 done_flag가 비워진 상태가 될 때까지 대기하는데, 비워진 상태는 상기 조절 프로세서가 임의의 이전 결과를 처리했음을 나타낸다.
- [0296] 10. 상기 슬레이브 프로세서와 연계되는 인터럽트 벡터(interrupt vector)가 -1로 설정되는 경우, done_flag를 설정함으로써, 어떠한 인터럽트도 생성되지 않을 것이다. 따라서 상기 조절 프로세서는 설정된 done_flag에 대해 폴링(polling)할 수 있다.
- [0297] 조절 프로세서가 done_flag가 설정되었음을 검출하면, 슬레이브 프로세서의 결과를 처리하고, 가능하면, 상기 슬레이브 프로세서에게 새로운 작업을 수행할 것을 제-할당할 수 있다. 상기 슬레이브 프로세서의 결과가 처리되면, 관련 조절 프로세서가 관련 done_flag를 비울 것이다.
- [0298] 슬레이브 프로세서와 연계되는 인터럽트 벡터가 -1이 아닌 경우, 관련 done_flag를 설정함으로써, 조절 프로세서는 인터럽트되고, 인터럽트 벡터 주소에서 인터럽트 핸들러를 실행하기 시작할 것이다.
- [0299] 관련 available_flag가 또한 설정된 경우, 조절 프로세서도 슬레이브 프로세서의 스택에 푸시되는 리턴 매개변수를 관독할 수 있다.
- [0300] 상기 인터럽트 핸들러는 슬레이브 프로세서의 결과를 처리하고, 가능하다면, 슬레이브 프로세서에게 새로운 작업을 수행할 것을 제-할당할 것이다. 슬레이브 프로세서의 결과가 처리되었으면, 조절 프로세서 상에서 실행되는 인터럽트 핸들러가 관련 done_flag를 비울 것이다.
- [0301] 11. done_flag가 비움 상태인 경우, 슬레이브 프로세서는 자신의 관련 done_flag를 설정하고, 새로운 start_time을 저장한다. 상기 슬레이브 프로세서는 작업을 계속 수행하거나, 이용 가능 상태로 복귀될 수 있다. 이용 가능 상태로 복귀하기 위해, 슬레이브 프로세서는 리턴 매개변수를 자신의 스택에 푸시하며, 뒤 이어, 스택 카운트를 푸시하고, 자신의 available_flag를 설정할 수 있다.
- [0302] **메모리 모드 레지스터를 이용하여 TOMI 프로세서 관리하기(Managing TOMI Processors Using Memory Mode Register)**
- [0303] 다수의 TOMI 프로세서를 구현하고 관리하기 위한 한 가지 기법은, 도 10A에서 나타나는 바와 같이, 듀얼 인라인 메모리 모듈(DIMM: dual inline memory module) 상에 TOMI 프로세서를 장착하는 것이 있다. TOMI/DIMM은 외부 메모리 컨트롤러와 범용 CPU로 구성된 시스템, 예를 들어 개인용 컴퓨터에 포함될 수 있다. 도 10B는 이러한 구성을 도시한다. 모드 레지스터(mode register)는 DRAM, SRAM 및 FLASH 메모리에서 일반적으로 발견된다. 상기 모드 레지스터는, 메모리 액세스에 독립적으로, 외부 메모리 컨트롤러에 의해 기록될 수 있는 래치(latch)의 세트이다. 타이밍, 리프레시 컨트롤(refresh control) 및 출력 버스트 길이(output burst length) 등의 매개변수를 특정하기 위해, 종종 메모리 모드 레지스터의 비트들이 사용된다. 메모리 모드 레지스터의 하나 이상의 비트가 TOMI CPU를 활성화, 또는 비활성화시키기 위해, 할당될 수 있다. 예를 들어, 모드 레지스터에 의해 TOMI CPU가 비활성화될 때, TOMI CPU를 포함하는 메모리가 보통의 DRAM, SRAM, 또는 FLASH 메모리로서 기능할 것이다. 모드 레지스터가 TOMI CPU 초기화를 활성화시킬 때, 도 10C에서 설명되는 바와 같이, 시퀀스가 수행될 것이다. 이 실시예에서, 하나의 프로세서가 마스터 프로세서이도록 결정된다. 이 프로세서는 RESET 연산 후에 항상 제일 먼저 시작하는 레지스터이다. 초기화의 끝 부분에서, 마스터 프로세서는 최대 속도로 실행되며, 희망 애플리케이션 프로그램을 실행할 것이다. TOMI CPU가 실행되는 동안, DRAM, SRAM, 또는 FLASH 메모리를 액세스할 수 없을 것이다. 메모리 모드 레지스터는 종종 외부 메모리 컨트롤러에 의해 TOMI CPU의 실행을 중단할 것을 지시받을 수 있다. TOMI CPU가 중단(halt)될 때, 범용 CPU에 부착되는 외부 메모리 컨트롤러에 의해 DRAM, SRAM, 또는 FLASH의 내용이 관독될 수 있다. 이러한 방식으로, 결과가 범용 CPU로 전달될 수 있으며, 추가적인 데이터, 또

는 실행물(executables)이 DRAM, SRAM, 또는 FLASH 메모리로 기록될 수 있다.

[0304] 범용 CPU가 DRAM, SRAM, 또는 FLASH 메모리의 임의의 판독, 또는 기록을 완료했으면, 외부 메모리 컨트롤러가 모드 레지스터 비트를, HALT에서 RUN으로 기록하고, TOMI CPU의 실행이, 그만 둔 부분부터, 계속 실행할 것이다. 도 10D는 DRAM, SRAM, 또는 FLASH 메모리에서의 통상적인 메모리 모드 레지스터를 도시하고, TOMI CPU를 제어하기 위해 상기 레지스터가 어떻게 수정되는가를 도시한다.

[0305] 프로세서 클럭 속도 조정하기(Adjusting Processor Clock Speed)

[0306] 프로세서 클럭 속도는 프로세서 전력 소모를 결정한다. 하나를 제외한 모든 프로세서가 중단되게 함으로써, TOMI 아키텍처가 저전력 소모를 가능하게 한다. 덧붙이자면, 마스터 프로세서를 제외한 각각의 프로세서는, 도 2D에서 나타난 로직을 이용하여, 성능, 또는 전력 소모를 최적화하기 위해 조정된 클럭 속도를 가질 수 있다.

[0307] TOMI 프로세서 관리의 또 다른 실시예

[0308] 일부 컴퓨터 소프트웨어 알고리즘은 재귀적이다. 즉, 알고리즘의 주 함수가 자기 자신을 호출한다. “Divide and Conquer(이하 DAC)” 라고 알려진 알고리즘의 클래스는 종종, 재귀 기법을 이용하여 구현된다. DAC는 데이터의 검색 및 정렬과, 특정 수학적 함수에 적용될 수 있다. 다수의 프로세서(예를 들어, TOMI 아키텍처에서 이용 가능한 프로세서)를 이용하여 이러한 알고리즘을 병렬처리하는 것이 가능할 수 있다. 이러한 알고리즘을 구현하기 위해, 하나의 TOMI CPU가 또 하나의 TOMI CPU에게 작업을 전달하고, 상기 CPU로부터 결과를 수신할 수 있어야 한다. TOMI 프로세서의 또 하나의 실시예에 의해, 임의의 프로세서가 마스터 프로세서가 될 수 있고, 그 밖의 다른 이용 가능한 임의의 TOMI 프로세서를 슬레이브 프로세서로서 다룰 수 있다. TOMI 프로세서를 시작하고 중단하기, 프로세서 간에 통신하기 및 독립적 및 종속적 스레드 관리하기가 프로세서 관리에 대한 이 실시예에서 지원된다.

[0309] TOMI CPU 중단하기(Stopping a TOMI CPU)

[0310] TOMI CPU는, 자신의 PC에 모두 1을 기록함으로써, 중단될 수 있다. TOMI CPU가 중단될 때, 상기 CPU의 클럭은 실행되지 않으며, 전력을 소모하지 않는다. 어떠한 TOMI CPU라도 자신의 PC에 모두 1을 기록할 수 있다.

[0311] TOMI CPU 시작하기(Starting TOMI CPU)

[0312] TOMI CPU는, 모두 1인 값 외의 값이 자신의 PC에 기록될 때, 실행을 시작할 수 있다. 도 10D에서 도시되는 바와 같이, 마스터 프로세서는 모드 레지스터에 의해 재설정될 때 자신의 PC에 기록되는 값으로 0의 값을 갖는다.

[0313] 독립적 프로세서 스레드(Independent Processor Thread)

[0314] 하나의 단일 범용 프로세서 상에서 다수의 스레드가 실행될 때, 상기 스레드들은 매우 느슨하게 연결되어, 서로 거의 드물게 통신할 수 있다. 실행, 결과 리턴 및 중단하는 과정 대신, 일부 스레드는 영원히 실행되어, 결과를 지속적이고 영구적으로 전달할 수 있다. 이러한 스레드의 예는 네트워크 통신 스레드, 또는 마우스 장치 판독 스레드일 것이다. 상기 마우스 스레드는 지속적으로 실행되어, 마우스 위치 및 클릭 정보를, 폴링될 수 있는 공유 메모리 영역으로 전달하거나, 즉시 제공될 수 있는 콜백 루틴(callback routine)으로 전달할 수 있다.

[0315] 이러한 독립적 스레드는 주로, 성능을 강화하기 보다는, 프로그래밍을 단순화하기 위해 사용된다. 유사한 스레드가 멀티프로세서 시스템(가령, TOMI)에서 실행될 수 있다. 그 결과는 공유 메모리 영역으로 전달될 수 있다. 일부 경우, 공유 변수(shared variable)를 통해 통신이 이뤄질 수 있다.

[0316] TOMI 아키텍처에서, 변수는, 메모리 RAS 사이클이 완전한 로우(row)를 X_cache, 또는 Y_cache로 로드할 것을 요구하지 않을 수 있기 때문에, 고유 변수가 공유 메모리보다 효율적일 수 있다. 변수 사용의 예는, TOMI CPU 모니터링 네트워크 트래픽을 위한 수신 버퍼를 가리키는 입력 포인터일 것이다. CPU를 모니터링하는 네트워크는,

데이터가 수신될 때, 변수를 증연산할 것이다. 데이터 소비 CPU는 때때로 변수를 관독하고, 충분한 데이터 존재할 때, 메모리 로우를 X_cache, 또는 Y_cache로 로드하기 위한 연산을 수행할 것이다. 그 후, 수신된 네트워크 데이터가 캐시로부터, 공유 변수가 가리키는 값까지 관독될 수 있다.

[0317] 종속적 프로세서 스레드(Dependent Processor Thread)

[0318] 일부 알고리즘(예를 들어, divide and conquer로 분류된 알고리즘)이 몇 개의 프로세서 상의 알고리즘의 조각을 동시에 실행하고, 그 결과를 조합함으로써, 병렬처리를 달성할 수 있다. 이러한 설계에서, 하나의 마스터 프로세서가 다수의 슬레이브 프로세서로부터 작업을 요청하며, 그 후, 슬레이브들이 작업을 병렬로 수행하는 동안 대기할 것이다. 이러한 방식으로, 마스터 프로세서는 슬레이브 프로세서들에 의해 완료되고 있는 작업에 종속적이다.

[0319] 슬레이브 프로세서의 작업이 완료될 때, 마스터 프로세서가 부분 결과들을 관독하고, 이들을 조합하여 최종 결과를 도출할 것이다. 이러한 기능에 의해, TOMI 아키텍처는 “divide and conquer”라고 알려져 있는 알고리즘의 클래스를 효과적으로 처리할 수 있다. 더 일반적인 간단한 DAC 알고리즘 중 검색과 정렬이 있다.

[0320] 멀티프로세서 관리 인스트럭션 세트 확장(Multiprocessor Management Instruction Set Extension)

[0321] 기본 TOMI 인스트럭션 세트에 대한 일련의 확장은 독립적 스레드 관리와 종속적 스레드 관리 모두를 가능하게 한다. 이들 인스트럭션은, 도 2B에서 나타난 이용 가능한 NOOP 코드 중 일부를 이용하여 구현될 것이다. 이들 관리 확장 인스트럭션은 도 2C에서 요약되어 있다.

[0322] GETNEXTPROCESSOR - 이 인스트럭션은 프로세서 가용도 테이블에 질의하고, 다음 번 이용 가능한 프로세서와 연관되어 있는 번호를 갖는 ACC를 로드한다.

[0323] SELECTPROCESSOR - 이 인스트럭션은 ACC를 프로세서 셀렉트 레지스터(Processor Select Register)로 기록한다. 상기 프로세서 셀렉트 레지스터는, TESTDONE 및 READSHARED VARIABLE에 의해 평가될 프로세서를 선택한다.

[0324] STARTPROCESSOR - 이 인스트럭션은 프로세서 셀렉트 레지스터에 의해 선택된 프로세서의 PC로 기록한다. 이 인스트럭션은, 마스터 프로세서가 중단된 슬레이브 프로세서를 시작하기를 원할 때, 실행될 가능성이 매우 높다. 슬레이브 프로세서는 자신의 PC가 모두 1인 경우, 중단된다. 하나의 값을 슬레이브 프로세서의 PC로 기록함으로써, 상기 마스터 프로세서에 의해, 슬레이브 프로세서는 기록된 PC의 위치에서 프로그램을 실행시키기 시작할 것이다. 연산이 성공적이었다면, ACC는 선택된 프로세서로 기록되는 PC 값을 포함할 것이다. 연산이 성공적이지 않았다면, ACC는 -1을 포함할 것이다. 실패된 연산에 대한 가장 가능성 높은 이유는 선택된 프로세서가 이용가능하지 않았기 때문이다.

[0325] TESTDONE - 이 인스트럭션은 프로세서 셀렉트 레지스터에 의해 선택된 프로세서의 PC를 테스트하고, PC가 모두 1이면, 호출 프로세서의 C 비트를 “1”로 설정한다. 이러한 방식으로, PC를 테스트하는 루프가 다음과 같이 생성될 수 있다:

[0326] `LOADI, processorNumber`

[0327] `SELECTPROCESSOR`

[0328] `LOADACC, LOOPADDRESS`

[0329] `TOP TESTDONE`

[0330] `STOREACC, PC_COND` \ \ PC가 모두 1인 값을 갖고 선택된 프로세서가 중단될 때까지 TOP까지 루프

[0331] TESTAVAILABLE - 이 인스트럭션은, 프로세서 셀렉트 레지스터에 의해 선택된 프로세서에 대해 프로세서 가용도 테이블에서 “이용 가능 상태” 비트를 테스트하고, 선택된 프로세서가 이용 가능한 경우, 호출 프로세서의 C 비트를 “1”로 설정한다. 이러한 방식으로, 추가적인 작업에 대한 가용도를 테스트하기 위한 루프가 다음과 같이 생성될 수 있다:

[0332] `LOADI, processorNumber`

- [0333] SELECTPROCESSOR
- [0334] LOADACC, LOOPADDRESS
- [0335] TOP TESTAVAILABLE
- [0336] STOREACC, PC_COND \ \선택된 프로세서가 이용 가능해질 때까지 TOP까지 루프
- [0337] SETAVAILABLE - 이 인스트럭션은 프로세서 셀렉트 레지스터에 의해 선택된 프로세서에 대해 프로세서 가용도 테이블에서 “이용 가능 상태” 비트를 설정한다. 이 인스트럭션은, 하나의 프로세서에 의해 실행될 가능성이 매우 높을 것이다. 도 10E에서 도시되는 바와 같이, 상기 하나의 프로세서는 또 다른 프로세서가 작업을 수행할 것을 요청했던 프로세서이다. 작업 중인 프로세서가 자신의 작업을 완료할 때, 상기 프로세서는 자신의 PC를 모두 1로 설정함으로써 멈춘다. 요청 프로세서는 작업 프로세서에 대해 주기적으로 TESTDONE할 것이다. 작업 프로세서가 자신의 작업을 완료했으며, 상기 요청 프로세서는 공유 메모리 로케이션, 또는 공유 변수를 통해, 결과를 판독할 것이다. 결과가 판독되면, 작업 프로세서는 또 다른 작업을 재-할당받을 수 있는 상태가 되고, 또 다른 프로세서가 상기 작업 프로세서에게 추가적인 작업을 할 것을 요청할 수 있도록, 상기 요청 프로세서는 SETAVAILABLE를 사용할 것이다.
- [0338] READSHAREDVARIABLE - 이 인스트럭션은 프로세서 셀렉트 레지스터에 의해 선택된 프로세서의 공유 변수를 판독한다. 이 인스트럭션은 하나의 프로세서에 의해 실행될 가능성이 높을 것이다. 상기 하나의 프로세서는 또 다른 프로세서에게 작업을 수행할 것을 요청했던 프로세서이다. 공유 변수가 임의의 프로세서에 의해 판독되어, 할당된 작업의 진전을 결정할 수 있다. 예를 들면, 작업 프로세서는 고속 네트워크로부터 수신된 데이터를 처리하도록 할당될 수 있다. 공유 변수는, 판독된 적 있으며, 다른 프로세서에게 이용 가능했던 데이터의 크기를 가리킬 것이다. 각각의 프로세서는 공유 변수를 포함한다. 상기 공유 변수는 그 밖의 다른 임의의 프로세서에 의해 판독될 수 있지만, 공유 변수는 자신의 고유 프로세서에 의해서만 기록될 수 있다.
- [0339] STORESHAREDVARIABLE - 이 인스트럭션은 ACC의 값을 상기 인스트럭션을 실행하는 프로세서의 공유 변수로 기록한다. 상기 공유 변수는 그 밖의 다른 임의의 프로세서에 의해 판독될 수 있으며, 상태와 결과를 그 밖의 다른 프로세서에게 전달하기 위해 사용될 수 있다.
- [0340] **데이터 레디 래치를 이용하는 인터프로세서 통신(Interprocessor Communications Using Data Ready Latches)**
- [0341] 도 10F는 TOMI 프로세서 간의 통신의 한 가지 가능한 하드웨어 구현예를 도시한다. SELECTPROCESSOR 커맨드를 이용하여 하나의 TOMI 프로세서가 자신과 또 하나의 TOMI 프로세서 간의 연결을 확립할 수 있다. 이 인스트럭션은, 공유 레지스터와 READSHARED VARIABLE 및 STORESHAREDVARIABLE 커맨드를 이용하여, 선택하는 프로세서와 선택되는 프로세서가 데이터를 교환할 수 있게 하는 논리적 연결(logical connection)을 생성한다.
- [0342] 도 10F의 상단 절반이, 프로세서가 레디 플래그 레지스터(ready flag register)에 의해 제어되는 다른 프로세서에게 데이터를 전송하기 위한 로직을 도시한다. 도 10F의 하단 절반은, 프로세서가 레디 플래그 레지스터에 의해 제어되는 다른 프로세서로부터 데이터를 수신하기 위한 로직을 도시한다.
- [0343] 공유 레지스터의 상태가, 선택하는 프로세서, 또는 선택되는 프로세서의 C 비트로 판독될 수 있다. 동작 중에, 프로세서가 자신의 공유 레지스터로 데이터를 기록하며, 이로 인해서, 관련 데이터 레디 플래그가 설정될 수 있다. C 비트가 관련 데이터 레디 플래그가 설정되었다고 나타낼 때까지 연결된 프로세서가 자신의 공유 레지스터를 판독한다. 판독 작업은 레디 플래그를 비워서(clear), 프로세서가 반복될 수 있다.
- [0344] **프로세서 할당 중재하기(Arbitrating Processor Allocation)**
- [0345] 앞서 언급된 바와 같이, TOMI 프로세서는 GETNEXTPROCESSOR에 의해 판단된 이용 가능 상태를 갖는 또 다른 TOMI 프로세서에게 작업을 위임할 수 있다. GETNEXTPROCESSOR는 프로세서가 이용 가능한 상태인가의 여부를 판단한다. 이용 가능한 프로세서란, 현재 작업을 수행하고 있지 않으며, READ SHAREDVARIABLE에 의해 아직 검색되지 않은 5개의 이전 작업의 결과를 보유하고 있지 않은 프로세서를 말한다.
- [0346] 도 10G는 작업이 위임될 수 있는 이용 가능한 프로세서를 식별하기 위한 하나의 하드웨어 구현예를 도시한다.

도 10H는 바람직한 프로세서 중재의 이벤트를 도시한다. 상기 프로세스는 다음과 같다:

- [0347] 1. 요청하는 프로세서는 GETNEXTPROCESSOR 인스트럭션을 실행하며, 이는 "Request Next Available Processor (다음 번 이용 가능한 프로세서 요청)" 라인을 중재 로직으로 끌어내린다(pull down).
- [0348] 2. 상기 중재 로직은 "Next Available Processor" 라인 상의 TOMI CPU에 대응하는 번호를 생성한다.
- [0349] 3. 요청 프로세서는 요청하는 프로세서의 프로세서 셀렉트 레지스터(PSR: Processor Select Register)의 번호를 저장하는 SELECTPROCESSOR 인스트럭션을 실행한다.
- [0350] 4. 그 후, 요청 프로세서는, 상기 프로세서 셀렉트 레지스터에 의해 선택되는 프로세서의 PC로 기록되는 STARTPROCESSOR 인스트럭션을 실행한다. 연산이 성공적인 경우, 선택된 프로세서의 번호가 중재 로직에 저장되어, 선택된 프로세서는 더 이상, 작업을 수행하라고 할당 받기에 이용 가능 상태가 아님을 나타낼 수 있다. 연산이 성공적이지 않은 경우, 그 이유는 아마도 선택된 프로세서가 이용 가능 상태가 아니었기 때문일 수 있다. 요청 프로세서는 또 하나의 이용 가능한 프로세서를 찾기 위해 또 하나의 GETNEXTPROCESSOR를 실행할 것이다.
- [0351] 5. 선택된 프로세서가 STORESHAREDVARIABLE을 실행하여, 자신의 결과를 이용 가능 상태로 만들 때, 중재 로직은 선택된 프로세서가 판독되기를 대기하고 있는 결과를 갖고 있다고 통지받는다.
- [0352] 6. 선택된 프로세서가 자신의 PC에 -1을 기록함으로써, 중단될 때, 상기 중재 로직은 선택된 프로세서가 이용 가능 상태라고 통지받는다.
- [0353] 7. READSHARED VARIABLE에 의해 선택된 프로세서의 결과가 검색될 때, 상기 중재 로직은 선택된 프로세서의 결과가 판독된 적이 있다고 통지받는다.

[0354] 메모리 잠금(Memory Locking)

- [0355] TOMI 프로세서는 자신의 캐시를 통해 시스템 메모리를 판독 및 기록한다. 완전히 캐싱된 컬럼은 한 번에 판독, 또는 기록된다. 임의의 프로세서가 시스템 메모리의 임의의 부분을 판독할 수 있다. 개개의 프로세서는 자신만의 배타적 기록 작업을 위해 메모리의 컬럼을 잠글 수 있다. 이러한 잠금 메커니즘은 프로세서 간 메모리 기록 작업의 충돌을 방지한다.

[0356] 제안되는 애플리케이션

- [0357] 병렬처리는 애플리케이션을 효과적으로 가속시키며, 이때 상기 애플리케이션은 개개의 프로세서에 대한 작업의 독립적인 조각으로 분할(factor)될 수 있다. 잘 분할되는 하나의 애플리케이션으로는 로봇 비전(robotic vision)을 위한 이미지 조작(image manipulation)이 있다. 이미지 조작 알고리즘은 상관(correlation), 등화(equalization), 에지 식별(edge identification) 및 그 밖의 다른 연산을 포함한다. 대부분은 매트릭스 조작(matrix manipulation)에 의해 수행된다. 도 11에서 도시되는 바와 같이, 알고리즘은 대부분 잘 분할된다. 도 11에서, 예시적 이미지맵이 24개의 프로세서를 도시하며, 이때, 하나의 프로세서가 전체 이미지맵의 장방형 서브셋에 대해 이미지 데이터를 조작하기 위해 할당된다.
- [0358] 도 12는 하나의 실시예에서, TOMI 시스템의 RAM이 할당될 수 있는 방식을 도시한다. 시스템의 RAM의 하나의 블록이 이미지 캡처 픽셀을 보유하며, 또 다른 블록은 처리된 결과를 보유한다.
- [0359] 동작 중에, 조절 프로세서(coordinating processor)가, 고정된 시간 마다 외부 소스로부터 내부 시스템 RAM으로 이미지 픽셀을 전송하기 위한 DMA 채널을 할당한다. 이미지 캡처의 통상의 속도는 초 당 60개의 이미지일 수 있다.
- [0360] 그 후, 조절 프로세서에 의해, X 레지스터에 의해 사용될 이미지맵의 주소와, Y 레지스터에 의해 사용될 처리된 이미지의 주소와, 프로세서 2의 매개변수 카운트와, 이미지 처리 알고리즘의 주소를 푸시함으로써, 슬레이브 프로세서 1이 활성화된다. 상기 조절 프로세서는 프로세서 2 내지 25를 잇달아서 유사한 방식으로 활성화시킨다. 이미지 처리 알고리즘이 종료될 때까지, 상기 프로세서들은 독립적으로, 병렬로, 계속 실행된다.
- [0361] 상기 알고리즘이 종료되면, 각각의 프로세서가 프로세서 가용도 테이블에서의 자신의 관련 done_flag를 설정한다. 그 결과는 조절 프로세서에 의해 처리되는데, 상기 조절 프로세서는 종료에 대해 폴링하거나, "완료" 이

벤트에 관련하여 인터럽트에 응답한다.

- [0362] 도 13A 및 13B가 온-칩 시스템 메모리와 함께 구현되는 64개의 TOMI 프로세서의 모놀리식 어레이에 대한 바람직한 평면도를 도시한다. 상기 평면도는 회로 설계도, 메모리 회로의 레이아웃 및 전체 칩 아키텍처에 따라 달라질 수 있다.
- [0363] **TOMI Peripheral Controller Chip (TOMIPCC)**
- [0364] TOMI 아키텍처의 하나의 실시예가 하나 이상의 TOMI CPU를 표준 DRAM 다이에 내장한다. 표준 DRAM 핀아웃을 이용하여, 최종 다이가 표준 DRAM 패키지에 패키징된다. 패키징된 부분은 표준 DRAM DIMM(Dual Inline Memory Module) 상에 장착될 수 있다.
- [0365] 동작 중에, 이 실시예는, 내장된 TOMI CPU가 DRAM 모드 레지스터를 통해 활성화될 때를 제외하고, 표준 DRAM처럼 기능한다. TOMI CPU가 활성화되고 동작 중일 때, 상기 TOMI CPU가 외부 프로세서에 의해 DRAM으로 로드되는 프로그램을 실행한다. TOMI CPU 계산의 결과는 공유 DRAM을 통해 외부 프로세서로 제공된다.
- [0366] 일부 경우에, 외부 프로세서는 PC에 의해 제공될 것이다. 또 다른 경우, TOMI DRAM 칩에 대해 다음의 기능을 수행하기 위해 특수화된 프로세서가 제공될 수 있다. 도 14A는 이러한 프로세서의 하나의 실시예, TOMI 주변 컨트롤러 칩(TOMIPCC: Peripheral Controller Chip)을 도시한다. 이 프로세서의 기능은,
- [0367] 1. 데이터 및 인스트럭션을 관련 영구 저장 장치로부터 TOMI 칩 DRAM으로 사용되기 위해 전송하기 위한 수단을 제공하는 기능(다수의 시스템에서, 영구 상기 저장 장치는 플래쉬 RAM일 수 있음)
- [0368] 2. TOMI 칩과 실세계 장치(가령, 디스플레이 및 네트워크) 간에 입/출력 인터페이스를 제공하는 기능
- [0369] 3. TOMI CPU 연산을 조절하기 위해 필요한 운영 체제 기능의 작은 세트를 실행하는 기능
- [0370] 이다.
- [0371] 도 14B는 TOMIPCC를 이용하는 매우 작은 시스템을 도시한다. 예시적 셀폰 언어 번역기는 단 3개의 칩, 즉, 플래쉬 RAM, TOMIPCC 및 단일 TOMI CPU/DRAM으로만 구성된다. 이러한 최소한의 경우에서, 단일 TOMI CPU/DRAM은 표준 8-비트 DRAM I/O (D0 - D7)를 통해 통신할 것이다.
- [0372] 플래시 RAM은 구어(spoken language)를 정의하는 음소와 구문의 사전뿐 아니라, 상기 구어를 하나의 언어에서 다른 언어로 해석하고 번역할 때 필요한 인스트럭션을 내포할 수 있다.
- [0373] TOMIPCC는 아날로그 형식의 구어(또는 이에 상당하는 것)를 수신하여, 이를 디지털 표현으로 변환하고, 해석과 번역을 위해 TOMI DRAM에게 제공할 것이다. 최종 디지털화된 음성이 TOMI DRAM로부터 TOMIPCC로 다시 전달되어, 아날로그 음성 표현으로 변환되고, 셀폰 사용자에게 출력될 것이다.
- [0374] 도 14C는 TOMIPCC를 이용하는 대형 시스템을 도시한다. 이러한 시스템의 예로는 메모리-중심 데이터베이스 기구(즉, MCDB)가 있다. MCDB 시스템은, 훨씬 더 느린 디스크, 또는 저장 장치로부터 메모리의 조각을 페이징(paging)하는 것 대신, 전체 데이터베이스를 고속 메모리에서 운영한다. TOMI 아키텍처의 경우, 메모리-중심 데이터베이스와 동일한 칩 상에 위치하는 TOMI CPU 상에서 실행되는 이른바 divide and conquer 알고리즘을 이용하여, 빠른 검색 및 정렬이 가능하다.
- [0375] 이러한 시스템은 다수의 TOMI CPU 칩을 포함하는 TOMI DIMM(듀얼 인-라인 메모리 모듈)을 이용하여 구축될 가능성이 매우 높다. 표준 240-핀 DIMM까지의 데이터 경로는 64비트이다. 따라서 메모리-중심 데이터베이스 애플리케이션에서의 TOMIPCC는 64-비트 폭의 데이터베이스(D0 - D63)를 구동할 것이다.

도면의 간단한 설명

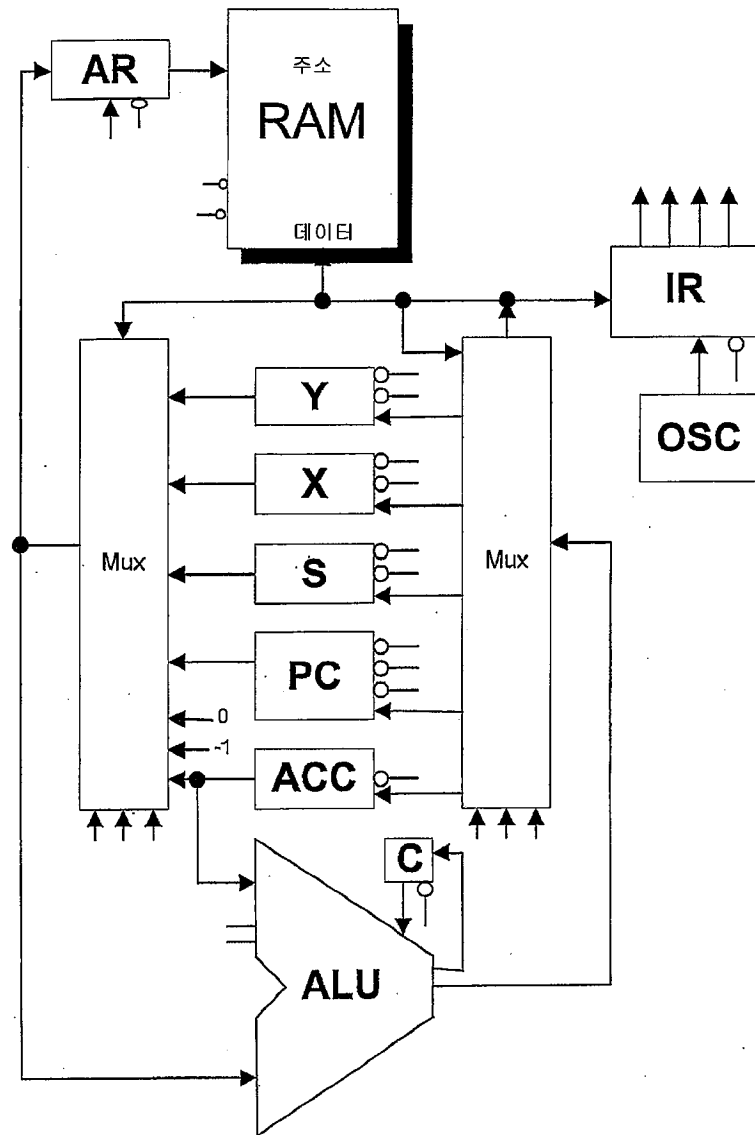
- [0100] 도 1은 하나의 실시예의 바람직한 TOMI 아키텍처를 도시한다.
- [0101] 도 2는 기본 인스트럭션 세트를 도시한다.
- [0102] 도 2A는 연산 중 순방향 브랜치를 도시한다.

- [0103] 도 2B는 바람직한 TOMI 인스트럭션 세트에 대한 인스트럭션 맵을 도시한다.
- [0104] 도 2C는 다수의 TOMI 프로세서 관리 인스트럭션 확장의 바람직한 세트를 도시한다.
- [0105] 도 2D는 TOMI 프로세서에 대한 클록 프로그래밍 회로를 도시한다.
- [0106] 도 2E는 인스트럭션 확장의 바람직한 세트를 도시한다.
- [0107] 도 3은 여러 다른 주소 지정 방식의 유효 주소를 도시한다.
- [0108] 도 4는 4 - 32비트로부터 데이터 경로가 간편하게 생성되는 방식을 도시한다.
- [0109] 도 5는 바람직한 로컬 캐시를 도시한다.
- [0110] 도 6은 바람직한 캐시 관리 상태를 도시한다.
- [0111] 도 7A는 광대역 시스템 RAM 버스를 이용하도록 구성된 추가적인 프로세싱 기능의 하나의 실시예를 도시한다.
- [0112] 도 7B는 TOMI 프로세서에 의해 액세스되는 2개의 메모리 뱅크에 대한 데이터 라인을 인터리빙하기 위한 바람직한 회로를 도시한다.
- [0113] 도 8은 바람직한 메모리 맵을 도시한다.
- [0114] 도 9는 바람직한 프로세스 가용도 테이블을 도시한다.
- [0115] 도 10은 프로세서 할당의 3가지 구성요소를 도시한다.
- [0116] 도 10A는 DIMM 패키지 상의 다수의 TOMI 프로세서의 하나의 실시예를 도시한다.
- [0117] 도 10B는 범용 CPU와 인터페이싱하는 DIMM 패키지 상의 다수의 TOMI 프로세서의 하나의 실시예를 도시한다.
- [0118] 도 10C는 다수의 TOMI 프로세서의 하나의 실시예에 대한 바람직한 TOMI 프로세서 초기화를 도시한다.
- [0119] 도 10D는 TOMI 프로세서 초기화에 대한 메모리 모드 레지스터의 사용을 도시한다.
- [0120] 도 10E는 바람직한 TOMI 프로세서 가용도 상태 다이어그램을 도시한다.
- [0121] 도 10F는 바람직한 인터프로세서 통신 회로 설계를 도시한다.
- [0122] 도 10G는 작업을 수행하기 위한 TOMI 프로세서를 식별하기 위한 바람직한 하드웨어 구현예를 도시한다.
- [0123] 도 10H는 바람직한 프로세서 중재 다이어그램을 도시한다.
- [0124] 도 11은 바람직한 분할(factoring)을 도시한다.
- [0125] 도 12는 바람직한 시스템 RAM을 도시한다.
- [0126] 도 13A는 64 TOMI 프로세서의 모놀리식 어레이를 위한 바람직한 평면도를 도시한다.
- [0127] 도 13B는 TOMI 프로세서의 모놀리식 어레이를 위한 또 하나의 바람직한 평면도를 도시한다.
- [0128] 도 14A는 TOMI 주변 제어 칩(TOMIPCC)에 대한 바람직한 평면도를 도시한다.
- [0129] 도 14B는 TOMIPCC를 이용하는 셀폰 언어 번역기 애플리케이션에 대한 바람직한 설계를 도시한다.

도면

도면1

레지스터 및 데이터 경로



도면2

인스트럭션 Op 코드 맵

Op 코드	니모닉	연산	캐리
00b bbbbb	LOADI, IMM	IMM -> ACC	No change
010 bbbbb	LOADACC, R	R -> ACC	No change
011 bbbbb	STOREACC, R	ACC -> R	No change
100 bbbbb	ADD, R	ACC + R -> R	C = carry
101 bbbbb	AND, R	ACC AND R -> R	If zero, C=1
110 bbbbb	XOR, R	ACC XOR R -> R	If zero, C=1
111 bbbbb	INC, R	R + 1 -> R	If zero, C=1

b-단일 이진 비트
 IMM-6-비트 즉시 피연산자
 R-레지스터

도면2a

순방향 브랜치

	MSB		LSB
PC	xxxxxxxx	xxxxxxxx	xxwwwwww
ACC	00000000	00000000	00zzzzzz

순방향 브랜치가 취해짐, **zzzzzz** → **wwwwww**

if **wwwwww** > **zzzzzz**,

then **xxxxxxxx xxxxxxxx xx + 1** → **xxxxxxxx xxxxxx xx**

도면2b

인스트럭션 맵

Op code bits 7 6 5	7 6 5	7 6 5	7 6 5	7 6 5	7 6 5	7 6 5	7 6 5
(000)	(001)	(010)	(011)	(100)	(101)	(110)	(111)
LOADI	LOADI	LOADACC	STOREACC	ADD	AND	XOR	INC
00000		SAVELOOP	if C=0, ACC → PC	if C=0, ACC+PC → PC	LOOP_IF	NOOP	if C=0, PC+1 → PC
00001		SHIFTLOADBYTE	ACC → PC	ACC+PC → PC	LOOP	SETMSB	PC+1 → PC
00010		NOTACC	ROTATELEFT8	ACC+ACC → ACC modify C	testACC modify C	XOR ACC modify C	INC ACC modify C
00011		GETNEXT- PROCESSOR	SELECTNEXT- PROCESSOR	START- PROCESSOR	TESTDONE	TEST- AVAILABLE	SET- AVAILABLE
00100		S → ACC	ACC → S	ACC+S → S	NOOP	NOOP	NOOP
00101		S → ACC	ACC → (S)	(S)+ACC → ACC	(S)&ACC → ACC	S/ACC → ACC	S/ACC → ACC
00110		POP	NOOP	NOOP	NOOP	NOOP	NOOP
00111		RIGHTSHIFTACC	PUSH	(S)+ACC → ACC S+	(S)&ACC → ACC S+	S/ACC → ACC S+	S/ACC → ACC S+
01000		X → ACC	ACC → X	ACC+X → X	NOOP	NOOP	NOOP
01001		Operate on X_cache register pointed to by X					
01010		Pre-decrement X, operate on X_cache register pointed to by X					
01011		Operate on X_cache register pointed to by X, post-increment X					
01100		Y → ACC	ACC → Y	ACC+Y → Y	NOOP	NOOP	NOOP
01101		Operate on Y_cache register pointed to by Y					
01110		Pre-decrement Y, operate on Y_cache register pointed to by Y					
01111		Operate on Y_cache register pointed to by Y, post-increment Y					
10000		Shared Variable					
10001		범용 레지스터					
10010							
10011							
10100							
10101							
10110							
10111							
11000							
11001							
11010							
11011							
11100							
11101							
11110							
11111							

FIG. 2B

도면2c

멀티프로세서 관리 인스트럭션 세트 확장

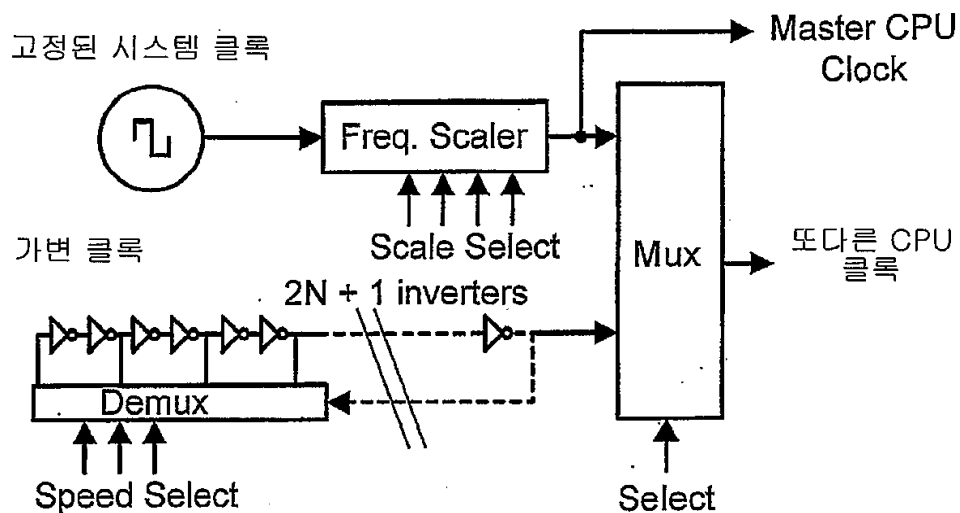
니모직	연산	캐리 효과
GETNEXTPROCESSOR	N -> ACC	변화 없음
SELECTPROCESSOR	ACC -> PSR	변화 없음
STARTPROCESSOR	ACC -> processor[PC]	성공적이면, C=1
TESTDONE	Test processor [PC]=-1	PC=-1인 경우, C=1로 설정
TESTAVAILABLE	TEST processor[AvailableFlag]	이용가능상태인 경우, C=1로 설정
SETAVAILABLE	Set AvailableFlag to 1	변화 없음
READSHAREDVARIABLE	processor[SharedVariable]->ACC	변화 없음
STOresharedvariable	ACC -> SharedVariable	변화 없음

NOTE: Op 코드는 경우에 따라 다르며, 도 2B에서 나타난 이용가능한 Op 코드 중에서 선택된다.

b-단일 이진 비트
IMM - 6-비트 즉시 피연산자
R - 레지스터
N - 32-비트 정수
PSR - 프로세서 선택 레지스터
processor-프로세서 선택 레지스터가 가리키는 프로세서

도면2d

TOMI 클록 프로그래밍



도면2e

인스트럭션 세트 확장

니모닉	연산	캐리 효과
SAVELOOP	PC → -(S)	변화 없음
SHIFTLOADBYTE	Shift ACC left 8 bits, next byte → ACC	변화 없음
LOOP	(S) → PC	변화 없음
LOOP_IF	if C=0, (S) → PC	변화 없음
NOTACC	~ACC → ACC	ACC=0이면, C=1로 설정
ROTATELEFT8	Rotate ACC 8 bits left	변화 없음
ORSTACK	ACC (S) → ACC	ACC=0이면, C=1로 설정
ORSTACK+	ACC (S)+ → ACC	ACC=0이면, C=1로 설정
RIGHTSHIFTACC	Shift ACC right	LSB → C
SETMSB	2 ³¹ → ACC	변화 없음

NOTE: Op코드는 경우에 따라 다르며, 도 2B에서 나타난 이용가능한 Op코드 중에서 선택된다.

b - 단일 이진 비트
 IMM - 6-비트 즉시 피연산자
 R - fpw/txj
 N - 32-비트 정수
 PSR - 프로세서 선택 레지스터
 processor - 프로세서 선택 레지스터가 가리키는 프로세서

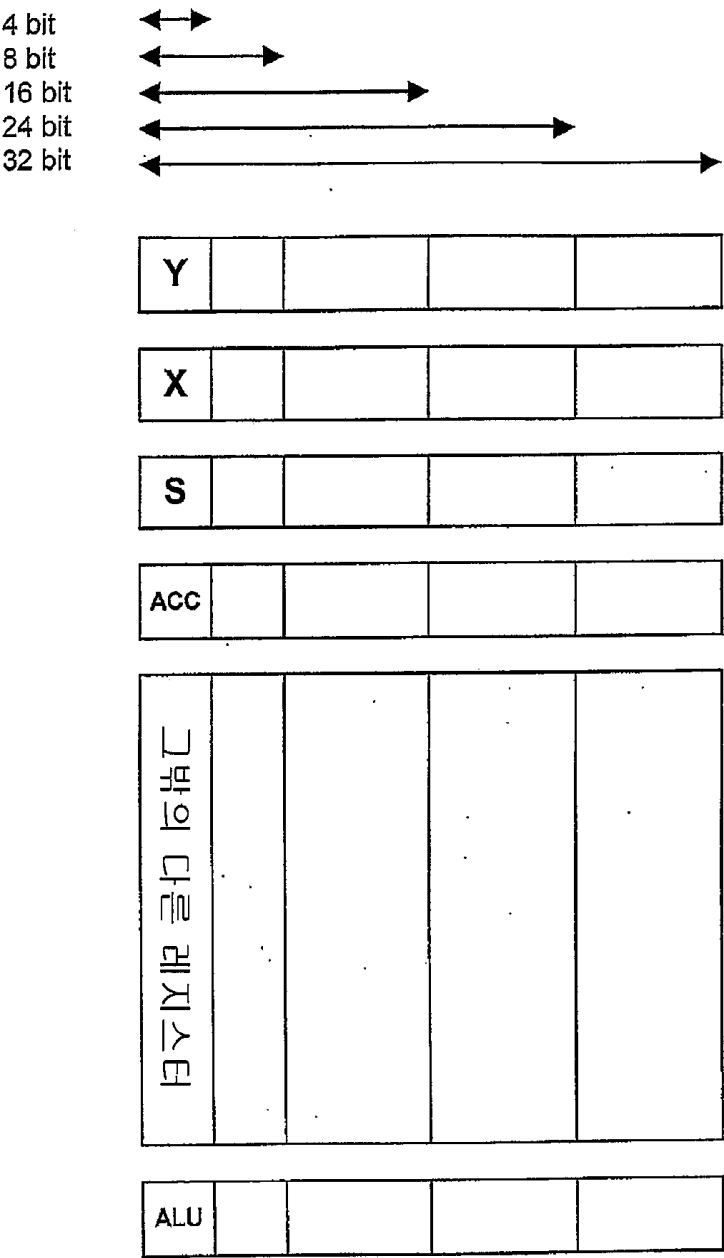
도면3

유효 주소

레지스터 명칭	유효 피연산자	사이클	특수 연산
00000 PC0	PC+3 for read, else PC	1	Write is conditional on C=0
00001 PC1	load 8-bit imm. read, else PC	1	
00010 ACC	ACC	1	
00011			
00100 S	S	1	
00101 (S)	data addressed by S	2	
00110 -(S)	(S) predecrement	2	
00111 (S)+	(S) postincrement	2	
01000 X	X	1	
01001 (X)	data addressed by X	2	
01010 -(X)	(X) predecrement	2	
01011 (X)+	(X) postincrement	2	
01100 Y	Y	1	
01101 (Y)	data addressed by Y	2	
01110 -(Y)	(Y) predecrement	2	
01111 (Y)+	(Y) postincrement	2	
10000 R10	Register 10	1	
10001 R11	Register 11	1	
10010 R12	Register 12	1	
10011 R13	Register 13	1	
10100 R14	Register 14	1	
10101 R15	Register 15	1	
10110 R16	Register 16	1	
10110 R17	Register 17	1	
10111 R18	Register 18	1	
11000 R19	Register 19	1	
11001 R1A	Register 1A	1	
11010 R1B	Register 1B	1	
11011 R1C	Register 1C	1	
11100 R1D	Register 1D	1	
11101 R1E	Register 1E	1	
11111 R1F	Register 1F	1	

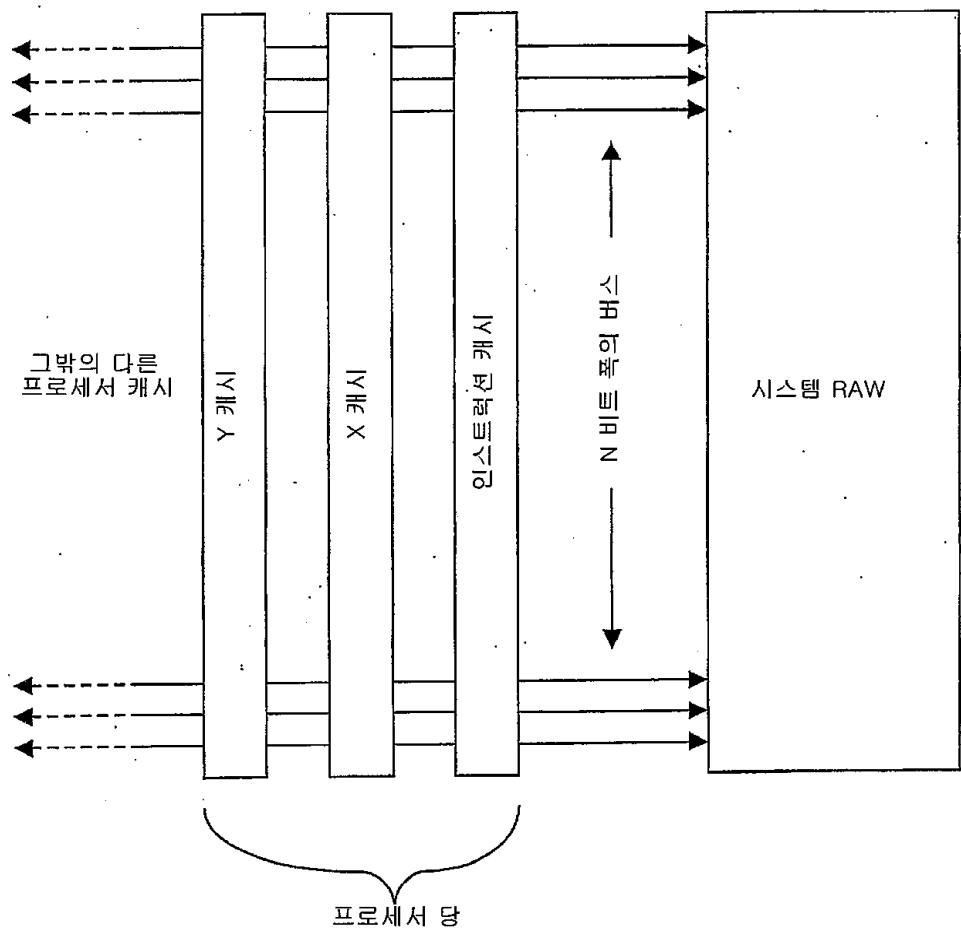
도면4

아키텍처 확장성

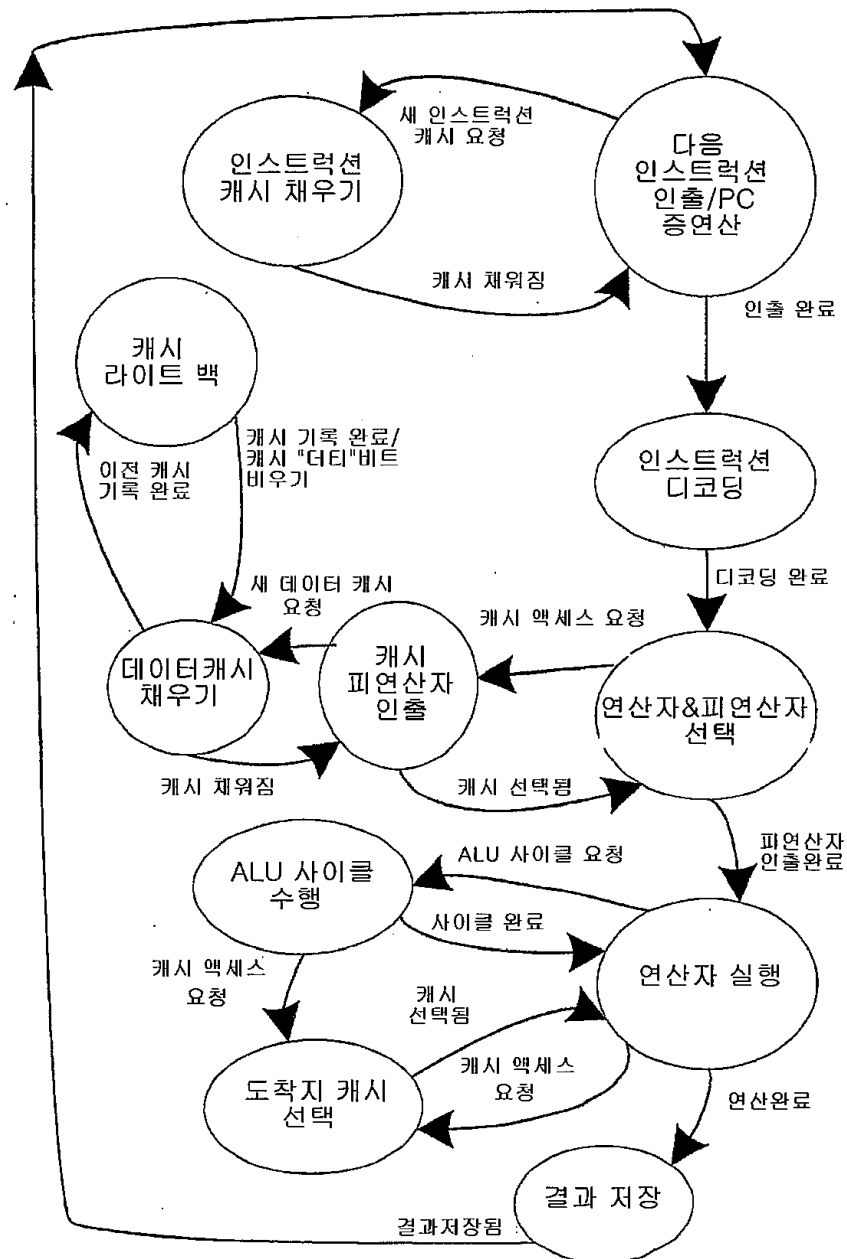


도면5

로컬 캐시

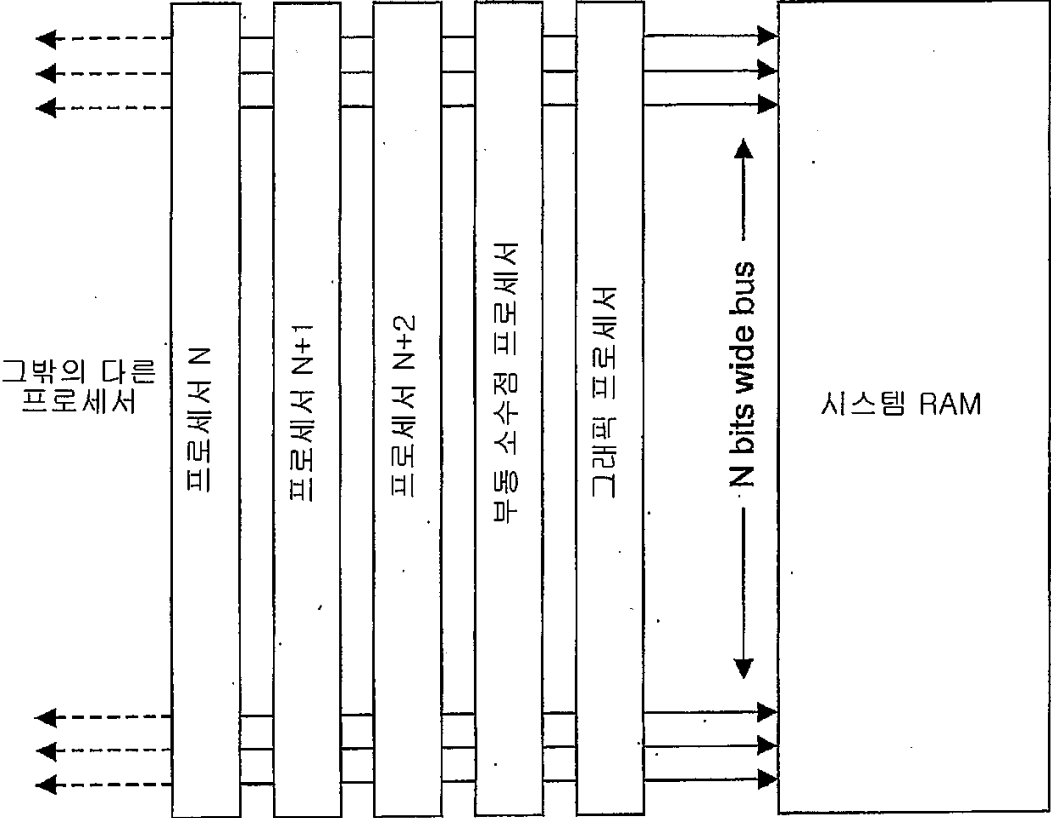


X 및 Y 캐시에 대한 캐시 관리 상태

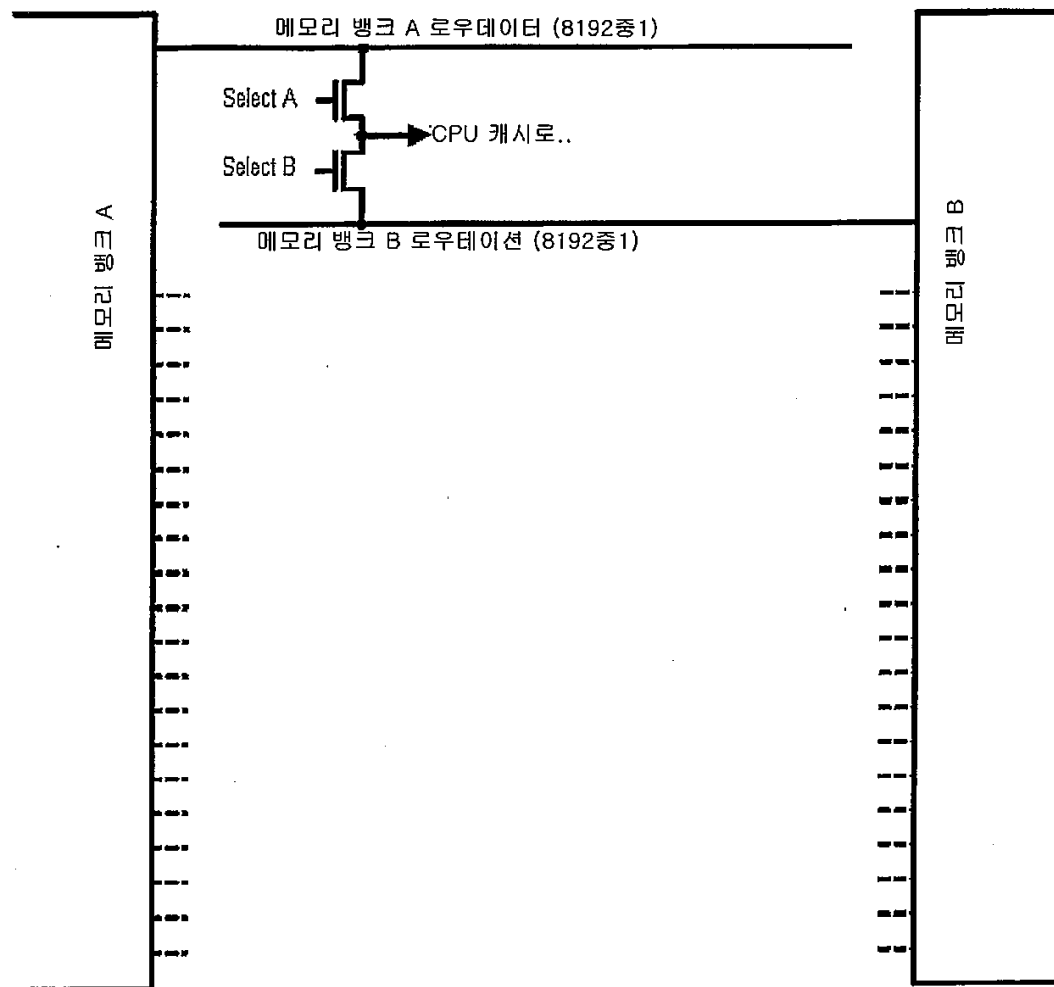


도면7a

추가 기능



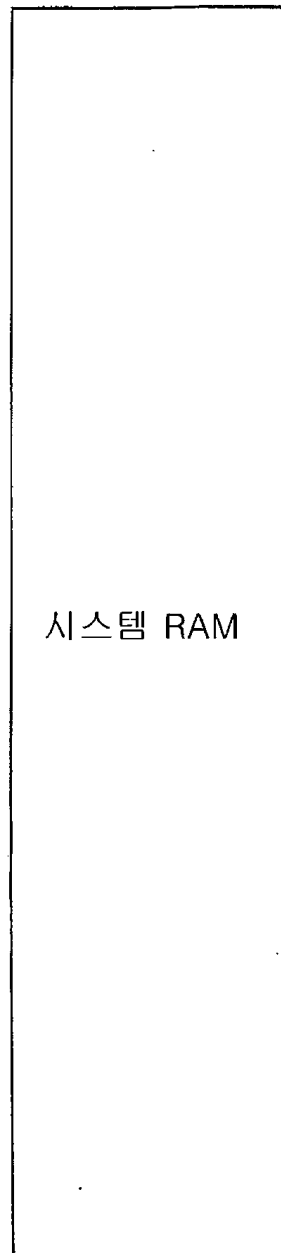
도면7b



메모리 뱅크 공유하기

도면8

시스템 메모리 맵



시스템 RAM

시스템 메모리를 항상 주소 지정
(모든 프로세서에 액세스 가능)

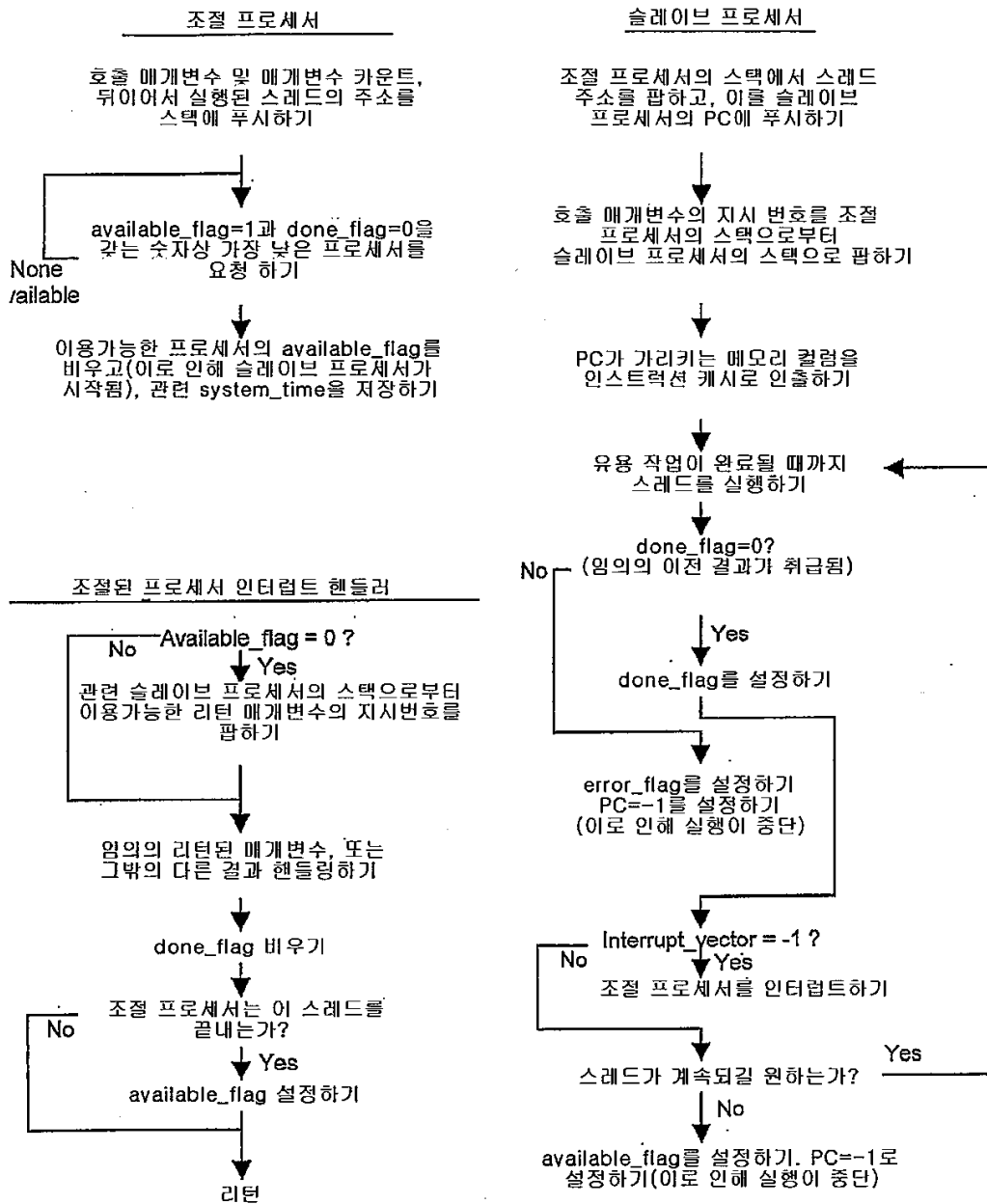
도면9

프로세서 가용도 테이블

Processor	available_flag	done_flag	Interrupt Vector	time_started	error_flag
3F					
3E					
3D					
3C					
3B					
3A					
30					
2F					
2E					
					
08					
07					
06					
05					
04					
03					
02					
01					

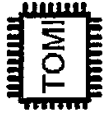
도면10

슬레이브 프로세서 할당 상태



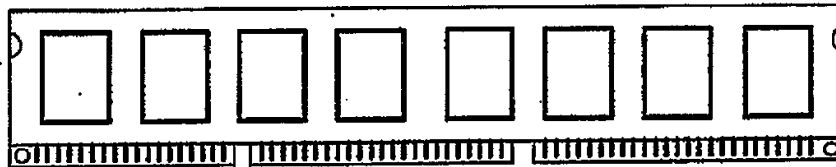
도면10a

DIMM에 장착된 TOMI 프로세서



64개의 32-비트 TOMI CPU+64Mbyte RAM
(64Mbyte DRAM을 가정)

단일 TOMI 패키지

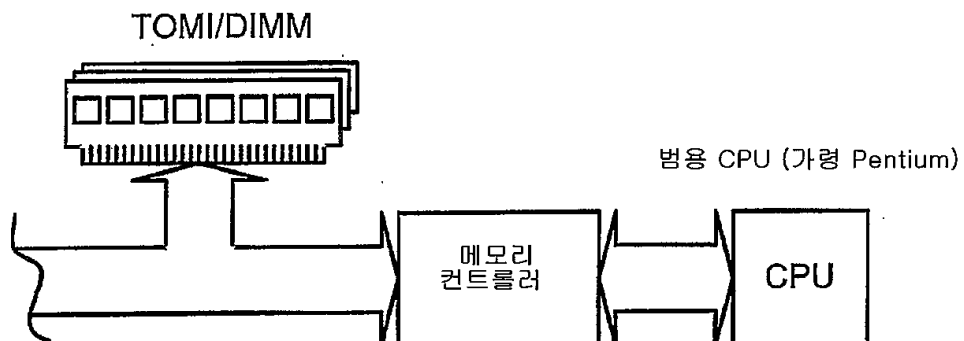


하나의 DIMM 상의 16개의 TOMI 패키지

1024 32-bit TOMI CPUs + 1Gbyte RAM

도면10b

범용 CPU를 포함하는 TOMI 시스템



도면10c

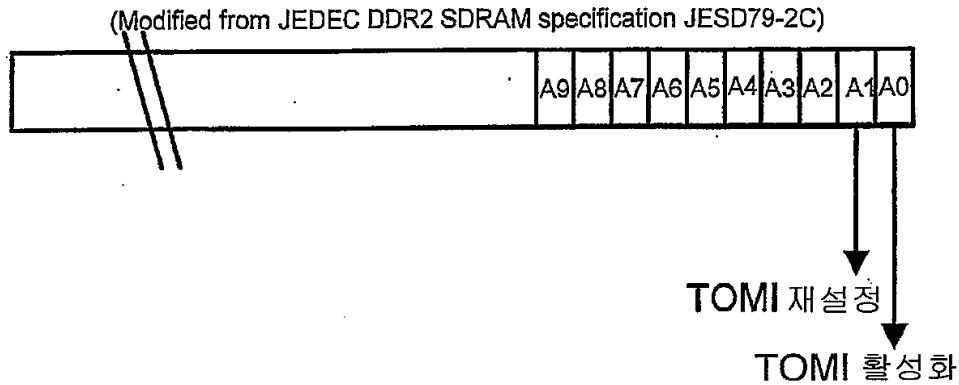
TOMI 프로세서 초기화

1. TOMI CPU가 메모리 모드 레지스터에 의해 활성화된다.
2. 재설정 구성에서,
 - 마스터 프로세서를 제외한 모든 프로세서가 중단되고
 - 마스터 프로세서의 클록이 느린 속도로 설정되며
 - 마스터 프로세서의 PC는 0으로 설정되고
 - 마스터 프로세서 인스트럭션 캐시에 POST(power-on-self-test)와 실행 시작 주소를 포함하는 주소 0이 로드된다.
3. 마스터 프로세서는 POST를 수행한다.
4. 마스터 프로세서는 자신의 최적 클록 속도를 결정한다.
5. 마스터 프로세서는
 - 각각의 슬레이브 프로세서를 개별적으로 시작시키고
 - 느린 속도로 테스트하며,
 - 프로세서를 중단시킨다.
6. 마스터 프로세서는
 - 각각의 슬레이브 프로세서를 개별적으로 시작시키고
 - 각각의 슬레이브 프로세서에 대한 최적의 클록 속도를 결정하며
 - 프로세서를 중단시킨다.
7. 마스터 프로세서는 실행 시작 주소에서 실행을 시작한다.

도면10d

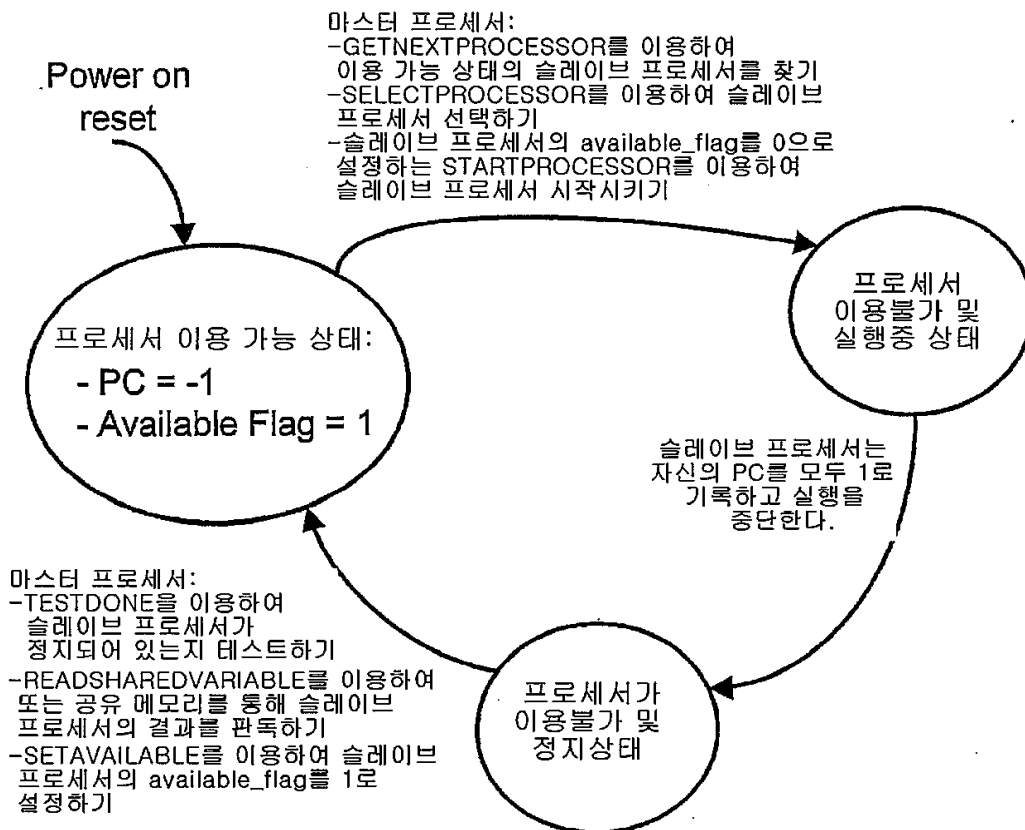
메모리 모드 레지스터의 사용

확장 모드 레지스터 3, EMR(3)

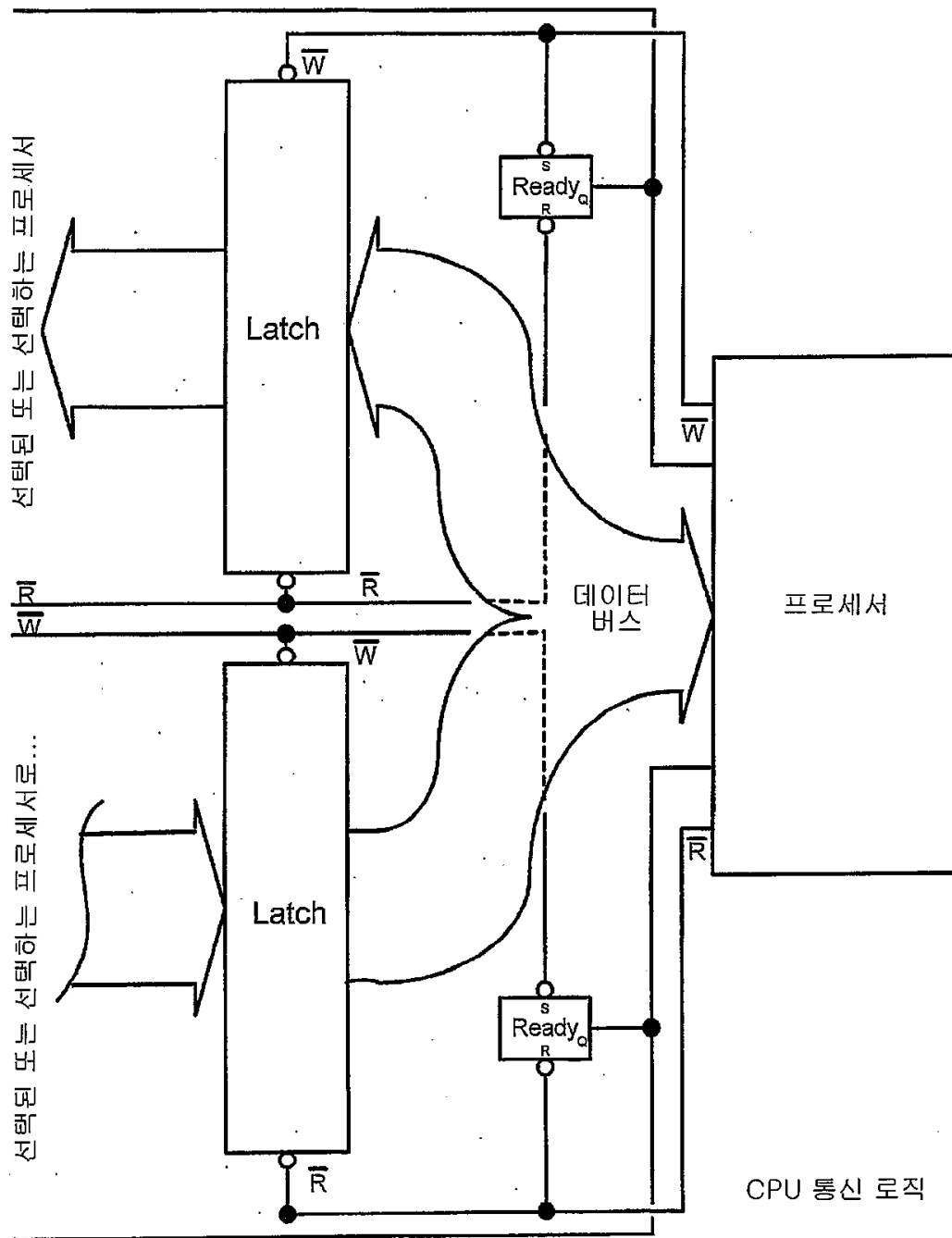


도면10e

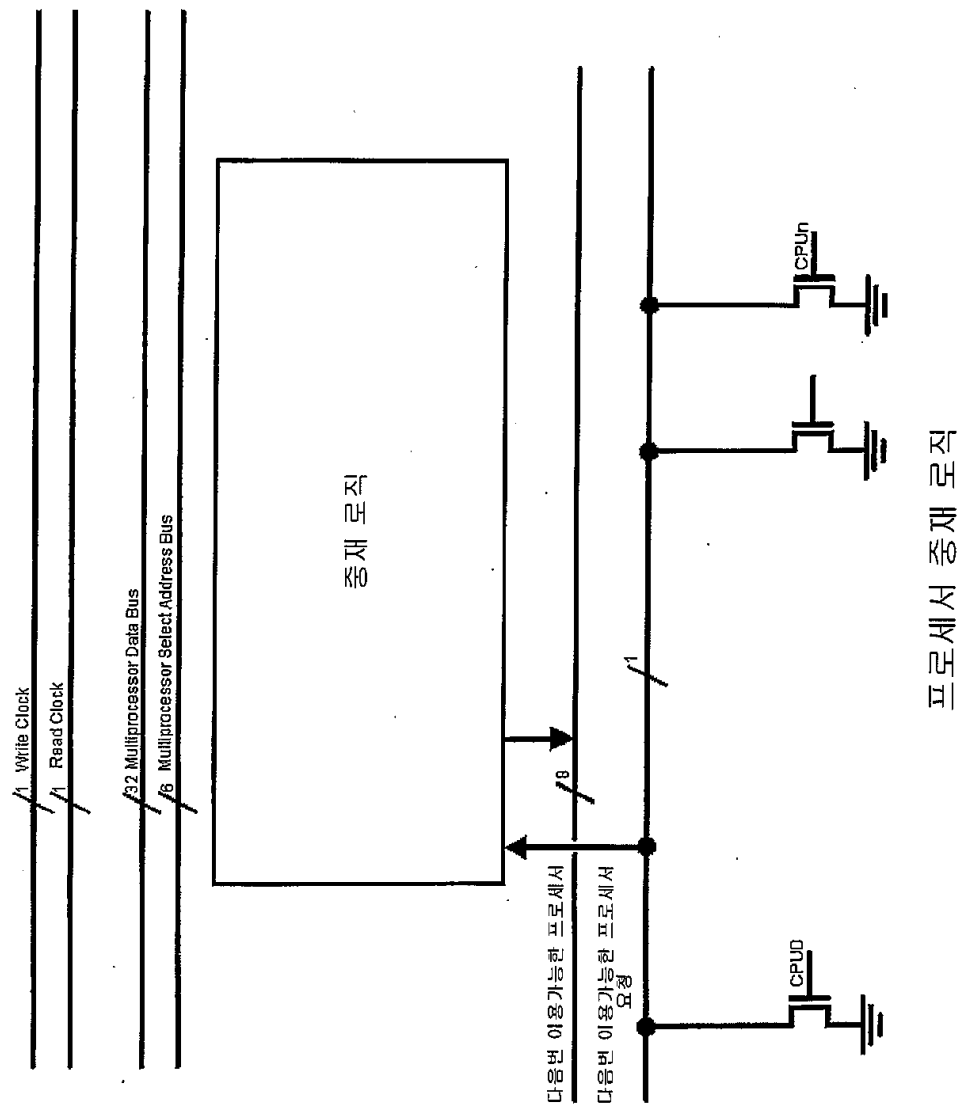
프로세서 가용도 상태 다이어그램 (단일 슬레이브 프로세서의 경우)



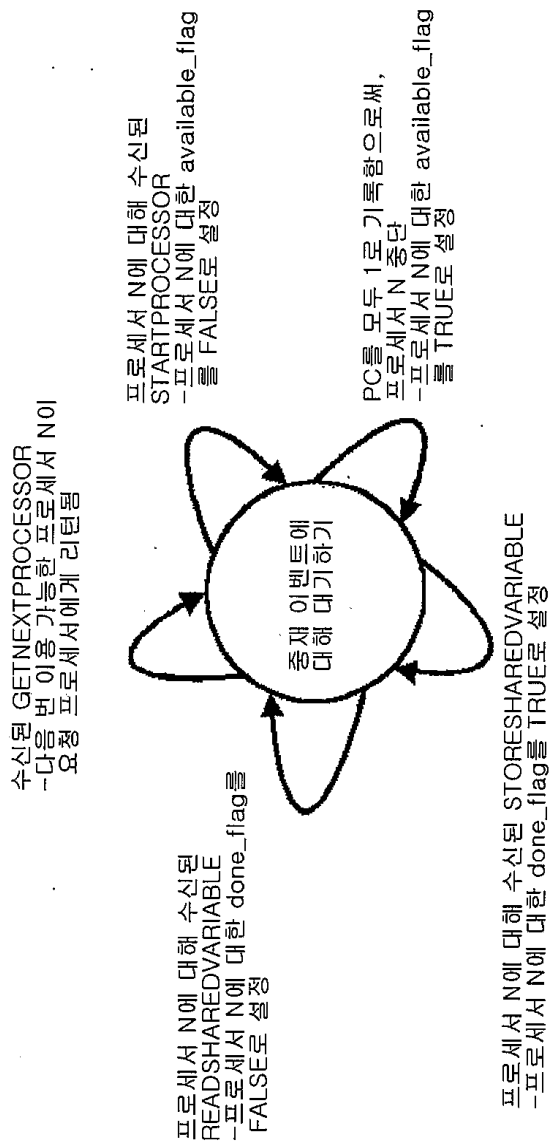
도면10f



도면10g



도면10h



프로세서 중재 상태 머신 이벤트

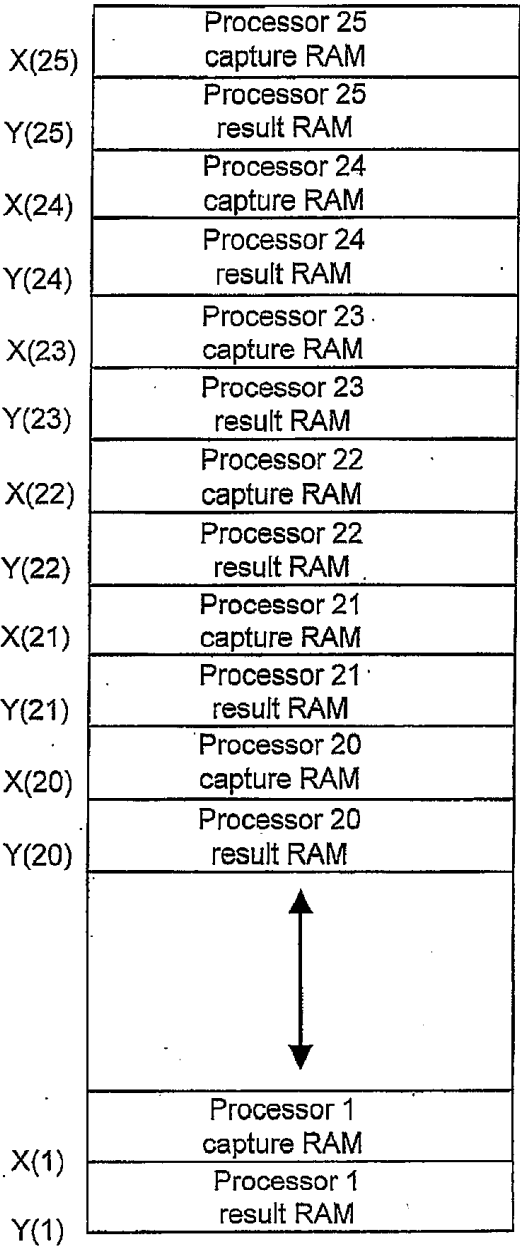
도면11

이미지맵의 분할 컴퓨테이션

processor 1	processor 2	processor 3	processor 4	processor 5	processor 6
processor 7	processor 8	processor 10	processor 11	processor 12	processor 13
processor 14	processor 15	processor 16	processor 17	processor 18	processor 19
processor 20	processor 21	processor 22	processor 23	processor 24	processor 25

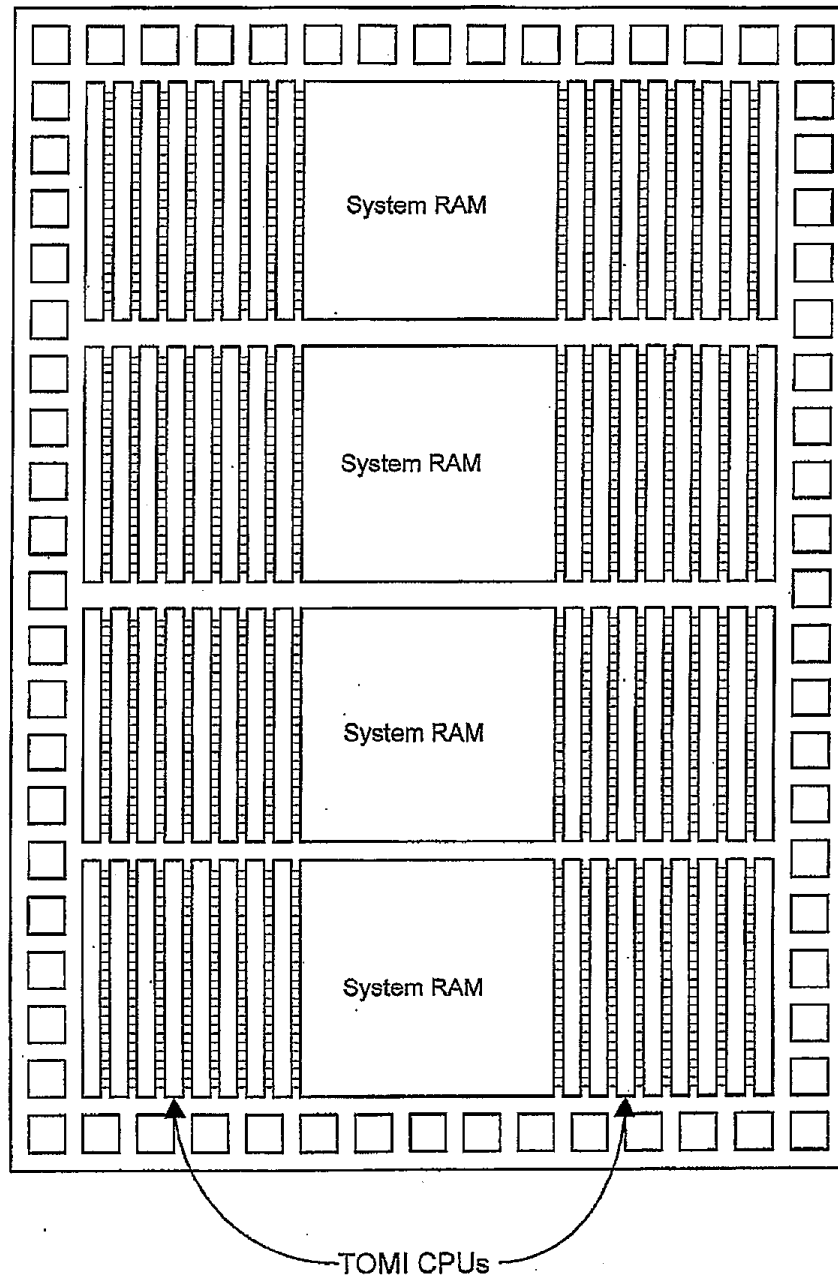
도면12

분할 컴퓨테이션 메모리 맵



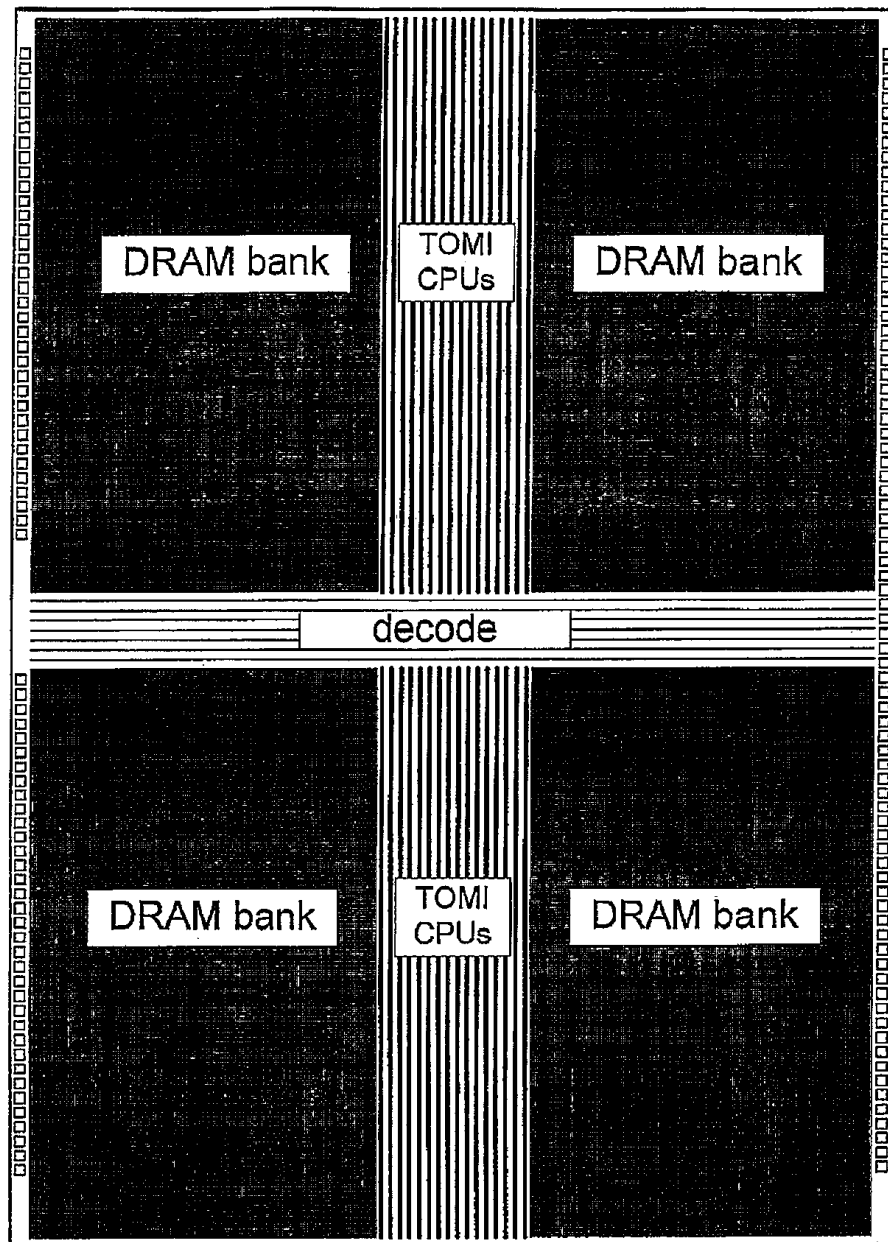
도면13a

64 프로세서 시스템의 가능한 평면도

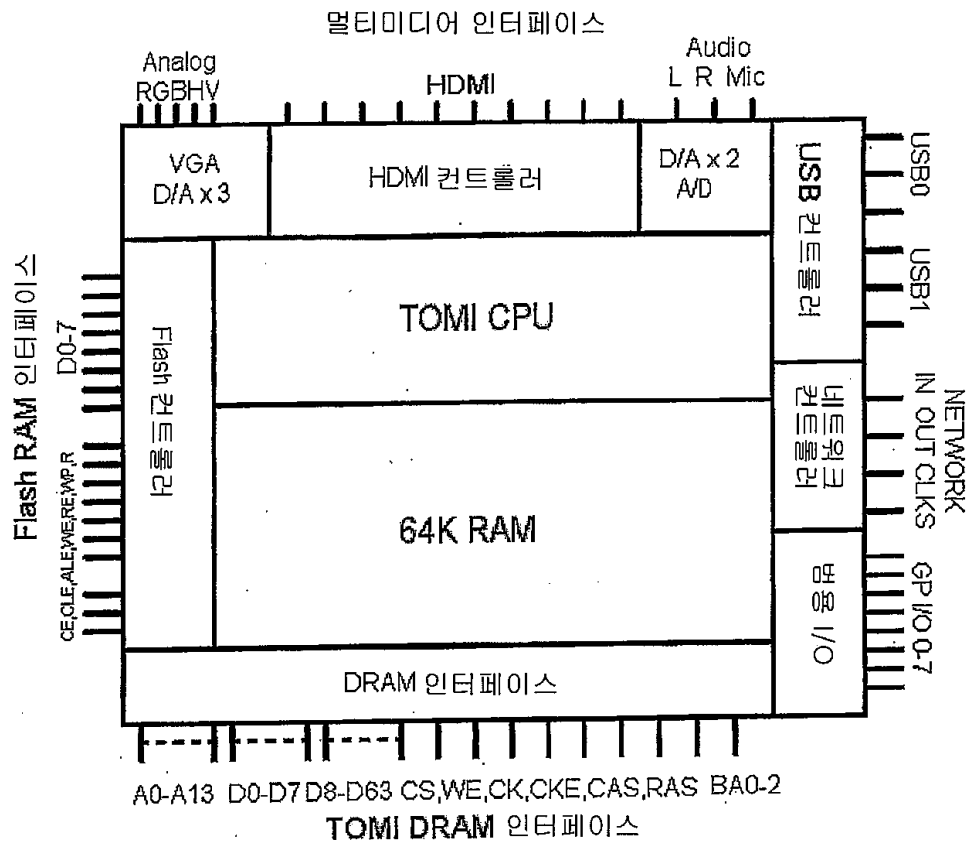


도면13b

또 다른 가능한 평면도

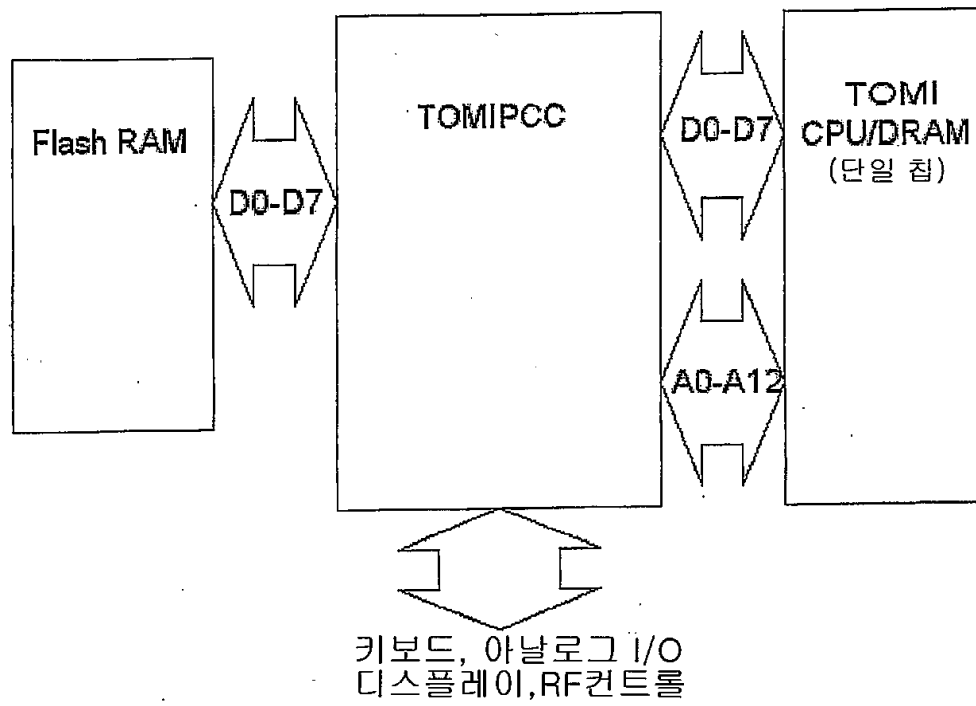


도면14a



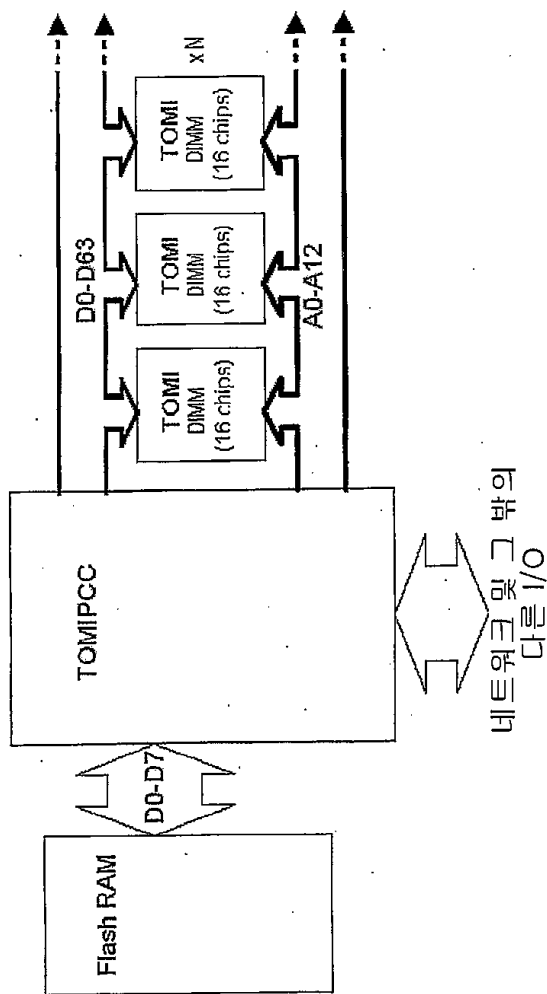
TOMI 주변 컨트롤러 칩

도면14b



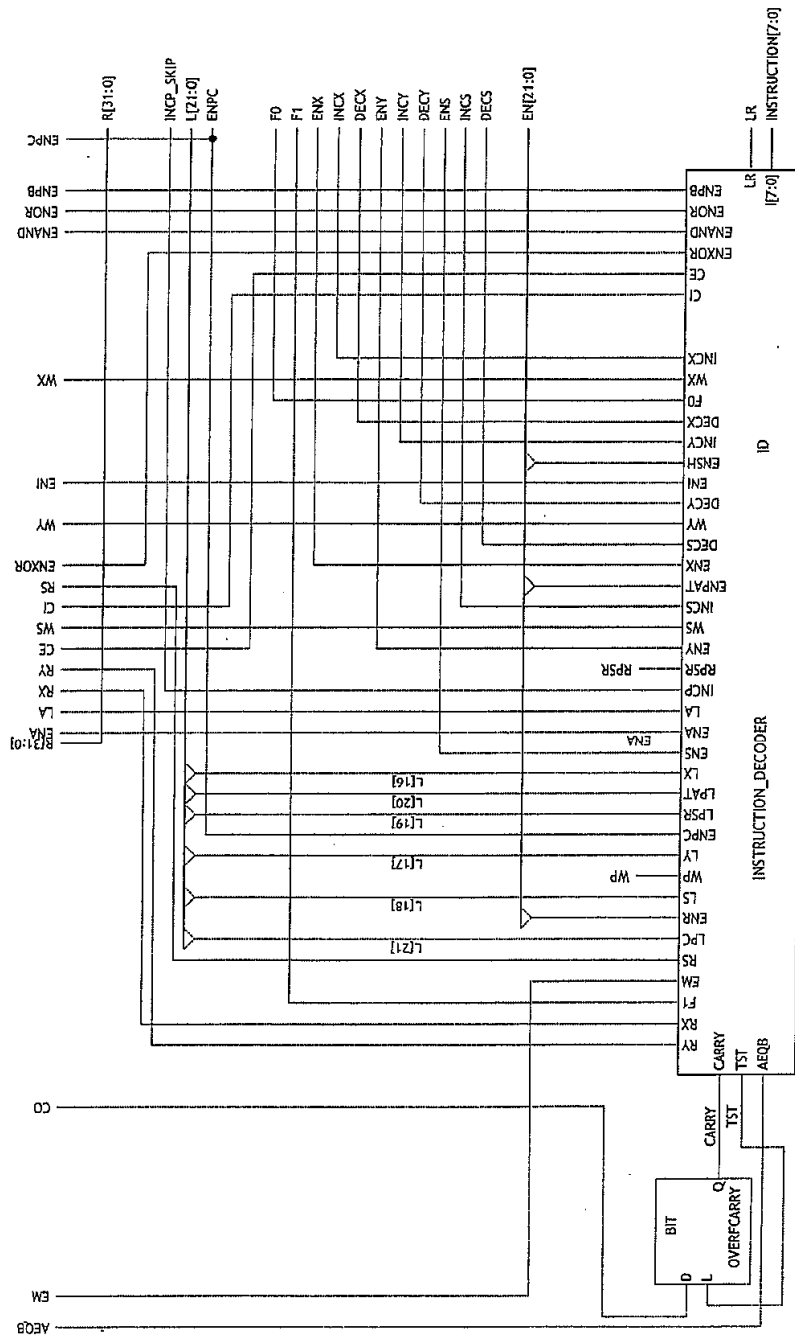
셀폰 언어 번역기

도면14c

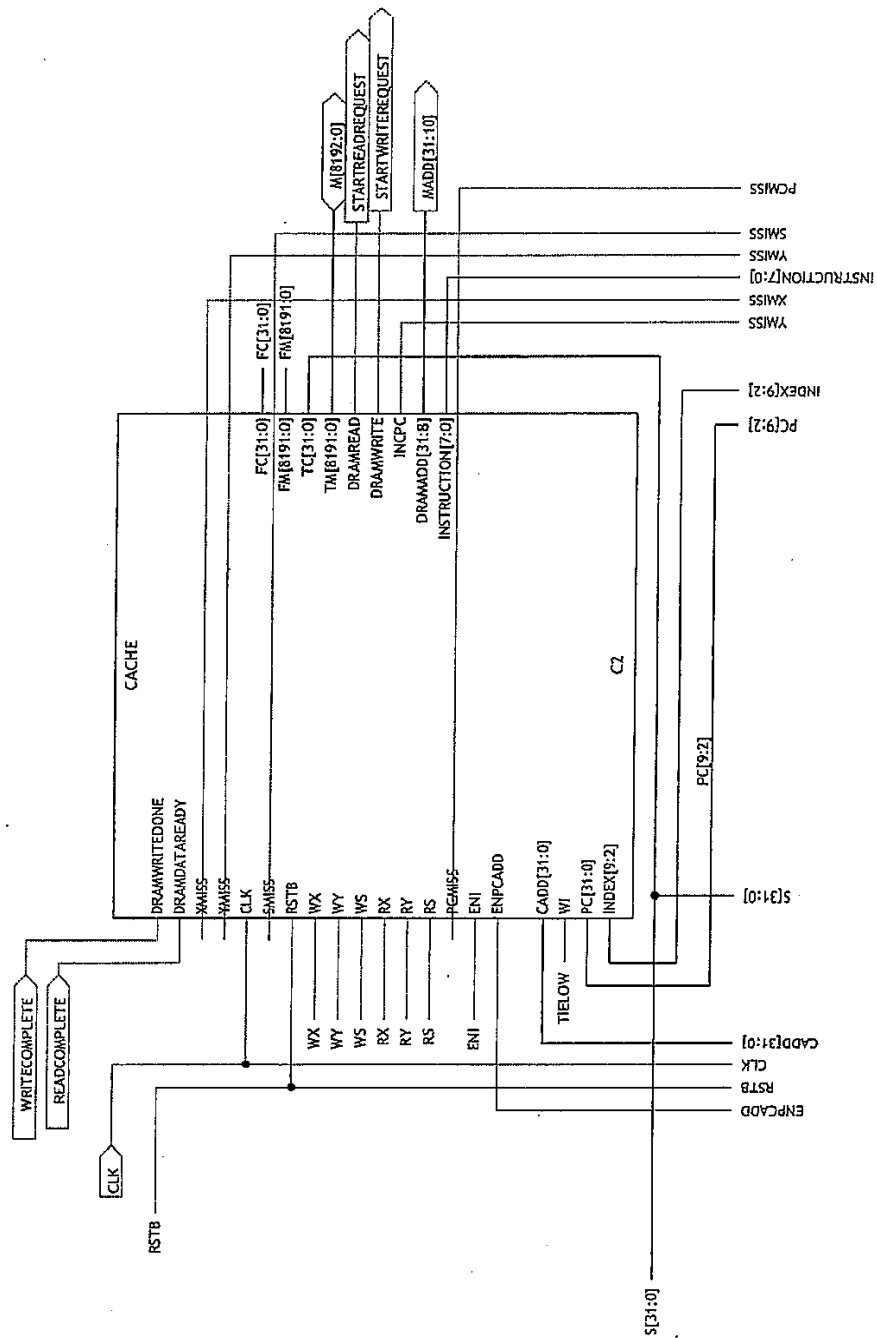


MCDB 데이터베이스 기기

도면15b



도면15c



도면15e

Signal Name	Direction	From/To	Description
RSTB	In	global	Reset Bar Input
CLK	In	global	System Clock Input
M8[9:1:0]	I/O	memory	Wide memory data bus (8K bits)
MA00[3:1:0]	Out	memory	high order memory bits addressing 8K bit sets of memory per address
STARTREADREQUEST	Out	memory	Request read data from DRAM bank.
STARTWRITEREQUEST	Out	memory	Request write data to DRAM bank.
WRITECOMPLETE	In	memory	Feedback from memory controller that operation is complete.
READCOMPLETE	In	memory	Feedback from memory controller that operation is complete.
To be determined:			
WP	Out	arbiter	Write Processor Indexed by Processor Select Register
WPSR	Out	arbiter	Read Processor Select Register
PSR[5:0]	I/O	PSR	Processor Select Data Bus
Signal Name	From Block	To Block	
ENOR	INSTRUCTION_DECODER	ALU	Enable OR function between A and B inputs, result at S
ENPB	INSTRUCTION_DECODER	ALU	Enable pass B function from B to S
ENAND	INSTRUCTION_DECODER	ALU	Enable AND function between A and B inputs, result at S
ENXOR	INSTRUCTION_DECODER	ALU	Enable XOR function between A and B inputs, result at S
CI	INSTRUCTION_DECODER	ALU	ALU Carry in input used for addition, increment, decrement instructions.
CE	INSTRUCTION_DECODER	ALU	Carry enable turns on carry lookahead adder unit for addition inc, dec.
CO	INSTRUCTION_DECODER	ALU	Overflow result or Carry Out from ALU for operations using Carry
A[31:0]	ACCUMULATOR	ALU	Accumulator Value used as one ALU input.
B[31:0]	REGISTERS or CACHE	ALU	Register and Cache value used as the second ALU input.
S[31:0]	ALU	Multiple locations,	ALU Output Sum or Result
EM	INSTRUCTION_DECODER	Optional Multiplier	Enable Multiplier Output
CARRY	OVERCARRY	INSTRUCTION_DECODE	Indicates if a carry occurred after the last test "TST" instruction.
TST	INSTRUCTION_DECODER	OVERCARRY	Latches the carry result to a register for use by a later branch instruction.
AEQB	COMPARE	INSTRUCTION_DECODE	Instruction decode input indicating comparison result between A and B
WX	INSTRUCTION_DECODER	CACHE	Write X Register
WY	INSTRUCTION_DECODER	CACHE	Write Y Register
WS	INSTRUCTION_DECODER	CACHE	Read X Register
RX	INSTRUCTION_DECODER	CACHE	Read Y Register
RY	INSTRUCTION_DECODER	CACHE	Enable Index
ENI	INSTRUCTION_DECODER	CACHE	Allows the Accumulator to drive both ALU inputs at the same time.
FNA	INSTRUCTION_DECODER	TRISTATE32	This signal would allow instruction cache to write back to memory. Not useful.
TIELOW TO WI	UNUSED	CACHE	Program Counter
PC[31:0]	REGISTERS	CACHE	Index Value of 32 bit set within 8Kbits.
INDEX[9:2]	REGISTERS	CACHE	Indicates a X Cache miss and will generate a Read or Write Memory request.
YMISS	REGISTERS	CACHE	Indicates a Y Cache miss and will generate a Read or Write Memory request.
YMISS	REGISTERS	CACHE	Indicates a Stack Cache miss and will generate a Read or Write Memory request.
SMISS	REGISTERS	CACHE	Indicates a Program Counter Cache miss and will generate a Read Memory request.
PCMISS	REGISTERS	CACHE	Used to prevent PC from counting while waiting for a cache miss to resolve.
INCPC	CACHE	REGISTERS	Used to increment PC twice in order to skip an instruction.
INCP_SKIP	INSTRUCTION_DECODER	REGISTERS	Load control lines for all registers.
L[21:0]	INSTRUCTION_DECODER	REGISTERS	Enable Program Counter Output
ENPC	REGISTERS	CACHE	Force 0000 hex. Can be used with F1 to force 8000 hex.
F0	REGISTERS	CACHE	Force 0001 hex. Can be used with F0 to force 8000 hex.
F1	REGISTERS	CACHE	Enable X Register Out
ENX	REGISTERS	CACHE	Increment X
INCX	REGISTERS	CACHE	Decrement X
DECX	REGISTERS	CACHE	Enable Y Register Out
ENY	REGISTERS	CACHE	Increment Y
INCY	REGISTERS	CACHE	Decrement Y
DECY	REGISTERS	CACHE	Enable Stack Pointer
ENS	REGISTERS	CACHE	Increment Stack Pointer
INCS	REGISTERS	CACHE	Decrement Stack Pointer
DECS	REGISTERS	CACHE	Load Data Register
LR	INSTRUCTION_DECODER	REGISTER_DECODE	Eight Bit TONI instruction
INSTRUCTION[7:0]	CACHE	INSTRUCTION_DECODE	